

AK High-Dimensional Projection Structural Theory

Version 19.0.0: Collapse Structures, Group Simplification, and Persistent Projection Geometry

Atsushi Kobayashi

April 2026

1 Chapter 1: Scope, Claim Discipline, and v19 Metric-Repair Contract

Scope Box (v19 Core guard-rails; threshold-gap metric repair and audit-ready).

All statements in this chapter, and all subsequent statements labeled as *Core*, are made under the following v19.0.0 guard-rails.

- **Release scope.** AK-HPDST v19.0.0 is the *Threshold-Gap Metric Repair Release*. Its purpose is not to enlarge the Core, but to repair the metric claim attached to hard positive-threshold barcode deletion and to make the certification architecture explicitly typed and auditable. Under the standard bottleneck metric, hard positive-threshold deletion is not globally non-expansive. The former global non-expansiveness claim is rejected and replaced by two-sided threshold-gap-safe stability.
- **Core baseline and downstream discipline.** Chapters 1–8 and Appendix A determine the v19.0.0 Core baseline. Later Foundation, Technical Extension, Problem Interface, Search/Platform, Claim Register, Micro-Theorem, Toy Bridge, and Companion materials may register, elaborate, instantiate, transport, or implement this Core. They do not silently strengthen or alter it.
- **Constructible one-parameter persistence over a field.** We work in

$$\text{Pers}_k^{\text{cons}} \subset [(\mathbb{R}, \leq), \text{Vect}_k]$$

over a field k . Filtered limits and colimits are computed objectwise and are consumed only after constructibility or an explicitly declared repaired-profile regime has been verified.

- **Standard barcode metric convention.** Barcode comparisons in the Core use the standard bottleneck distance d_B , unless another metric is explicitly declared outside the Core. Infinite bars have threshold length $+\infty$. In finite-cost matchings, infinite bars match only infinite bars, with cost

$$\text{cost}([a, \infty), [a', \infty)) = |a - a'|.$$

A finite bar does not match an infinite bar at finite cost, and an infinite bar does not match the diagonal.

- **Hard finite-threshold deletion.** For $\tau > 0$, the Core uses the barcode-level operation

$$T_\tau^{\text{bar}},$$

which deletes finite bars of length at most τ , retains finite bars of length greater than τ , and retains all infinite bars. It is endpoint-independent, idempotent, infinite-bar preserving, and threshold-monotone. These algebraic properties do not imply global non-expansiveness, exactness, a Serre localization, or a global functor on all morphisms.

- **Threshold-gap-safe replacement for global stability.** If

$$d_B(B, C) \leq \varepsilon$$

and the finite bars of both B and C avoid the closed band

$$[\tau - \varepsilon, \tau + \varepsilon],$$

then

$$d_B(T_\tau^{\text{bar}} B, T_\tau^{\text{bar}} C) \leq \varepsilon.$$

The hypothesis is two-sided, and the half-width ε is sharp for this two-sided certificate.

- **Windowed evaluation first.** Windows are right-open intervals

$$W = [u, u'), \quad u' \in \mathbb{R} \cup \{\infty\}.$$

The fixed evaluation order is

$$\text{Eval}_{i, W, \tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

No unconditional commutation

$$W_W T_\tau^{\text{bar}} \cong T_\tau^{\text{bar}} W_W$$

belongs to the Core. On a bounded window, an unwindowed infinite bar may become a short finite clipped bar and must therefore be inspected by any crop-collapse criterion.

- **Repaired evaluation, exact comparison, and metric certification.** A repaired evaluation is definitionally meaningful once the displayed barcode expression exists. A threshold-gap certificate is mandatory whenever an inference uses a metric perturbation, overlap transport, Restart transport, representative comparison by a metric bound, or tower comparison by a metric bound through hard deletion. A separately proved exact equality or profile isomorphism may be used as an exact comparison, but it is not automatically a positive-radius robustness certificate.
- **Typed scalarization.** Metric ledger values live in a declared commutative quantale

$$(\mathbb{V}, \oplus, 0_{\mathbb{V}}, \preceq).$$

Any comparison with real-valued threshold clearance passes through a declared monotone, subadditive scalarization

$$\text{val}_B : \mathbb{V} \longrightarrow [0, +\infty].$$

The shorthand

$$\Sigma\delta < \text{gap}_\tau$$

means

$$\text{val}_B(\mathbf{d}_{\text{met}}) < \text{gap}_\tau(\mathfrak{B}),$$

together with evidence that the scalarized value upper-bounds the actual bottleneck discrepancy consumed by the proof.

- **Threshold-clearance discipline.** For a barcode B ,

$$\text{clear}_\tau(B) := \inf_{I \in B_{\text{fin}}} |\ell(I) - \tau|,$$

with $\text{clear}_\tau(B) = +\infty$ if B has no finite bars. For a finite comparison family,

$$\text{gap}_\tau(B_1, \dots, B_m) := \min_j \text{clear}_\tau(B_j).$$

Thus $\text{gap}_\tau = \text{clear}_\tau$ on a single profile; no factor $1/2$ is present. The measurement is taken on

$$B_{i,W}(F) := W_W \mathbf{P}_i(F)$$

after windowing and before hard deletion.

- **Non-scalar obligations.** Representative existence, actual comparison morphisms, realization, eligibility, terminal tameness, threshold reconciliation, higher Čech coherence, Restart transition certification, claim status, manifest consistency, and artifact identity are not numerical ledger components and cannot be repaired by numerical slack.
- **Representative discipline.** A filtered representative

$$C_{\tau,W}F$$

is not a global collapse functor by default. It represents the repaired profile in declared monitored degrees. Persistence-profile equivalence does not automatically imply chain isomorphism, chain-homotopy equivalence, or derived equivalence. A bottleneck matching is not automatically a persistence morphism.

- **Bridge policy.** The forward bridge

$$\text{PH}_1 \implies \text{Ext}^1$$

belongs to the Core only in the eligible realization regime and only in the proved direction. No unconditional reverse implication or global equivalence is claimed.

- **Type IV discipline and ν -notation.** Type IV diagnostics require an actual comparison artifact

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty).$$

Every run records exactly one of

$$\text{not_invoked}, \quad \text{undefined}, \quad \text{certified_zero}, \quad \text{obstructed}.$$

Numerical diagnostics

$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}})$

are reported only in invoked, well-typed cases. No numerical pair is imputed when the status is undefined or not_invoked; in particular, undefined is not encoded as (0, 0). The legacy Type IV symbols $u_{i,W,\tau}$ and u_{Collapse} are deprecated; $\nu_{i,W,\tau}$ and ν_{Collapse} are canonical.

- **Audit-first doctrine.** Search scores, scalar potentials, AI confidence values, rankings, and advisory indicators may guide proposal generation. They do not change a Core verdict, establish a theorem, or repair a missing artifact.
- **Manifest discipline.** A certified proof object carries immutable scope, metric and threshold policies, evaluation order, threshold-gap records, representative and eligibility records, ledgers and scalarizations, non-scalar obligations, Type IV scope and artifacts, typed gate verdicts, claim references, repair ancestry, and artifact identities. Manifest completeness is necessary for certification but is not sufficient for mathematical truth. Chapter 7 is canonical for the v19 manifest. Appendix F elaborates typed obligation and readiness semantics; Appendix G elaborates the serialization-neutral record schema.
- **Claim-register, micro-theorem, toy-bridge, and NBT boundary.** Appendix CR-v19 governs claim use, Appendix CM-v19 records reusable Core micro-theorems, Appendix TB records deliberately restricted toy bridge theorems, and Appendix NBT is v19.1 candidate material unless promoted by a successor release. Registration is not proof, and none of these appendices silently enlarges the Core into an external-domain theorem.

Accordingly, this chapter fixes the admissible language for v19.0.0 and the boundaries between theorem, conditional theorem, operational policy, interface statement, bridge program, toy bridge, Search artifact, Spec, and explicit non-claim.

1.1 Claim taxonomy and document discipline

Definition 1.1 (Claim Taxonomy). Every mathematically meaningful statement is assigned a document role. The roles are:

- (i) **Core definition.** Typed language internal to the Core; not by itself an existence, uniqueness, exactness, or external-domain theorem.
- (ii) **Core theorem.** A proved statement inside the standing v19 Core regime, valid only under its complete hypotheses.
- (iii) **Core lemma.** A proved supporting statement used as an internal step.
- (iv) **Core micro-theorem.** A small reusable Core theorem recorded in Appendix CM-v19 at its exact strength.
- (v) **Metric-repair theorem.** A theorem replacing, constraining, or rejecting an earlier metric statement.
- (vi) **Operational policy.** A binding protocol for admissible certification; not an external mathematical theorem.
- (vii) **Interface definition.** A definition preparing external data for Core readability; it does not prove realization faithfulness or external transport.

- (viii) **Interface theorem.** A proved conditional theorem connecting declared external data and a Core-visible statement, or a Core certificate and an external conclusion.
- (ix) **Bridge candidate.** A precisely typed but not fully proved cross-layer implication.
- (x) **Bridge program.** A structured proof program for promoting a Bridge candidate.
- (xi) **Bridge theorem.** A proved cross-layer theorem with declared source and target semantics, realization, artifacts, defects, hypotheses, and failure modes.
- (xii) **Toy bridge theorem.** A theorem proved only over a deliberately small toy source category and recorded in Appendix TB.
- (xiii) **Search artifact.** A proposal, map, ranking, heuristic, or AI-assisted output; never by itself proof.
- (xiv) **Companion template.** An operational or illustrative template; not a theorem or certificate.
- (xv) **Spec.** A speculative, schematic, conjectural, programmatic, or design-level statement.
- (xvi) **v19.1 candidate.** Material intentionally excluded from v19.0.0 theorem claims.
- (xvii) **Non-claim.** A statement explicitly excluded from the asserted content.
- (xviii) **Deprecated or rejected claim.** A historical or proposed claim that is no longer admissible because it is false, unsupported, overstrong, ill-typed, or incompatible with v19 guard-rails.

Definition 1.2 (Three-axis claim typing). Every registered statement is typed along three independent axes:

- (i) its document role from Definition 1.1;
- (ii) its Claim Register evidence status, chosen from

`proved, conditional, operational, spec, deprecated, rejected, unknown;`
- (iii) its invocation role, chosen from

`theorem_evidence, operational_rule, specification_target, provenance_only.`

The admissible status–role combinations are fixed before verdict computation:

- (a) theorem evidence requires status `proved`, or status `conditional` with every registered hypothesis discharged;
- (b) an operational rule uses an `operational` item only as a typed procedure;
- (c) a specification target uses a `spec` item only as a schema, target, or acceptance criterion;
- (d) provenance-only use contributes no passing theorem clause.

The three axes are not interchangeable. A document role does not prove a statement, an evidence status does not change its mathematical type, and an invocation role cannot promote a weaker item to theorem evidence.

Remark 1.3 (Interpretive rule). The taxonomy separates mathematical, operational, interface, bridge, Search, and evidentiary authority. Whenever a statement’s role, status, scope, or hypotheses are ambiguous, the weaker safe interpretation prevails.

Remark 1.4 (Claim-register sovereignty). A claim may be used only at the strength jointly supported by its underlying proof or artifact and its Appendix CR-v19 entry. If the source and the register disagree, the weaker classification governs until the inconsistency is repaired.

Remark 1.5 (Registration is not proof). A statement does not become proved merely because it receives a ClaimID. A proved entry must point to an exact proof location; a conditional entry must record all hypotheses; an operational entry must not be read as an external theorem.

Remark 1.6 (Relation to Appendix CR-v19). Appendix CR-v19 records Claim IDs, exact statements, document roles, evidence statuses, source labels, hypotheses, permitted uses, prohibited stronger readings, proof locations, artifact references, and repair ancestry. It governs use but does not create proof by registration.

Remark 1.7 (Relation to Appendix CM-v19). Appendix CM-v19 records reusable Core micro-theorems after metric repair. They are valid only under their registered hypotheses and do not imply external-domain conclusions by themselves.

Remark 1.8 (Relation to Appendix TB). Appendix TB records theorem-level toy bridge models over deliberately small source categories. A toy bridge does not replace a problem-specific Bridge Theorem.

Remark 1.9 (Relation to Appendix NBT). Appendix NBT, if present, is v19.1 candidate material. It may organize future finite-length module tower work but may not discharge a v19.0.0 theorem obligation.

1.2 v19 Metric-Repaired Unified Collapse Contract

Theorem 1.10 (Threshold-gap-safe stability under the standard bottleneck metric). *Let B and C be finite barcodes, possibly including infinite bars, under the standard bottleneck metric and the v19 infinite-bar convention. Let $\tau > 0$, $\varepsilon \geq 0$, and assume*

$$d_B(B, C) \leq \varepsilon.$$

Assume that no finite bar of either barcode has length in

$$[\tau - \varepsilon, \tau + \varepsilon].$$

Then

$$d_B(T_\tau^{\text{bar}} B, T_\tau^{\text{bar}} C) \leq \varepsilon.$$

This is the central v19 metric-repair theorem replacing the rejected global non-expansiveness claim.

Remark 1.11 (Proof location for the metric-repair theorem). The proof is supplied in Chapter 2, Theorem 2.33, and independently in Appendix A, Theorem 1.33 (the metric-repair theorem). Chapter 1 fixes the exact statement later chapters may consume.

Definition 1.12 (v19 Metric-Repaired Unified Collapse Contract: mathematical data). A v19 metric-repaired UCC mathematical contract consists of the following clauses.

(i) **Hard deletion.**

$$T_\tau^{\text{bar}}(B) = \{I \in B \mid \ell(I) > \tau \text{ or } \ell(I) = +\infty\}.$$

(ii) **Barcode algebra.** The operation is endpoint-independent and idempotent, and

$$T_{\sigma}^{\text{bar}} T_{\tau}^{\text{bar}}(B) = T_{\max\{\sigma, \tau\}}^{\text{bar}}(B).$$

(iii) **Infinite-bar invariance.** If $I = [a, \infty)$, then

$$T_{\tau}^{\text{bar}}(I) = I.$$

(iv) **Rejected global non-expansiveness.** No inequality

$$d_B(T_{\tau}^{\text{bar}} B, T_{\tau}^{\text{bar}} C) \leq d_B(B, C)$$

is asserted for arbitrary barcodes.

(v) **Windowed repaired evaluation.**

$$\text{Eval}_{i, W, \tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

(vi) **Metric-use discipline.** Threshold-gap evidence is mandatory whenever a metric discrepancy is transported through hard deletion. Exact comparison uses a separately proved equality or isomorphism and is not automatically a positive-radius robustness statement.

(vii) **No unconditional crop-collapse commutation.** Such commutation may be used only under the exact criterion of Appendix A or a separately declared exact-admissible theorem.

(viii) **Representative compatibility.** If $C_{\tau, W}F$ is used, then in every monitored degree

$$\mathbf{P}_i(C_{\tau, W}F) \cong \text{Eval}_{i, W, \tau}(F),$$

at the explicitly declared equivalence strength.

(ix) **Non-scalar obligations.** Representative existence, actual morphisms, eligibility, terminal tameness, higher coherence, claim status, and artifact identity remain independent proof obligations.

(x) **Exact-admissible strengthening.** Strict functoriality, exactness, or canonical tower comparison may be used only under a separately proved and registered strengthening.

Policy 1 (v19 Metric-Repaired Unified Collapse Contract: audit policy). *A certified run declares at least:*

(i) *the barcode metric and infinite-bar convention;*

(ii) *the windows, thresholds, monitored degrees, and deletion convention;*

(iii) *the collapse policy*

barcode_level, tau_split, exact_admissible;

(iv) *the evaluation order;*

(v) *exactly one comparison mode for every consumed comparison:*

exact, metric_gap_certified, not_compared;

- (vi) in mode *exact*, the actual equality or persistence-profile isomorphism, its source and target, scope, equivalence strength, and immutable proof artifact;
- (vii) in mode *metric_gap_certified*, every required crop, window-edge, discrepancy, scalarization, and two-sided threshold-gap record;
- (viii) mode *not_compared* only when the governing claim consumes no comparison inference; it cannot discharge an invoked overlap, transport, coherence, representative-comparison, or tower-comparison obligation;
- (ix) representative and eligibility policies when invoked;
- (x) the non-scalar obligation registry;
- (xi) for each metric-mode comparison, the metric quantale, bottleneck scalarization, finite protected profile family $\mathfrak{B}$, actual discrepancy upper-bound evidence, and strict safety inequality

$$\text{val}_{\mathfrak{B}}(\mathfrak{d}_{\text{met}}) < \text{gap}_{\tau}(\mathfrak{B});$$

- (xii) the Type IV scope and, when invoked, the comparison artifact, terminal-tameness evidence, fixed $(W, \tau, I_{\text{mon}})$, and defect record;
- (xiii) Claim, CM, and TB references together with evidence status and invocation role;
- (xiv) repair ancestry and immutable artifact references.

No metric ledger, positive threshold reserve, or gap certificate is fabricated for a purely exact comparison. Conversely, an exact comparison does not by itself certify positive-radius robustness. Non-scalar obligations cannot be discharged by numerical slack.

Proof and status. Theorem 1.10 is the only metric theorem in the opening contract block. Definition 1.12 records mathematical data and admissibility clauses. Policy 1 records operational audit discipline.

1.3 UCC-Contract, threshold-gap records, and repaired evaluation

Definition 1.13 (v19 UCC-Contract). A v19 UCC-Contract consists of:

- (1) a metric-ledger quantale and, when required, a bottleneck scalarization;
- (2) a barcode metric and infinite-bar convention;
- (3) right-open windows and separate partition or overlap data;
- (4) a threshold policy or threshold sweep;
- (5) the hard deletion and evaluation rules;
- (6) a comparison-mode policy;
- (7) threshold-gap, crop, and window-edge policies;
- (8) representative and eligibility policies when invoked;
- (9) metric and non-scalar obligation policies;

- (10) gate and tower-comparison policies;
- (11) Claim, CM, TB, repair, and artifact-identity policies.

Definition 1.14 (Right-open windows, partition covers, and overlap covers). A window is a right-open interval

$$W = [u, u'), \quad u' \in \mathbb{R} \cup \{\infty\}.$$

A right-open partition cover is pairwise disjoint and ordered when sequential continuation is invoked. An overlap-admitting cover records every nonempty finite intersection used by the certificate logic and carries the required nerve, simplex, and face-compatibility data. The two cover structures are not silently identified.

Definition 1.15 (Window restriction). Define

$$W_W : \text{Pers}_k^{\text{cons}} \longrightarrow \text{Pers}_k^{\text{cons}}$$

by restriction to W , with extension by zero outside W when a globally indexed object is required. At barcode level, interval summands are clipped to W and empty intersections are discarded. If $u' < \infty$, the infinite bar $[a, \infty)$ becomes $[\max\{a, u\}, u')$ whenever the intersection is nonempty.

Definition 1.16 (Windowed pre-threshold profile). For $F \in \text{FiltCh}((k))$, define

$$B_{i,W}(F) := W_W \mathbf{P}_i(F).$$

All threshold clearance used for certification is measured on this profile before hard deletion.

Definition 1.17 (Windowed repaired profile). Define

$$\mathbf{P}_{i,W,\tau}(F) := \text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(B_{i,W}(F)).$$

This is a repaired evaluation record. It is metric-gap-certified only after the relevant two-sided record is supplied.

Definition 1.18 (Canonical comparison mode). Every comparison consumed by a v19 claim records exactly one mode:

$$\text{exact}, \quad \text{metric_gap_certified}, \quad \text{not_compared}.$$

The meanings are:

- (i) **exact**: an actual equality or persistence-profile isomorphism between the declared source and target is proved and artifact-backed;
- (ii) **Metric-gap-certified mode**. A real pre-threshold bottleneck upper bound and a two-sided threshold-gap certificate are supplied at the same protected budget;
- (iii) **not_compared**: the governing claim consumes no comparison conclusion between the declared objects.

An exact comparison establishes only the stated exact relation for the stated profiles; it is not automatically a positive-radius robustness certificate. The mode **not_compared** cannot discharge a comparison obligation that the claim actually invokes.

Definition 1.19 (Exact comparison record). An exact comparison record contains:

- (i) immutable identifiers for the source and target profiles;

- (ii) the monitored degree, window, threshold, crop/restriction convention, and declared scope;
- (iii) an actual equality or persistence-profile isomorphism at the declared equivalence strength;
- (iv) the proof object and immutable artifact identifiers needed to reconstruct the exact comparison;
- (v) status `pass`, `reject`, or `undefined` under the Chapter 7 atomic status calculus.

No scalarized metric budget or positive threshold reserve is inferred merely from this record.

Definition 1.20 (Threshold clearance). For a barcode B ,

$$\text{clear}_\tau(B) := \inf_{I \in B_{\text{fin}}} |\ell(I) - \tau|.$$

If $B_{\text{fin}} = \emptyset$, set $\text{clear}_\tau(B) = +\infty$. If a finite bar has length exactly τ , then $\text{clear}_\tau(B) = 0$.

Definition 1.21 (Threshold safety margin). For a finite family $B_1, \dots, B_m$, define

$$\text{gap}_\tau(B_1, \dots, B_m) := \min_j \text{clear}_\tau(B_j).$$

For one barcode, $\text{gap}_\tau(B) = \text{clear}_\tau(B)$. The notation is a ledger-facing alias and introduces no second invariant.

Definition 1.22 (Window-edge clearance and crop budget). A window-edge clearance record, denoted $\text{clear}_{\partial W}(B)$, measures the separation of relevant endpoints from active window boundaries. A crop budget $\delta_{\text{crop}}(W)$ bounds discrepancy introduced by restriction, clipping, or moving boundaries before hard deletion. A source-level perturbation bound cannot be used as a windowed pre-threshold bound without a theorem or artifact controlling this crop step.

Definition 1.23 (Metric threshold-gap record). A threshold-gap record is used only in metric-gap-certified mode. Its canonical mode label is

`metric_gap_certified`.

The record contains at least:

- (i) immutable identifiers for the compared pre-threshold profiles;
- (ii) the two profiles, window, threshold, and the fixed mode `metric_gap_certified`;
- (iii) a declared ε and evidence for $d_B(B, C) \leq \varepsilon$;
- (iv) crop and window-edge data when required;
- (v) $\text{clear}_\tau(B)$, $\text{clear}_\tau(C)$, and $\text{gap}_\tau(B, C)$;
- (vi) confirmation that neither finite-bar family meets $[\tau - \varepsilon, \tau + \varepsilon]$;
- (vii) the scalarization and its discrepancy upper-bound evidence when the budget is ledger-derived;
- (viii) reconstructible artifact references;
- (ix) one specialized status:

`gap_certified`, `gap_failed`, `gap_pending`.

A one-sided record is not a v19 threshold-gap certificate. This record is not required for a purely exact comparison.

Remark 1.24 (Specialized status mapping). The specialized gap statuses map to the canonical atomic statuses by

$$\text{gap_certified} \mapsto \text{pass}, \quad \text{gap_failed} \mapsto \text{reject}, \quad \text{gap_pending} \mapsto \text{undefined}.$$

They do not create an independent audit-status algebra.

Remark 1.25 (Exact comparison versus robustness certification). An exact equality or proved profile isomorphism may establish equality of specific repaired outputs without a positive metric budget. It does not prove robustness under positive perturbation. A profile with a finite bar of length exactly τ may be exactly compared with itself, but it is not threshold-gap certified at τ .

Remark 1.26 (Bottleneck and interleaving metrics in the constructible one-parameter regime). In the constructible one-parameter persistence regime over a field, the isometry theorem identifies

$$d_{\text{int}}(P, Q) = d_{\text{B}}(\mathcal{B}(P), \mathcal{B}(Q)).$$

The identification is read under the v19 infinite-bar convention. Interleaving-distance records may be used only when their barcode profiles are precisely those entering the threshold-gap record.

Definition 1.27 (Typed pipeline ledger and scalarized budget). For one declared metric-gap-certified comparison, let $\delta_j \in \mathbb{V}$ be its metric contributions. Define

$$\mathfrak{d}_{\text{met}} := \bigoplus_j \delta_j.$$

A certified scalarization satisfies

$$\text{val}_{\text{B}}(0_{\mathbb{V}}) = 0, \quad \text{val}_{\text{B}}(q \oplus q') \leq \text{val}_{\text{B}}(q) + \text{val}_{\text{B}}(q').$$

Set

$$\varepsilon_{\text{B}}(\mathfrak{d}_{\text{met}}) := \text{val}_{\text{B}}(\mathfrak{d}_{\text{met}}).$$

The record must also establish

$$d_{\text{B}}(B, C) \leq \varepsilon_{\text{B}}(\mathfrak{d}_{\text{met}})$$

for each comparison to which the budget is applied. Non-scalar obligations do not belong to $\mathfrak{d}_{\text{met}}$.

Remark 1.28 (Sharpness of the ε -band). The sharp closed forbidden band for the two-sided theorem is

$$[\tau - \varepsilon, \tau + \varepsilon].$$

A half-width 2ε policy may be used only as an explicitly non-sharp conservative policy and may not be described as the sharp Core constant.

Remark 1.29 (Core sovereignty over Search). The Core accepts or rejects through declared mathematical conditions, typed obligations, diagnostics, budget checks, and artifact-supported records. No Search-side quantity upgrades a reject or undefined result.

1.4 Overlap Gate and B-Gate⁺

Definition 1.30 (Overlap Gate). Let

$$W_{\alpha\beta} := W_\alpha \cap W_\beta.$$

For every monitored degree, both local objects are evaluated directly on $W_{\alpha\beta}$ at one declared common threshold, with an explicit threshold-reconciliation record when their local thresholds differ. Exactly one comparison mode is then recorded.

An invoked pairwise Overlap Gate passes only under one of the following two branches.

- (i) **Exact branch.** Mode `exact` is declared and an exact overlap record supplies the directly evaluated common-window profiles and an actual equality or persistence-profile isomorphism

$$\text{Eval}_{i, W_{\alpha\beta}, \tau}(F_\alpha) \xrightarrow{\sim} \text{Eval}_{i, W_{\alpha\beta}, \tau}(F_\beta),$$

together with immutable source, target, proof, crop/restriction, and threshold-reconciliation artifacts.

- (ii) **Metric branch.** Mode `metric_gap_certified` is declared; the directly evaluated pre-threshold profiles

$$B_{i, W_{\alpha\beta}}^\alpha := W_{W_{\alpha\beta}} \mathbf{P}_i(F_\alpha), \quad B_{i, W_{\alpha\beta}}^\beta := W_{W_{\alpha\beta}} \mathbf{P}_i(F_\beta)$$

satisfy an artifact-supported estimate

$$d_B(B_{i, W_{\alpha\beta}}^\alpha, B_{i, W_{\alpha\beta}}^\beta) \leq \varepsilon_{\alpha\beta, i};$$

a two-sided metric threshold-gap record protects both profiles; crop, window-edge, comparison, and threshold-reconciliation contributions are declared; and

$$\varepsilon_{\alpha\beta, i} < \text{gap}_\tau(B_{i, W_{\alpha\beta}}^\alpha, B_{i, W_{\alpha\beta}}^\beta).$$

Theorem 1.10 then yields

$$d_{\text{int}}(\text{Eval}_{i, W_{\alpha\beta}, \tau}(F_\alpha), \text{Eval}_{i, W_{\alpha\beta}, \tau}(F_\beta)) \leq \varepsilon_{\alpha\beta, i}$$

in the declared constructible one-parameter regime.

Mode `not_compared` records that no overlap inference is consumed; it does not constitute a passing Overlap Gate and cannot discharge pairwise or higher Čech coherence. In either passing branch, every invoked representative, realization, and Type IV clause has compatible provenance; Type IV is either explicitly `not_invoked` by scope or is `certified_zero`; and every required artifact is present. A passing pairwise gate does not by itself produce higher-simplex coherence, a global certificate atlas, a global filtered object, or an external theorem.

Remark 1.31 (Exact and metric layers of the Overlap Gate). The metric branch has two barcode levels: it consumes a discrepancy bound on the directly windowed pre-threshold profiles and transports that bound to the repaired outputs. A post-threshold distance estimate alone does not certify pre-threshold classification stability.

The exact branch instead proves the declared equality or profile isomorphism directly. It may bypass metric transport for that exact claim, but it supplies no positive-radius threshold robustness unless a separate metric certificate is also proved.

Definition 1.32 (B-Gate⁺ (v19 threshold-gap repaired gate)). B-Gate⁺ passes on $(F; W, \tau)$ if all required clauses pass and no required clause is undefined.

(1) **Topological clause.**

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

If an admissible compatible representative is supplied, this is equivalently recorded as

$$\text{PH}_1(C_{\tau,W}F) = 0.$$

(2) **Comparison-mode clause.** Every consumed comparison declares exactly one mode from Definition 1.18. In mode `exact`, a valid exact comparison record is present. In mode `metric_gap_certified`, a valid two-sided metric threshold-gap record is present. Mode `not_compared` is admissible only when the governing scope consumes no comparison inference and cannot discharge an invoked comparison obligation.

(3) **Categorical clause.** If `Ext` is invoked, an admissible realization-compatible representative exists, the eligible regime and amplitude conditions hold, and

$$\text{Ext}^1(R(C_{\tau,W}F), k) = 0.$$

Missing eligibility is undefined, not pass.

(4) **Type IV clause.** Type IV is explicitly `not_invoked` or is `certified_zero` with a valid artifact and

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$$

at one fixed $(W, \tau, I_{\text{mon}})$.

(5) **Metric-budget clause.** For every comparison whose mode is `metric_gap_certified`, the protected family $\mathfrak{B}$ is finite, the scalarized value is proved to upper-bound the actual discrepancy, and

$$\text{val}_{\mathfrak{B}}(\mathfrak{d}_{\text{met}}) < \text{gap}_{\tau}(\mathfrak{B}).$$

No numerical budget or positive reserve is fabricated for a purely exact comparison.

(6) **Non-scalar obligations.** Every required representative, morphism, eligibility, terminal-tameness, coherence, claim-status, invocation-role, and artifact-identity obligation passes.

(7) **Manifest and claim-use clause.** The record is complete and consistent under Chapter 7; every consumed claim has an admissible evidence-status–invocation-role combination; and no rejected, deprecated, unknown, unproved, operational, or spec item is promoted to theorem evidence.

Remark 1.33 (Typed conjunction and no cancellation). B-Gate⁺ is a typed conjunction. A passing clause cannot cancel a reject or required undefined clause, and numerical reserve cannot repair a missing non-scalar artifact.

Remark 1.34 (Interpretation of B-Gate⁺). B-Gate⁺ is a Core acceptance predicate for a declared object, window, threshold, monitored scope, and proof-object record. It is not a Search score, probabilistic judgment, external-domain oracle, or transport theorem.

Remark 1.35 (No representative, no Ext-clause). The persistence clause may be evaluated directly from `Eval`. The `Ext` clause may not be invoked unless a realization-compatible admissible representative and every eligibility condition are supplied.

1.5 Failure landscape and the Type IV canon

Definition 1.36 (Failure landscape). The five failure types are:

- **Type I: topological failure.**

$$\text{Eval}_{1,W,\tau}(F) \neq 0.$$

- **Type II: categorical failure in the eligible regime.**

$$\text{Ext}^1(R(C_{\tau,W}F), k) \neq 0.$$

Missing eligibility makes the clause `undefined` rather than categorically obstructed.

- **Type III: operational, metric, or functorial instability.** This includes a failed exact or metric comparison, failed threshold-gap certification, invalid scalarization, insufficient discrepancy evidence, failed crop control, missing representative or comparison morphism data, misuse of `not_compared`, failed threshold reconciliation, or failed Restart transition control. Missing mathematical or operational evidence normally makes the affected clause `undefined`.

- **Type IV: tower-level terminal-generic obstruction.** A valid comparison artifact exists and

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0).$$

Missing artifact data are `undefined`, not obstructed.

- **Type V: claim-status, manifest, or provenance failure.** This includes use of a rejected claim, promotion of an operational or spec item to theorem evidence, concealment of missing evidence, identifying `undefined` with zero or pass, fabricating `not_invoked` or `not_compared`, artifact-identity or hash mismatch, incompatible versions, or inconsistent repair ancestry. A Type III `undefined` clause may therefore coexist with Type V when its absence is misreported or governance data are inconsistent.

Definition 1.37 (Type IV scope status at the Chapter 1 contract level). Every run records exactly one Type IV status:

`not_invoked, undefined, certified_zero, obstructed.`

They mean, respectively: tower semantics are outside scope; tower semantics are in scope but required data are missing or ill-typed; a valid diagnostic exists and is zero; or a valid diagnostic exists and is nonzero.

Definition 1.38 (Tower comparison artifact). A tower comparison artifact is actual declared morphism data

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}).$$

The source, target, morphism, kernel and cokernel profiles, and terminal-tameness evidence must be typed before Type IV diagnostics are computed. A bottleneck matching or expected limit correspondence is not automatically such a morphism.

Definition 1.39 (Terminal-generic Type IV diagnostics at contract level). When the artifact is defined and terminal tame, set

$$\begin{aligned} \mu_{i,W,\tau} &:= \text{tgdim}_W(\ker \phi_{i,W,\tau}), \\ \nu_{i,W,\tau} &:= \text{tgdim}_W(\text{coker } \phi_{i,W,\tau}). \end{aligned}$$

Fix a declared window–threshold pair (W, τ) and a declared finite set of monitored degrees I_{mon} . Define

$$\mu_{\text{Collapse}} := \sum_{i \in I_{\text{mon}}} \mu_{i, W, \tau},$$

$$\nu_{\text{Collapse}} := \sum_{i \in I_{\text{mon}}} \nu_{i, W, \tau}.$$

An aggregate over infinitely many monitored degrees, or across multiple window–threshold pairs, requires a separately declared aggregation and summability theorem. Chapter 5 supplies the precise terminal-generic definitions.

Remark 1.40 (Type IV canonical convention). The equality

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$$

means that no monitored terminal-generic kernel or cokernel defect is present. It does not imply full isomorphism, absence of finite interior defects, or object-level equivalence. The legacy Type IV symbols $u_{i, W, \tau}$ and u_{Collapse} are deprecated; $\nu_{i, W, \tau}$ and ν_{Collapse} are canonical. The legacy symbols create no normative alias in v19.0.0.

Remark 1.41 (Normative ν -migration rule). In theorem statements, hypotheses, gate clauses, diagnostic records, and manifest semantics, the cokernel diagnostic is written with ν . The Latin symbol u may occur only in explicit historical deprecation or migration prose and carries no v19.0.0 diagnostic meaning.

Definition 1.42 (Windowed Collapse Zone). Define

$$\mathfrak{C}_{W, \tau} := \left\{ F \left| \begin{array}{l} \text{Eval}_{1, W, \tau}(F) = 0, \\ \text{Ext}^1(R(C_{\tau, W}F), k) = 0 \text{ if the Ext clause is invoked} \end{array} \right. \right\}.$$

The Ext condition is meaningful only in the eligible realization regime. No converse from Ext-vanishing to repaired-profile vanishing is asserted.

Remark 1.43 (Collapse Zone is not a complete gate). Membership in $\mathfrak{C}_{W, \tau}$ does not by itself imply threshold-gap certification, metric-ledger pass, Type IV cleanliness, manifest completeness, B-Gate⁺ pass, or an external-domain conclusion.

1.6 Minimal Manifest Axiom and proof-object discipline

Axiom 1.1 (Minimal Manifest Axiom). A proof-object candidate submitted for Core certification contains at least:

- (i) immutable RunID, theory version, schema version, Claim Register version, claim scope, repair ancestry, and artifact hashes;
- (ii) source and realized objects, windows, monitored degrees, finite I_{mon} , and overlap structure;
- (iii) metric, infinite-bar, threshold, deletion, collapse, and evaluation policies;
- (iv) exactly one comparison mode for every consumed comparison;
- (v) in mode *exact*, the actual equality or profile-isomorphism proof, equivalence strength, and crop/restriction artifacts;

- (vi) *in mode `metric_gap_certified`, every required threshold-gap, crop, window-edge, quantale, scalarization, actual discrepancy upper-bound, protected-family, and strict-reserve record;*
- (vii) *for mode `not_compared`, the explicit scope exclusion proving that no comparison conclusion is consumed;*
- (viii) *representative existence, compatibility, equivalence strength, actual morphism data, and realization compatibility when invoked;*
- (ix) *amplitude, edge-realization, and eligibility data when `Ext` is invoked;*
- (x) *a non-scalar obligation registry with invocation state, dependency set, atomic status, evidence, and failure route;*
- (xi) *Type IV scope and, when invoked, comparison, source semantics, provenance authority, terminal-tameness, fixed $(W, \tau, I_{\text{mon}})$, degreewise and total μ, ν diagnostics, and interior-defect status;*
- (xii) *atomic clause statuses*

`pass, reject, undefined, not_invoked;`

- (xiii) *manifest state*

`complete, incomplete, inconsistent;`

- (xiv) *an aggregate typed-conjunction verdict and complete failure-routing record;*
- (xv) *every Claim, CM, TB, and candidate-only reference with its document role, evidence status, invocation role, exact source location, and permitted strength;*
- (xvi) *a complete finite or countable clause registry, every justified scope exclusion, and immutable transitive artifact references sufficient for independent reconstruction and replay.*

Remark 1.44 (Why the manifest axiom belongs to the Core). Concrete storage may live in a Companion or implementation layer, but the requirement that certification be supported by typed, reconstructible, artifact-backed records belongs to the Core audit discipline.

Remark 1.45 (Canonical v19 manifest integration). Axiom 1.1 gives the foundational minimum. Chapter 3 extends repaired-evaluation and representative records. Chapter 7 supplies the canonical manifest, immutable identity, typed dependency closure, verdict reconstruction, replayability, and repair-as-new-record semantics.

Remark 1.46 (Appendix F and Appendix G boundary). Appendix F elaborates obligation types, invocation semantics, atomic status algebra, dependency closure, non-compensation, failure routing, and certificate readiness. Appendix G elaborates the manifest field schema, immutable identifiers, reference slots, artifact links, and serialization-neutral proof-object structure. Neither appendix establishes a mathematical claim merely by recording it.

Remark 1.47 (Claim, micro-theorem, and repair traceability). Every claim, micro-theorem, toy bridge, and repaired rule must be identified by its exact registered item and strength. This prevents silent reuse of rejected claims and silent promotion of toy or Search material.

Definition 1.48 (Manifest-complete proof-object candidate). A manifest-complete proof-object candidate satisfies the required field, typing, identity, and consistency conditions of Axiom 1.1 and Chapter 7. Manifest completeness means that the required record is present and internally consistent; it does not establish the truth of the recorded mathematical claims.

Definition 1.49 (Certified proof object). A certified proof object is a manifest-complete candidate such that every required mathematical claim has valid proof support, every computational claim has auditable artifact support, every required non-scalar obligation passes, every required gate clause passes, no required clause is reject or undefined, every registered claim is used within its permitted strength, and the aggregate verdict is pass.

Remark 1.50 (Manifest completeness is not proof). A complete manifest may faithfully record a failed proof, an obstructed tower, or a rejected claim. Completeness supports auditability; it does not replace mathematical validity.

1.7 Forward contract for countable certificate atlases

Policy 2 (Countable atlas forward contract). *Chapter 1 does not assert that countability, definability, finite Čech depth, and ordinary summability alone produce a global certificate. A countable local-to-global claim is admissible only through the typed record-level theorem of Chapter 6, in particular Theorem 6.46, or through a separately proved stronger theorem.*

A countable atlas record declares at least:

- (i) *an audit-locally finite right-open overlap cover;*
- (ii) *a finite Čech-depth bound;*
- (iii) *complete local records;*
- (iv) *every nonempty nerve simplex required by the declared cover and claim;*
- (v) *face compatibility on every consumed restriction;*
- (vi) *a common-threshold or threshold-reconciliation policy;*
- (vii) *every consumed face or simplex comparison uses `exact` or `metric_gap_certified`. The mode `not_compared` cannot discharge coherence;*
- (viii) *exact proof artifacts for exact-mode simplices;*
- (ix) *scalarization, actual discrepancy upper-bound evidence, and positive sharp threshold reserve for metric-mode simplices only;*
- (x) *summable scalarized exposure over exactly the metric-mode simplices when a countable exposure policy is invoked;*
- (xi) *correctly typed Type IV scope;*
- (xii) *all required non-scalar obligations;*
- (xiii) *a complete countable clause registry and audit-readable artifact closure.*

The conclusion is a Čech-coherent certificate atlas at record level. No global filtered object, object-level descent, or external theorem follows without an additional theorem.

Remark 1.51 (Why the former countable-cover theorem is not retained). The earlier statement that a definable countable cover alone implies bounded global overlap error omitted typed scalarization, audit-local finiteness, higher-simplex coherence, face compatibility, Type IV scope discipline, and non-scalar obligations. The v19 repair delegates the valid mathematical theorem to Chapter 6.

1.8 Explicit non-claims of Chapter 1

The following exclusions are part of the meaning of the v19 Core.

- No unconditional global equivalence $\text{PH}_1 \iff \text{Ext}^1$ is claimed.
- No reverse implication from Ext-vanishing to repaired-profile vanishing is available under the standing Core axioms.
- No positive-threshold Serre subcategory, exact reflector, abelian localization, or universal quotient is claimed in general.
- No global non-expansiveness of hard deletion under the standard bottleneck metric is claimed.
- No one-sided threshold-gap record is a v19 certificate.
- No finite bar of length exactly τ is gap-safe.
- No exact comparison is automatically a positive robustness certificate.
- No metric ledger, positive gap, or reserve is fabricated merely to encode a purely exact comparison.
- No record whose comparison mode is `not_compared` discharges an invoked overlap, transport, coherence, representative-comparison, or tower-comparison obligation.
- No exact overlap certificate is automatically a positive-radius overlap-robustness certificate.
- No unconditional crop-collapse commutation is claimed.
- No crop-collapse criterion may ignore an infinite bar that becomes finite after bounded-window clipping.
- No global strict filtered lift C_τ is asserted by default.
- No identity $C_{\tau,W}F = (C_\tau F)|_W$ is asserted without a proved compatibility theorem.
- No bottleneck matching is automatically a persistence morphism.
- No tower comparison morphism is automatic outside a proved exact-admissible regime.
- No missing Type IV artifact is interpreted as zero.
- No no-tower claim is represented by a fabricated $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$.
- No zero terminal-generic diagnostic implies full isomorphism without a separate finite-defect theorem.
- No aggregate Type IV diagnostic is defined over an undeclared or non-summable monitored scope.
- No legacy Type IV u -notation has normative force outside explicit historical deprecation or migration prose.
- No unscalarized quantale element is compared directly with real-valued clearance.
- No scalarized value is used without evidence that it upper-bounds the actual discrepancy consumed by the proof.
- No numerical budget repairs a missing non-scalar obligation.

- No Search or AI artifact is a Core proof object or gate verdict.
- No scalar potential replaces the typed verifier.
- No external interpretation of a Core verdict follows without a separately proved transport theorem.
- No record-level gluing implies object-level descent.
- No countable-cover theorem follows from definability, finite depth, and ordinary summability alone.
- No manifest completeness is identified with mathematical truth.
- No Claim Register entry is proof merely because it is registered.
- No Claim Register entry may promote a statement beyond its underlying proof.
- No operational rule or specification target is used as theorem evidence merely because it is registered or satisfied procedurally.
- No Core micro-theorem is an external-domain theorem.
- No toy bridge is an arithmetic, PDE, geometric, categorical, or cryptographic theorem.
- No Bridge candidate or Bridge program is a Bridge theorem.
- No NBT statement is a v19.0.0 theorem claim.
- No reference to Appendix CR-v19, Appendix CM-v19, Appendix TB, Appendix F, Appendix G, or Appendix NBT enlarges the Core.
- No concrete YAML, JSON, database, or proof-store serialization format is fixed by Chapter 1.

1.9 Terminology and notation

Fix a base field k . We write Vect_k for finite-dimensional vector spaces and $\text{FiltCh}(()k)$ for finite-type filtered chain complexes. The degreewise persistence functor is $\mathbf{P}_i$, window restriction is W_W , and repaired evaluation is

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

The standard bottleneck distance is d_B , and the interleaving distance is d_{int} . Threshold clearance and safety margin are

$$\begin{aligned} \text{clear}_\tau(B) &= \inf_{I \in B_{\text{fin}}} |\ell(I) - \tau|, \\ \text{gap}_\tau(B_1, \dots, B_m) &= \min_j \text{clear}_\tau(B_j). \end{aligned}$$

Metric ledger values are compared with clearance through val_B . The canonical comparison modes are

`exact`, `metric_gap_certified`, `not_compared`.

The Type IV scope vocabulary is

`not_invoked`, `undefined`, `certified_zero`, `obstructed`.

The local diagnostics are

$$\mu_{i,W,\tau} = \text{tgdim}_W(\ker \phi_{i,W,\tau}), \quad \nu_{i,W,\tau} = \text{tgdim}_W(\text{coker } \phi_{i,W,\tau}),$$

For a fixed declared (W, τ) and finite monitored degree set I_{mon} , the aggregate pair is

$$\left(\sum_{i \in I_{\text{mon}}} \mu_{i,W,\tau}, \sum_{i \in I_{\text{mon}}} \nu_{i,W,\tau} \right) = (\mu_{\text{Collapse}}, \nu_{\text{Collapse}}).$$

The legacy Type IV symbols $u_{i,W,\tau}$ and u_{Collapse} are deprecated; $\nu_{i,W,\tau}$ and ν_{Collapse} are canonical.

For traceability, ClaimID, CMID, TBID, and RunID denote Claim, Core Micro-Theorem, Toy Bridge, and immutable run identifiers.

1.10 Provenance and status labels

Constructible interval decomposition, the standard bottleneck/interleaving isometry, and ordinary barcode stability belong to standard persistence theory. The hard deletion, repaired evaluation, threshold-clearance ledger, representative discipline, Type IV diagnostics, and typed audit architecture belong to the present AK-HPDST framework.

The v19 discipline replaces earlier overstrong language involving global positive-threshold localization, global exact reflectors, global hard-threshold non-expansiveness, automatic tower comparison, untyped ledger comparison, manifest completeness as proof, and object-level descent from record-level gluing.

Status labels for this chapter. *Core mathematical theorem / metric-repair theorem:* Theorem 1.10.

Core definitions and mathematical contracts: Definitions 1.1, 1.2, 1.18, 1.19, 1.12, 1.13, 1.14, 1.15, 1.16, 1.17, 1.20, 1.21, 1.22, 1.23, 1.27, 1.30, 1.32, 1.36, 1.37, 1.38, 1.39, 1.42, 1.48, 1.49.

Operational policy / audit axiom: Policy 1, Axiom 1.1, Policy 2.

Core doctrine and interpretive controls: Remark 1.3, Remark 1.4, Remark 1.5, Remark 1.6, Remark 1.7, Remark 1.8, Remark 1.9, Remark 1.11, Remark 1.24, Remark 1.25, Remark 1.26, Remark 1.28, Remark 1.29, Remark 1.31, Remark 1.33, Remark 1.34, Remark 1.35, Remark 1.40, Remark 1.41, Remark 1.43, Remark 1.44, Remark 1.45, Remark 1.46, Remark 1.47, Remark 1.50, Remark 1.51.

Explicit non-claims: Section 1.8.

2 Chapter 2: Barcode Metrics, Threshold Deletion, and Gap-Safe Stability

2.1 Scope of the v19 metric repair

This chapter supplies the persistence-theoretic foundation of the v19 Core. Its role is narrower and sharper than the corresponding v18 chapter: it repairs the metric status of positive-threshold barcode deletion under the standard bottleneck metric.

The central rule of Chapter 2 is the following.

For a positive threshold $\tau > 0$, hard barcode threshold deletion T_τ^{bar} deletes finite bars of length at most τ . It is endpoint-independent, idempotent, and threshold-monotone. It is *not* globally non-expansive under the standard bottleneck metric. The former global non-expansiveness claim is replaced by threshold-gap-safe stability.

Thus the v19 Core retains the operational collapse calculus needed by the AK framework, but it no longer treats hard positive-threshold deletion as a global 1-Lipschitz operation without a threshold-gap certificate.

Remark 2.1 (Metric repair relative to v18). Earlier v18 formulations correctly rejected the overstrong Serre-localization interpretation of positive-threshold deletion, but still retained language suggesting global barcode non-expansiveness. The v19 repair removes that remaining overclaim. The replacement theorem is the two-sided threshold-gap-safe stability theorem of Theorem 2.33.

Remark 2.2 (What this chapter does not do). This chapter is a Core persistence-theoretic chapter. It does not prove an arithmetic, PDE, geometric, categorical, or cryptographic theorem. Any external-domain conclusion requires a separately proved interface or bridge theorem.

2.2 Constructible one-parameter persistence

Fix a field k . Let

$$\text{Pers}_k := [(\mathbb{R}, \leq), \text{Vect}_k]$$

be the category of one-parameter persistence modules over k , indexed by the ordered real line.

Definition 2.3 (Constructible persistence module). A persistence module $M \in \text{Pers}_k$ is called *constructible* if:

- (i) $M(t)$ is finite-dimensional over k for every $t \in \mathbb{R}$;
- (ii) on every bounded interval $J \subset \mathbb{R}$, there is a finite critical set outside which all structure maps are isomorphisms;
- (iii) there is no accumulation of critical parameters inside a bounded interval.

The full subcategory of constructible persistence modules is denoted by

$$\text{Pers}_k^{\text{cons}} \subset \text{Pers}_k.$$

Remark 2.4 (Working regime). All Core statements in this chapter are made inside the pointwise finite-dimensional constructible one-parameter regime over k , or inside finite barcode profiles obtained from declared windowed Core evaluations. Wild, nonconstructible, infinite-rank, or uncontrolled multi-parameter persistence data are outside the scope of this chapter unless a separate interface reduces them to declared constructible one-parameter profiles.

Definition 2.5 (Finite-type filtered chain complex). Let $\text{Ch}^b(k)$ be the category of bounded chain complexes of finite-dimensional k -vector spaces. A *finite-type filtered chain complex* is a pair

$$F = (C_\bullet, \{F^t C_\bullet\}_{t \in \mathbb{R}})$$

such that:

- (i) $F^t C_\bullet \subseteq F^{t'} C_\bullet$ whenever $t \leq t'$;
- (ii) the filtration is exhaustive and left-bounded;
- (iii) for each homological degree i , the persistence module

$$\mathbf{P}_i(F) : \mathbb{R} \rightarrow \text{Vect}_k, \quad t \mapsto H_i(F^t C_\bullet)$$

belongs to $\text{Pers}_k^{\text{cons}}$.

The category of such objects, with filtration-preserving chain maps, is denoted by

$$\text{FiltCh}(\cdot)k).$$

Remark 2.6 (Persistence layer first). The Core reads a filtered object F through monitored persistence profiles

$$\mathbf{P}_i(F).$$

Any filtered-level or derived-level use requires an additional representative policy. The barcode-level collapse operation by itself does not supply a global strict filtered lift.

2.3 Barcode normal forms and bar lengths

Definition 2.7 (Interval module). For an interval $I \subseteq \mathbb{R}$, the interval module k_I is the persistence module defined by

$$k_I(t) = \begin{cases} k, & t \in I, \\ 0, & t \notin I, \end{cases}$$

with structure maps equal to the identity whenever both source and target lie in I , and zero otherwise.

Definition 2.8 (Bar and barcode). An interval summand k_I is called a *bar*. A barcode of a constructible persistence module M is an interval decomposition

$$M \cong \bigoplus_{\lambda \in \Lambda} k_{I_\lambda},$$

with locally finite barcode support on bounded intervals. The corresponding multiset of bars is denoted by

$$\mathcal{B}(M) = \{I_\lambda\}_{\lambda \in \Lambda}.$$

Theorem 2.9 (Constructible interval decomposition). *Every object $M \in \text{Pers}_k^{\text{cons}}$ admits a locally finite interval decomposition*

$$M \cong \bigoplus_{\lambda \in \Lambda} k_{I_\lambda}.$$

The barcode is unique up to permutation of interval summands.

Proof sketch. This is the standard interval-decomposition theorem for pointwise finite-dimensional constructible one-parameter persistence modules over a field. On each bounded interval, constructibility gives a finite representation of a linearly ordered finite poset, hence a decomposition into interval representations. Local finiteness assembles the global barcode, and Krull–Schmidt uniqueness gives uniqueness up to permutation. $\square$

Definition 2.10 (Length of a bar). Let $I \subset \mathbb{R}$ be a finite bar with endpoint values $a < b$. Its length is

$$\ell(I) := b - a.$$

If $I = [a, \infty)$, or differs from this only by endpoint bookkeeping at a , it is called an infinite bar and is assigned length

$$\ell(I) := +\infty$$

for thresholding purposes.

Remark 2.11 (Endpoint independence). The length $\ell(I)$ depends only on endpoint values, not on whether finite endpoints are open, closed, or half-open. All threshold rules in this chapter are therefore independent of harmless endpoint bookkeeping conventions.

2.4 Standard bottleneck metric and infinite-bar convention

Definition 2.12 (Standard bottleneck distance with infinite bars). Let B and C be finite barcode profiles, or locally finite constructible barcode profiles restricted to a declared bounded audit range. The bottleneck distance $d_B(B, C)$ is computed by matchings between bars and, for finite bars, the diagonal.

The convention for infinite bars is fixed as follows.

- (i) A finite bar may be matched to the diagonal with cost $\ell(I)/2$.
- (ii) An infinite bar is never matched to the diagonal at finite cost.
- (iii) A finite bar and an infinite bar are never matched at finite cost.
- (iv) Two infinite bars $[a, \infty)$ and $[a', \infty)$ may be matched with cost

$$\text{cost}([a, \infty), [a', \infty)) := |a - a'|.$$

Endpoint bookkeeping at finite endpoints is ignored according to the endpoint-independence convention.

Lemma 2.13 (Infinite-bar threshold invariance). *For every finite threshold $\tau > 0$, infinite bars are invariant under hard finite-threshold deletion:*

$$T_\tau^{\text{bar}}([a, \infty)) = [a, \infty).$$

Moreover, infinite bars are never contained in a finite threshold band

$$[\tau - \varepsilon, \tau + \varepsilon].$$

Proof. An infinite bar has length $+\infty$. Since T_τ^{bar} deletes only finite bars of length at most τ , an infinite bar is retained. The finite interval $[\tau - \varepsilon, \tau + \varepsilon]$ contains only finite real values, so $+\infty$ does not lie in it. $\square$

Remark 2.14 (Bottleneck–interleaving isometry). For q-tame or constructible one-parameter persistence modules over k , the usual isometry theorem identifies the interleaving distance and the bottleneck distance on barcodes:

$$d_{\text{int}}(M, N) = d_B(\mathcal{B}(M), \mathcal{B}(N)).$$

When infinite bars are present, this statement is read under the infinite-bar convention of Definition 2.12; namely infinite bars match infinite bars by birth-time displacement $|a - a'|$, and finite-to-infinite or infinite-to-diagonal matchings have infinite cost. Under this convention, a d_B -formulated stability statement may be transferred to a d_{int} -formulated comparison only when the persistence objects and barcode profiles being compared are exactly those appearing in the certified record.

2.5 Positive-threshold short bars are not Serre in general

The following observation remains one of the categorical guard-rails of the repaired Core.

Proposition 2.15 (Positive-threshold short-bar objects are not Serre in general). *Let $\tau > 0$. The class of constructible persistence modules whose barcode summands all have finite length at most τ is not extension-closed in general. Consequently, it is not a Serre subcategory of $\text{Pers}_k^{\text{cons}}$ in general.*

Proof. It suffices to give a counterexample. Take $\tau = 1$ and consider the short exact sequence of interval modules

$$0 \longrightarrow k_{[1,2)} \longrightarrow k_{[0,2)} \longrightarrow k_{[0,1)} \longrightarrow 0.$$

The subobject $k_{[1,2)}$ has length 1, and the quotient $k_{[0,1)}$ also has length 1. However, the middle object $k_{[0,2)}$ has length 2. Thus an extension of two short-bar objects can contain a longer bar. Therefore the class of bars of length at most τ is not extension-closed in general. $\square$

Corollary 2.16 (No global exact reflector claim). *For $\tau > 0$, hard threshold deletion is not asserted in the general Core to arise from an exact Serre localization, an abelian exact reflector, or a global exact endofunctor on all of $\text{Pers}_k^{\text{cons}}$.*

Remark 2.17 (Why this guard-rail matters). If positive-threshold deletion were treated as an exact Serre reflector without additional hypotheses, then exactness, functoriality, tower comparison maps, and filtered lifts would inherit unsupported categorical strength. The repaired formulation keeps the barcode-level operation while requiring stronger claims to be explicitly justified by τ -split or exact-admissible hypotheses.

2.6 Hard barcode threshold deletion

Definition 2.18 (Hard barcode threshold deletion). Let B be a barcode profile. For $\tau > 0$, define hard barcode threshold deletion by

$$T_\tau^{\text{bar}}(B) := \{ I \in B \mid \ell(I) > \tau \text{ or } \ell(I) = +\infty \}.$$

Equivalently, T_τ^{bar} deletes precisely the finite bars of length at most τ . Thus:

$$\begin{aligned} \ell(I) < \tau &\implies I \text{ is deleted,} \\ \ell(I) = \tau &\implies I \text{ is deleted,} \\ \ell(I) > \tau &\implies I \text{ is retained,} \\ \ell(I) = +\infty &\implies I \text{ is retained.} \end{aligned}$$

For a constructible persistence module M , the notation $T_\tau^{\text{bar}}(M)$ means threshold deletion applied to $\mathcal{B}(M)$, read at the barcode-profile level.

Remark 2.19 (Status of T_τ^{bar}). In the general Core regime, T_τ^{bar} is an operation on barcode normal forms and hence on isomorphism classes of constructible persistence modules. It is not asserted, without additional hypotheses, to be a strict exact endofunctor on all morphisms of $\text{Pers}_k^{\text{cons}}$.

Proposition 2.20 (Basic properties of hard threshold deletion). *For every $\tau > 0$, the operation T_τ^{bar} satisfies:*

(i) *endpoint independence;*

(ii) *idempotence:*

$$T_\tau^{\text{bar}} T_\tau^{\text{bar}}(B) = T_\tau^{\text{bar}}(B);$$

(iii) *infinite-bar preservation;*

(iv) *threshold monotonicity: if $0 < \tau \leq \sigma$, then every bar retained by $T_\sigma^{\text{bar}}(B)$ is retained by $T_\tau^{\text{bar}}(B)$.*

Proof. All assertions follow directly from the definition. Deleting all finite bars of length at most τ twice has the same effect as deleting them once. Infinite bars have length $+\infty$, hence are never deleted by a finite threshold. Endpoint choices do not affect length. Finally, increasing the threshold can only delete additional finite bars. $\square$

Proposition 2.21 (Threshold composition). *For $\tau, \sigma > 0$,*

$$T_\sigma^{\text{bar}} T_\tau^{\text{bar}}(B) = T_{\max\{\tau, \sigma\}}^{\text{bar}}(B)$$

at the barcode-profile level.

Proof. Applying T_τ^{bar} deletes bars of finite length at most τ . Applying T_σ^{bar} afterwards deletes bars of finite length at most σ among those remaining. The total effect is deletion of all finite bars of length at most $\max\{\tau, \sigma\}$. $\square$

2.7 Counterexample to global non-expansiveness

The next example is the metric repair point of v19.

Example 2.22 (Failure of global non-expansiveness). Fix $\tau > 0$, and let $0 < \eta < \tau/4$. Consider the one-bar barcodes

$$B = \{[0, \tau + \eta)\}, \quad C = \{[0, \tau - \eta)\}.$$

Before threshold deletion, matching the two bars to each other has cost 2η . Matching both bars to the diagonal has cost

$$\max \left\{ \frac{\tau + \eta}{2}, \frac{\tau - \eta}{2} \right\} = \frac{\tau + \eta}{2} > 2\eta,$$

where the strict inequality follows from $\eta < \tau/4$. Therefore

$$d_B(B, C) = 2\eta.$$

After hard threshold deletion,

$$T_\tau^{\text{bar}}(B) = \{[0, \tau + \eta)\}, \quad T_\tau^{\text{bar}}(C) = \emptyset.$$

Therefore the remaining bar of $T_\tau^{\text{bar}}(B)$ must be matched to the diagonal, giving

$$d_B(T_\tau^{\text{bar}}(B), T_\tau^{\text{bar}}(C)) = \frac{\tau + \eta}{2}.$$

Moreover,

$$\frac{\tau + \eta}{2} > 2\eta = d_B(B, C),$$

again because $\eta < \tau/4$. Consequently,

$$d_B(T_\tau^{\text{bar}}(B), T_\tau^{\text{bar}}(C)) \leq d_B(B, C)$$

does not hold in general.

Remark 2.23 (Meaning of the counterexample). The failure occurs exactly at the threshold boundary. A small endpoint perturbation can move a finite bar from length $> \tau$ to length $\leq \tau$, thereby changing whether it is retained or deleted. Thus the correct stability statement must exclude bars near the threshold.

Corollary 2.24 (Deprecated v18 metric claim). *The statement that hard positive-threshold deletion T_τ^{bar} is globally non-expansive under the standard bottleneck metric is rejected in v19. Any occurrence of that claim in earlier formulations is deprecated and may not be used in v19 proof construction.*

2.8 Threshold clearance and gap

Definition 2.25 (Threshold clearance). Let B be a barcode profile. Define the finite-bar threshold clearance by

$$\text{clear}_\tau(B) := \inf_{I \in B_{\text{fin}}} |\ell(I) - \tau|,$$

where B_{fin} denotes the finite bars of B . If B has no finite bars, set

$$\text{clear}_\tau(B) := +\infty.$$

If B contains a finite bar of length exactly τ , then

$$\text{clear}_\tau(B) = 0.$$

Definition 2.26 (Threshold gap as ledger alias). For a barcode profile B , define the ledger safety margin

$$\text{gap}_\tau(B) := \text{clear}_\tau(B).$$

For a finite family of barcode profiles $B_1, \dots, B_m$, define

$$\text{gap}_\tau(B_1, \dots, B_m) := \min_{1 \leq j \leq m} \text{clear}_\tau(B_j).$$

Thus gap_τ is not a second invariant distinct from threshold clearance; it is the ledger-facing alias for the minimum finite-bar clearance at the deletion threshold.

Definition 2.27 (Bottleneck scalarization and typed metric budget). Let $(V, \oplus, 0_V, \preceq)$ be the declared ledger quantale. A *bottleneck scalarization* is a monotone, subadditive map

$$\text{val}_B : V \longrightarrow [0, +\infty], \quad \text{val}_B(0_V) = 0, \quad \text{val}_B(q \oplus q') \leq \text{val}_B(q) + \text{val}_B(q').$$

For a metric ledger element $\mathfrak{d}_{\text{met}} \in V$, define the real-valued bottleneck budget

$$\varepsilon_B(\mathfrak{d}_{\text{met}}) := \text{val}_B(\mathfrak{d}_{\text{met}}).$$

Whenever the established shorthand

$$\Sigma\delta < \text{gap}_\tau$$

is used in this chapter, it means the typed real inequality

$$\varepsilon_B(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau$$

for the declared protected comparison family, together with theorem-level, verified-computation, or artifact evidence that the scalarized value upper-bounds the actual bottleneck discrepancy consumed by the proof. Only metric defects belong to $\mathfrak{d}_{\text{met}}$. Representative existence, comparison morphisms, eligibility, terminal tameness, claim status, and manifest completeness are non-scalar proof obligations and cannot be repaired by numerical slack.

Remark 2.28 (Sharp threshold-safety margin versus scalarization tightness). Under an ε -bottleneck matching, the length of a finite bar may change by at most 2ε . For a *two-sided* threshold-gap certificate, however, the sharp forbidden band is

$$[\tau - \varepsilon, \tau + \varepsilon],$$

not $[\tau - 2\varepsilon, \tau + 2\varepsilon]$. Consequently, for a scalarized budget

$$\varepsilon = \varepsilon_B(\mathfrak{d}_{\text{met}}),$$

the condition

$$\varepsilon < \text{gap}_\tau(B) = \text{clear}_\tau(B)$$

means that all finite bars of the relevant pre-threshold profile avoid

$$[\tau - \varepsilon, \tau + \varepsilon].$$

The former factor $1/2$ is not part of the v19.0.0 two-sided Core certificate. The word *sharp* refers to the optimal half-width of the threshold band for a given true bottleneck bound; it does not assert that a chosen scalarization val_B is numerically tight.

Definition 2.29 (Windowed pre-threshold profile). For $F \in \text{FiltCh}(\cdot)k$, monitored degree i , and declared window W , define the *windowed pre-threshold profile*

$$B_{i,W}(F) := W_W \mathbf{P}_i(F).$$

This is the barcode profile on which threshold clearance and threshold gap are measured. The repaired evaluation is then

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(B_{i,W}(F)).$$

Remark 2.30 (Clearance is measured before deletion, after windowing). The quantity clear_τ is never measured on the already deleted profile $\text{Eval}_{i,W,\tau}(F)$. It is measured on the windowed pre-threshold profile

$$B_{i,W}(F) = W_W \mathbf{P}_i(F).$$

This order matters: the threshold gap certifies that hard deletion will not change classification under the declared perturbation budget. Once a bar has already been deleted, its distance from the threshold can no longer be audited.

Definition 2.31 (Threshold-gap comparison record). A threshold-gap comparison record for two windowed pre-threshold profiles

$$B := B_{i,W}(F), \quad C := B_{i,W}(G)$$

at one declared monitored degree i , common window W , and threshold τ contains:

- (i) immutable identifiers for B and C ;
- (ii) the two pre-threshold profiles, the monitored degree, the common window, and the threshold;
- (iii) the finite-bar length data or certified summaries for both profiles;
- (iv) $\text{clear}_\tau(B)$, $\text{clear}_\tau(C)$, and

$$\text{gap}_\tau(B, C) = \min\{\text{clear}_\tau(B), \text{clear}_\tau(C)\};$$

- (v) the metric ledger element $\mathfrak{d}_{\text{met}}$, the declared scalarization val_B , and

$$\varepsilon_B := \text{val}_B(\mathfrak{d}_{\text{met}});$$

- (vi) theorem-level, verified-computation, or artifact evidence that

$$d_B(B, C) \leq \varepsilon_B;$$

- (vii) confirmation that neither finite-bar family meets the closed band

$$[\tau - \varepsilon_B, \tau + \varepsilon_B],$$

equivalently the strict two-sided condition

$$\varepsilon_B < \text{gap}_\tau(B, C);$$

- (viii) every crop, window-edge, endpoint, or threshold-reconciliation component used to obtain the windowed comparison bound;
- (ix) reconstructible artifact references;

(x) one specialized status

gap_certified, gap_failed, gap_pending,

mapping respectively to pass, reject, and undefined.

The record is two-sided by definition. A single-profile clearance record may be retained as a readiness record, but it is not a metric comparison certificate and does not establish transport through hard deletion.

Remark 2.32 (Length exactly τ is not gap-safe). A finite bar of length exactly τ gives $\text{clear}_\tau = 0$. Such a profile may still be evaluated by T_τ^{bar} , and the bar is deleted because deletion uses the rule $\ell(I) \leq \tau$. However, the profile is not positively gap-ready at threshold τ . It cannot support positive-radius metric transport through hard deletion at that threshold. A separately proved exact identity may still be recorded in exact comparison mode, but it is not a robustness certificate.

2.9 Threshold-gap-safe stability

Theorem 2.33 (Metric-repair theorem: sharp threshold-gap-safe stability). *Let B and C be barcodes with finitely many bars, possibly including infinite bars, interpreted under the bottleneck convention of Definition 2.12. Let $\tau > 0$ and $\varepsilon \geq 0$. Assume*

$$d_B(B, C) \leq \varepsilon.$$

Assume also that no finite bar of either B or C has length in the closed threshold band

$$[\tau - \varepsilon, \tau + \varepsilon].$$

Then

$$d_B(T_\tau^{\text{bar}}(B), T_\tau^{\text{bar}}(C)) \leq \varepsilon.$$

Proof. First consider the degenerate case $\varepsilon = 0$. Then $d_B(B, C) = 0$, so B and C are identical as barcode profiles under the stated barcode convention. The closed threshold band is the singleton $\{\tau\}$, and the hypothesis says that no finite bar has length exactly τ . Applying the same deterministic deletion rule to identical barcodes gives

$$T_\tau^{\text{bar}}(B) = T_\tau^{\text{bar}}(C),$$

hence the conclusion follows.

Now assume $\varepsilon > 0$. Choose a bottleneck matching between B and C of cost at most ε . Such a matching exists for the finite barcode profiles under consideration. We construct a matching between the retained barcodes after applying T_τ^{bar} by modifying the chosen matching only by deleting matched pairs that are removed on both sides.

Let $I \in B$ be a bar retained by T_τ^{bar} . If I is infinite, then by Definition 2.12 it cannot be matched at finite cost to the diagonal or to a finite bar. Hence it is matched to an infinite bar of C , which is also retained.

Suppose next that I is finite. Since I is retained, $\ell(I) > \tau$. Because no finite bar of B has length in $[\tau - \varepsilon, \tau + \varepsilon]$, this implies

$$\ell(I) > \tau + \varepsilon.$$

There are two cases.

If I is matched to the diagonal with cost at most ε , then the same diagonal match may be kept after threshold deletion. It contributes cost at most ε to the retained-barcode matching. This case is possible when $\ell(I)/2 \leq \varepsilon$, and it is the reason the proof must not assume that every retained finite bar has a finite matched partner.

If I is matched to a finite bar $J \in C$, then the endpoint displacement bound gives

$$|\ell(I) - \ell(J)| \leq 2\varepsilon.$$

If J were deleted, then $\ell(J) \leq \tau$. Since C also avoids $[\tau - \varepsilon, \tau + \varepsilon]$, this would force

$$\ell(J) < \tau - \varepsilon.$$

Together with $\ell(I) > \tau + \varepsilon$, this gives

$$\ell(I) - \ell(J) > 2\varepsilon,$$

contradicting $|\ell(I) - \ell(J)| \leq 2\varepsilon$. Therefore J is retained. Finite-to-infinite matching is impossible at finite cost by Definition 2.12.

The same argument, with B and C interchanged, shows that every retained finite bar of C that is not kept on the diagonal is matched to a retained finite bar of B , while every retained infinite bar is matched to a retained infinite bar. Hence the chosen matching restricts, with the permitted retained-to-diagonal matches, to a bottleneck matching between $T_\tau^{\text{bar}}(B)$ and $T_\tau^{\text{bar}}(C)$ of cost at most ε . Thus

$$d_B(T_\tau^{\text{bar}}(B), T_\tau^{\text{bar}}(C)) \leq \varepsilon.$$

□

Example 2.34 (Sharpness of the ε -band). The constant ε in the two-sided closed threshold band is sharp. Let $0 < c < \varepsilon$, choose $0 < \eta < \varepsilon - c$, and take τ large enough that the diagonal costs below exceed ε . Set

$$L_+ := \tau + c + \eta, \quad L_- := \tau - c - \eta,$$

and consider the centered finite bars

$$I = \left[-\frac{L_+}{2}, \frac{L_+}{2}\right), \quad J = \left[-\frac{L_-}{2}, \frac{L_-}{2}\right).$$

Then both lengths avoid the smaller closed band $[\tau - c, \tau + c]$, while

$$d_B(\{I\}, \{J\}) = c + \eta < \varepsilon.$$

However I is retained and J is deleted by T_τ^{bar} . Hence

$$T_\tau^{\text{bar}}(\{I\}) = \{I\}, \quad T_\tau^{\text{bar}}(\{J\}) = \emptyset.$$

For τ sufficiently large, $L_+/2 > \varepsilon$, so

$$d_B(T_\tau^{\text{bar}}(\{I\}), T_\tau^{\text{bar}}(\{J\})) = \frac{L_+}{2} > \varepsilon.$$

Thus no two-sided closed band of half-width $c < \varepsilon$ suffices in general. The sharp two-sided half-width is exactly ε .

Remark 2.35 (Two-sided certificate). The threshold-gap certificate is two-sided. It is not enough for B alone to avoid the threshold band. If C contains a finite bar in $[\tau - \varepsilon, \tau + \varepsilon]$, then a matched pair may cross the deletion threshold and change retained/deleted status. For one-sided certificates, a 2ε -type margin is a natural conservative constant; it is not the sharp constant for the two-sided v19 Core theorem.

Corollary 2.36 (Scalarized-ledger-safe threshold classification). *Let B and C be finite barcode profiles, possibly with infinite bars. Let $\mathfrak{d}_{\text{met}} \in \mathcal{V}$ be a declared metric ledger and set*

$$\varepsilon_B := \text{val}_B(\mathfrak{d}_{\text{met}}).$$

Suppose

$$d_B(B, C) \leq \varepsilon_B$$

and

$$\varepsilon_B < \text{gap}_\tau(B, C).$$

Then

$$d_B(T_\tau^{\text{bar}}(B), T_\tau^{\text{bar}}(C)) \leq \varepsilon_B.$$

Proof. The strict inequality

$$\varepsilon_B < \text{gap}_\tau(B, C) = \min\{\text{clear}_\tau(B), \text{clear}_\tau(C)\}$$

implies that no finite bar of either barcode has length in

$$[\tau - \varepsilon_B, \tau + \varepsilon_B].$$

Apply Theorem 2.33 with $\varepsilon = \varepsilon_B$. □

2.10 Window restriction and window-edge budget

Definition 2.37 (Right-open window). A domain window is a right-open interval

$$W = [u, u'),$$

where $u \in \mathbb{R}$, $u' \in \mathbb{R} \cup \{+\infty\}$, and $u < u'$. If $u' < +\infty$, W is called a bounded audit window. If $u' = +\infty$, W is called a terminal audit window.

Definition 2.38 (Window restriction). For $W = [u, u')$, define the window restriction

$$W_W(M)$$

by restricting M to W , and, when a globally indexed persistence module is required, extending by zero outside the declared window.

At the barcode level, W_W clips each bar I to

$$I \cap W,$$

discarding empty intersections.

Remark 2.39 (Infinite bars under window restriction). The infinite-bar convention of Definition 2.12 applies before windowing and on terminal windows $W = [u, \infty)$. On a bounded audit window $W = [u, u')$ with $u' < \infty$, an infinite bar $[a, \infty)$ is clipped to

$$[\max\{a, u\}, u')$$

whenever the intersection is nonempty. Thus it becomes a finite bar inside the bounded window and is then thresholded according to its clipped finite length. On a terminal audit window $W = [u, \infty)$, it remains infinite and is never deleted by T_τ^{bar} .

Proposition 2.40 (No unconditional crop-collapse commutation). *In general,*

$$W_W T_\tau^{\text{bar}}(B) \neq T_\tau^{\text{bar}} W_W(B).$$

Proof. Let $B = \{[0, 10]\}$, let $W = [0, 1)$, and take $\tau = 2$. Since the global bar has length $10 > \tau$, it survives T_τ^{bar} . Therefore

$$W_W T_\tau^{\text{bar}}(B) = \{[0, 1)\}.$$

On the other hand,

$$W_W(B) = \{[0, 1)\},$$

whose length is $1 \leq \tau$. Thus

$$T_\tau^{\text{bar}} W_W(B) = \emptyset.$$

Hence the two operations do not commute in general. $\square$

Definition 2.41 (Window-edge clearance). A window-edge clearance record for a source-level perturbation problem records a declared margin between the bar endpoints or critical parameters relevant to $\mathbf{P}_i(F)$ and the window boundary points u, u' . Its purpose is to certify that passage from source-level perturbation to windowed pre-threshold profile comparison does not silently gain or lose bars at the crop boundary.

Definition 2.42 (Crop budget). A crop budget is a declared ledger component

$$\delta_{\text{crop}}(i, W)$$

bounding the discrepancy introduced when moving from a source-level comparison to a comparison of windowed pre-threshold profiles:

$$B_{i,W}(F) = W_W \mathbf{P}_i(F).$$

Whenever a proof uses a source-level perturbation statement to infer

$$d_B(B_{i,W}(F), B_{i,W}(G)) \leq \varepsilon,$$

the contribution of window restriction, boundary clipping, endpoint uncertainty, and crop instability must either be proved zero by a window-edge clearance record or included in the ledger as $\delta_{\text{crop}}(i, W)$.

Remark 2.43 (Gap-safe stability consumes windowed profile distance). Theorem 2.33 does not assert that a raw perturbation of filtered objects automatically gives a bottleneck bound after window restriction. It consumes the hypothesis

$$d_B(B, C) \leq \varepsilon$$

for the barcode profiles to which threshold deletion is applied. In AK applications, those profiles are the windowed pre-threshold profiles

$$B_{i,W}(F) = W_W \mathbf{P}_i(F).$$

Therefore window-edge stability belongs to the ledger and is separate from threshold-gap stability.

Example 2.44 (Infinite-bar obstruction to the former finite-only condition). Let

$$B = \{[0, +\infty)\}, \quad W = [0, 1), \quad \tau = 2.$$

The infinite bar is retained by global threshold deletion, so

$$W_W T_\tau^{\text{bar}}(B) = \{[0, 1)\}.$$

After windowing first, however, the bar has finite clipped length $1 \leq \tau$, and hence

$$T_\tau^{\text{bar}} W_W(B) = \emptyset.$$

Thus any commutation criterion that checks only globally retained finite bars is insufficient: globally retained infinite bars must also be tested after clipping to a bounded window.

Definition 2.45 (τ -window-adapted condition). A pair (B, W) is called τ -window-adapted if every bar $I \in B$ retained by T_τ^{bar} , including every infinite bar, satisfies one of the following:

(i) $I \cap W = \emptyset$;

(ii) the clipped bar $I \cap W$ is retained by threshold deletion on the window, equivalently

$$\ell(I \cap W) > \tau \quad \text{or} \quad \ell(I \cap W) = +\infty.$$

For a bounded window, clause (ii) requires every nonempty crop of a globally retained infinite bar to have finite clipped length strictly greater than τ . For a terminal window, a nonempty crop of an infinite bar remains infinite and automatically satisfies clause (ii).

Proposition 2.46 (Exact crop–collapse commutation criterion). *For a barcode profile B , a right-open window W , and $\tau > 0$, the following are equivalent:*

(i) (B, W) is τ -window-adapted;

(ii)

$$W_W T_\tau^{\text{bar}}(B) = T_\tau^{\text{bar}} W_W(B)$$

as barcode multisets, including multiplicities.

Proof. Assume first that (B, W) is τ -window-adapted. If a finite bar I is deleted globally, then $\ell(I) \leq \tau$; clipping cannot increase length, so every nonempty crop $I \cap W$ is also deleted after windowing. If a bar I is retained globally, the adaptedness condition says that it is either disjoint from W or its nonempty crop is retained after windowing. Thus every bar contributes with the same multiplicity in both orders.

Conversely, suppose the two barcode multisets are equal and let I be retained globally with $I \cap W \neq \emptyset$. If $I \cap W$ were a finite bar of length at most τ , then the occurrence of $I \cap W$ contributed by I would appear in $W_W T_\tau^{\text{bar}}(B)$ but would be deleted from $T_\tau^{\text{bar}} W_W(B)$. The multiplicities would differ, contradicting equality. Hence the crop is retained, and (B, W) is τ -window-adapted. $\square$

Remark 2.47 (Default policy and audit use). The default Core policy is not to assume τ -window-adaptedness. Gate-relevant quantities are evaluated by the fixed order

$$\mathbf{P}_i(F) \longmapsto W_W \mathbf{P}_i(F) \longmapsto T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Conditional commutation may be used only when Definition 2.45 is verified for all globally retained bars, including infinite bars, and the verification is recorded in the manifest.

2.11 Core windowed evaluation

Definition 2.48 (Windowed collapsed persistence evaluation). Let $F \in \text{FiltCh}((k))$, let i be a monitored degree, let W be a declared right-open window, and let $\tau > 0$. The Core windowed collapsed evaluation is

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)) = T_\tau^{\text{bar}}(B_{i,W}(F)).$$

Remark 2.49 (Fixed order). The expression

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F))$$

means the following ordered pipeline:

$$F \mapsto \mathbf{P}_i(F) \mapsto B_{i,W}(F) = W_W \mathbf{P}_i(F) \mapsto T_\tau^{\text{bar}}(B_{i,W}(F)).$$

The Core first restricts to the declared window and only then applies hard threshold deletion.

Definition 2.50 (Repaired evaluation record). A repaired evaluation record for $(F; i, W, \tau)$ contains:

- (i) immutable source-object and profile identifiers;
- (ii) the monitored degree i , declared window W , and threshold τ ;
- (iii) the windowed pre-threshold profile $B_{i,W}(F)$;
- (iv) the repaired profile

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(B_{i,W}(F));$$

- (v) the comparison mode, chosen from

`exact, metric_gap_certified, not_compared;`

- (vi) reconstructible artifact references.

Definition 2.51 (Threshold-gap-certified repaired evaluation comparison). A metric comparison between two repaired evaluation records is *threshold-gap-certified* only when the compared windowed pre-threshold profiles are exactly the profiles appearing in a valid two-sided record of Definition 2.31. The record must establish both

$$d_B(B, C) \leq \varepsilon_B$$

and

$$\varepsilon_B < \text{gap}_\tau(B, C).$$

The associated comparison mode is `metric_gap_certified`. A separately proved equality or profile isomorphism is recorded instead with comparison mode `exact`; it does not require a fabricated positive-radius gap certificate and does not by itself assert robustness under perturbation.

Remark 2.52 (Evaluation is definable without certification). The profile $\text{Eval}_{i,W,\tau}(F)$ is definitionally available once the windowed barcode profile is given. Whenever a metric discrepancy is transported through hard deletion—including metric overlap, Restart, representative, or tower comparisons—the stronger two-sided threshold-gap comparison record is required. A separately proved exact equality or profile isomorphism may instead be used in `exact` mode; such an exact record is not automatically a positive-radius robustness certificate.

2.12 Overlap Gate: post-threshold comparison and pre-threshold gap

The Overlap Gate has two different metric layers. This distinction is essential in v19.

Definition 2.53 (Overlap pre-threshold profiles). For two local inputs F_α, F_β and an overlap window

$$W_{\alpha\beta} := W_\alpha \cap W_\beta,$$

define the overlap pre-threshold profiles by

$$B_{i,W_{\alpha\beta}}^\alpha := \mathbb{W}_{W_{\alpha\beta}} \mathbf{P}_i(F_\alpha), \quad B_{i,W_{\alpha\beta}}^\beta := \mathbb{W}_{W_{\alpha\beta}} \mathbf{P}_i(F_\beta).$$

The overlap repaired profiles are

$$\text{Eval}_{i,W_{\alpha\beta},\tau}(F_\alpha) = T_\tau^{\text{bar}}(B_{i,W_{\alpha\beta}}^\alpha), \quad \text{Eval}_{i,W_{\alpha\beta},\tau}(F_\beta) = T_\tau^{\text{bar}}(B_{i,W_{\alpha\beta}}^\beta).$$

Remark 2.54 (Two-level structure of the Overlap Gate). The metric comparison clause of the Overlap Gate compares the *post-threshold* repaired profiles:

$$d_{\text{int}}(\text{Eval}_{i,W_{\alpha\beta},\tau}(F_\alpha), \text{Eval}_{i,W_{\alpha\beta},\tau}(F_\beta)).$$

By contrast, the threshold-gap clause is computed on the *pre-threshold* profiles:

$$B_{i,W_{\alpha\beta}}^\alpha, \quad B_{i,W_{\alpha\beta}}^\beta.$$

Thus the gate deliberately uses two barcode levels:

gap is measured before deletion, agreement is checked after deletion.

Proposition 2.55 (Overlap transfer under a scalarized threshold-gap certificate). *Let $\mathfrak{d}_{\alpha\beta,i}$ be the declared overlap metric ledger and set*

$$\varepsilon_{\alpha\beta,i} := \text{val}_B(\mathfrak{d}_{\alpha\beta,i}).$$

Suppose

$$d_B(B_{i,W_{\alpha\beta}}^\alpha, B_{i,W_{\alpha\beta}}^\beta) \leq \varepsilon_{\alpha\beta,i}.$$

If the two-sided threshold-gap record satisfies

$$\varepsilon_{\alpha\beta,i} < \text{gap}_\tau(B_{i,W_{\alpha\beta}}^\alpha, B_{i,W_{\alpha\beta}}^\beta),$$

then

$$d_B(\text{Eval}_{i,W_{\alpha\beta},\tau}(F_\alpha), \text{Eval}_{i,W_{\alpha\beta},\tau}(F_\beta)) \leq \varepsilon_{\alpha\beta,i}.$$

Equivalently, by the bottleneck–interleaving isometry in the declared constructible regime, the same bound may be read using d_{int} .

Proof. This is Corollary 2.36 applied to the two overlap pre-threshold profiles and the overlap metric ledger $\mathfrak{d}_{\alpha\beta,i}$. The final sentence follows from Remark 2.14. $\square$

2.13 Exact-admissible regimes

The general Core does not treat T_τ^{bar} as a global exact functor. Nevertheless, stronger exact calculus may be available under separately declared hypotheses.

Definition 2.56 (τ -split object). An object $M \in \text{Pers}_k^{\text{cons}}$ is called τ -split if it is equipped with a declared decomposition

$$M \cong M_{\leq \tau} \oplus M_{> \tau}$$

such that:

- (i) every finite bar of $M_{\leq \tau}$ has length at most τ ;
- (ii) every finite bar of $M_{> \tau}$ has length greater than τ ;
- (iii) all infinite bars, if any, lie in $M_{> \tau}$;
- (iv) the decomposition is part of the declared data, not merely inferred informally.

Definition 2.57 (τ -split morphism). A morphism

$$f : M \rightarrow N$$

between τ -split objects is called τ -split if it respects the declared decompositions:

$$f(M_{\leq \tau}) \subseteq N_{\leq \tau}, \quad f(M_{> \tau}) \subseteq N_{> \tau}.$$

Definition 2.58 (τ -split regime). A τ -split regime is a declared working subcategory or workflow domain in which all objects and all morphisms under consideration are τ -split in the above sense.

Proposition 2.59 (Functorial deletion in a τ -split regime). *Inside a declared τ -split regime, the assignment*

$$M \mapsto M_{> \tau}$$

is functorial and agrees with $T_\tau^{\text{bar}}(M)$ on barcode profiles.

Proof. Since every morphism respects the declared short/long decomposition, it restricts to a morphism

$$M_{> \tau} \rightarrow N_{> \tau}.$$

Therefore $M \mapsto M_{> \tau}$ is functorial in the τ -split regime. By construction, $M_{> \tau}$ consists exactly of the bars retained by T_τ^{bar} . $\square$

Definition 2.60 (Exact-admissible collapse regime). An exact-admissible collapse regime is a declared τ -split regime in which the retained-long-bar assignment

$$M \mapsto M_{> \tau}$$

is additionally verified to preserve the exact sequences used in the relevant Core argument.

Remark 2.61 (Exactness is conditional). Exactness of positive-threshold collapse is not a global Core theorem. It is a conditional property available only in an exact-admissible regime. Whenever such a regime is used, the proof object must declare:

- (i) the τ -split decomposition policy;
- (ii) the morphism compatibility policy;
- (iii) the exact sequences being tested;
- (iv) the artifact or proof establishing exact preservation.

2.14 Filtered representatives of windowed collapse

Because T_τ^{bar} is treated in the general Core as a barcode-level operation, filtered-level collapse must be phrased in terms of representatives.

Definition 2.62 (Windowed collapsed persistence profile). For $F \in \text{FiltCh}(()k)$, monitored degree i , window W , and threshold τ , define the monitored windowed collapsed profile by

$$\mathcal{P}_{i,W,\tau}(F) := \text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(B_{i,W}(F)).$$

Definition 2.63 (Admissible filtered representative). Let $F \in \text{FiltCh}(()k)$, let W be a declared window, and let $\tau > 0$. A filtered object

$$C_{\tau,W}F$$

is called an admissible filtered representative of the windowed collapse of F if, for every monitored degree i ,

$$\mathbf{P}_i(C_{\tau,W}F) \cong \mathcal{P}_{i,W,\tau}(F)$$

at the persistence barcode level, up to the declared representative equivalence policy.

Remark 2.64 (No global strict lift by default). The general Core does not assert a strict global endofunctor

$$C_\tau : \text{FiltCh}(()k) \rightarrow \text{FiltCh}(()k)$$

realizing positive-threshold collapse in every degree and on every morphism. The default object is the windowed representative $C_{\tau,W}F$, whose existence and properties must be declared in the proof object or implementation record.

Definition 2.65 (Representative equivalence). Two admissible representatives $C_{\tau,W}F$ and $C'_{\tau,W}F$ are considered equivalent for Core gate purposes if their monitored persistence profiles agree:

$$\mathbf{P}_i(C_{\tau,W}F) \cong \mathbf{P}_i(C'_{\tau,W}F)$$

for all monitored degrees i , under the declared barcode equivalence policy.

Remark 2.66 (Filtered quasi-isomorphism). When filtered quasi-isomorphism data are available, representative equivalence may be strengthened to filtered quasi-isomorphism. However, filtered quasi-isomorphism is not required merely to define the windowed collapsed barcode profile. It becomes necessary only when the argument uses filtered-level or derived-level constructions.

2.15 Consequences for $B\text{-Gate}^+$

The repaired Chapter 2 affects how the Core gate is read.

Definition 2.67 (Windowed B -side input). The B -side input for a monitored degree i , window W , and threshold τ is the collapsed profile

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(B_{i,W}(F)),$$

or an admissible filtered representative $C_{\tau,W}F$ realizing this profile.

Definition 2.68 (Windowed topological clause). The topological clause of $B\text{-Gate}^+$ on $(F; W, \tau)$ is read as

$$\text{Eval}_{1,W,\tau}(F) = 0$$

at the monitored persistence-profile level. Equivalently, if an admissible representative $C_{\tau,W}F$ is supplied and realizes the declared profile, the clause may be read as

$$\text{PH}_1(C_{\tau,W}F) = 0.$$

Definition 2.69 (Windowed categorical clause). The categorical clause of $B\text{-Gate}^+$ may be invoked only when an admissible filtered representative $C_{\tau,W}F$ is supplied and the eligible realization condition holds:

$$R(C_{\tau,W}F) \in D^{[-1,0]}(k\text{-mod}).$$

Only then is the Ext-clause

$$\text{Ext}^1(R(C_{\tau,W}F), k) = 0$$

well-typed for the Core bridge.

Definition 2.70 (Comparison-typed $B\text{-Gate}^+$ input). A $B\text{-Gate}^+$ input records:

- (i) the post-threshold repaired profile $\text{Eval}_{1,W,\tau}(F)$;
- (ii) the pre-threshold profile $B_{1,W}(F)$;
- (iii) the comparison mode for every comparison involving that profile;
- (iv) for `metric_gap_certified` mode, the two compared pre-threshold profiles, their two-sided threshold-gap comparison record, and every ledger component needed to justify the actual bottleneck bound, including any crop budget;
- (v) for `exact` mode, the equality or profile-isomorphism proof and its artifact reference.

A single-profile clearance value is readiness information only and is not substituted for a two-sided metric comparison record.

Remark 2.71 (PH before Ext). The barcode-level profile $\text{Eval}_{1,W,\tau}(F)$ is the primary persistence datum. The Ext-clause is not a replacement for it. Ext is read only through an eligible representative and the one-way bridge established later.

2.16 Tower comparison and typed Type IV input

Since T_{τ}^{bar} is not a global strict exact functor in the general Core, tower comparison maps must be declared rather than assumed. The full terminal-generic diagnostic is defined in Chapter 5; the present section fixes only the well-typed input contract.

Definition 2.72 (Windowed collapsed tower objects). Let

$$F_0 \rightarrow F_1 \rightarrow F_2 \rightarrow \dots$$

be a directed tower in the declared filtered or persistence regime. For a monitored degree i , window W , and threshold τ , write

$$M_{n;i,W,\tau} := \text{Eval}_{i,W,\tau}(F_n).$$

The sequence of objects $M_{n;i,W,\tau}$ is not by itself a directed system: actual profile morphisms and their composition laws must be declared, unless supplied by a τ -split or exact-admissible regime.

Definition 2.73 (Tower comparison artifact). A tower comparison artifact for (i, W, τ) is declared data specifying:

- (i) a repaired tower-profile system with actual persistence-profile morphisms;
- (ii) a source profile $\text{ColimProfile}_{i,W,\tau}(F_{\bullet})$ with declared provenance;

(iii) an apex profile $\text{ApexProfile}_{i,W,\tau}(F_\infty)$;

(iv) an actual comparison morphism

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty);$$

(v) the typing, kernel/cokernel, terminal-tameness, and artifact data required by Chapter 5.

A sequence of barcode multisets or bottleneck matchings without actual profile morphisms is not a tower comparison artifact.

Remark 2.74 (No automatic tower comparison). The comparison morphism $\phi_{i,W,\tau}$ is not produced automatically by hard threshold deletion, by a barcode matching, or by the existence of a source tower and an apex object. It must be explicitly constructed and typed, or supplied canonically by a declared exact-admissible regime.

Definition 2.75 (Typed Type IV scope status). Every Chapter 2 run records exactly one Type IV scope status:

$$\text{not_invoked}, \quad \text{undefined}, \quad \text{certified_zero}, \quad \text{obstructed}.$$

They mean respectively:

- (i) no tower, apex, limit, or colimit comparison belongs to the claim;
- (ii) tower semantics are invoked, but the artifact or a required terminal-generic input is missing or ill-typed;
- (iii) a valid artifact exists and the Chapter 5 terminal-generic kernel/cokernel diagnostic is zero;
- (iv) a valid artifact exists and that diagnostic is nonzero.

A missing artifact is undefined, never a numerical zero and never not_invoked unless the scope policy explicitly excludes tower semantics.

Definition 2.76 (Collapse diagnostics notation contract). For a valid tower comparison artifact whose kernel and cokernel defect profiles are terminal-tame on W , Chapter 5 defines

$$\mu_{i,W,\tau} := \text{tgdim}_W(\ker \phi_{i,W,\tau}), \quad \nu_{i,W,\tau} := \text{tgdim}_W(\text{coker } \phi_{i,W,\tau}).$$

On a right-unbounded window, tgdim_W is the eventual generic dimension and counts defect bars cofinal to $+\infty$. On a bounded window, it is the left limit of the dimension at the right endpoint and counts defect bars cofinal to that endpoint; it must not be replaced by the dimension after zero extension to $+\infty$. For a fixed declared pair (W, τ) and a declared finite monitored degree set I_{mon} , define

$$\mu_{\text{Collapse}} := \sum_{i \in I_{\text{mon}}} \mu_{i,W,\tau}, \quad \nu_{\text{Collapse}} := \sum_{i \in I_{\text{mon}}} \nu_{i,W,\tau}.$$

An infinite monitored scope requires a separately declared aggregation or summability theorem. The legacy symbols $u_{i,W,\tau}$ and u_{Collapse} are deprecated; $\nu_{i,W,\tau}$ and ν_{Collapse} are the sole canonical cokernel-diagnostic notation in v19.0.0.

Remark 2.77 (Type IV remains Core, but zero is typed). Type IV remains part of the Core gate discipline, but its comparison map is not silently inherited from threshold deletion. The shorthand

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$$

may be used only under status `certified_zero`. Under `obstructed`, a valid diagnostic is nonzero. Under `undefined` or `not_invoked`, no numerical pair is assigned. In particular, missing or excluded tower data are never represented by a fabricated zero pair. The zero pair means absence of monitored terminal-generic kernel and cokernel defect relative to the declared artifact; it does not imply that the comparison morphism is an isomorphism on finite interior bars.

2.17 Manifest consequences

A certified proof object using Chapter 2 must record the following data.

Definition 2.78 (Collapse policy manifest entry). A collapse policy manifest entry contains:

- (i) the threshold τ , monitored degrees i , and window W ;
- (ii) the window type: bounded audit window or terminal audit window;
- (iii) the evaluation order

$$\mathbf{P}_i \rightarrow W_W \mathbf{P}_i \rightarrow T_\tau^{\text{bar}}(W_W \mathbf{P}_i);$$
- (iv) the collapse status

$$\text{barcode_level}, \quad \text{tau_split}, \quad \text{exact_admissible};$$
- (v) the repaired evaluation record and comparison mode for every comparison;
- (vi) for each metric discrepancy transported through hard deletion, the two-sided threshold-gap comparison record; for each exact comparison, the exact equality or profile-isomorphism proof;
- (vii) the metric ledger $\mathfrak{d}_{\text{met}}$, bottleneck scalarization val_B , and scalarized budget ε_B whenever metric mode is invoked;
- (viii) window-edge clearance or crop-budget data when source-level perturbations are transported through window restriction;
- (ix) any τ -window-adapted commutation certificate, including explicit verification of globally retained infinite bars on bounded windows;
- (x) whether an admissible filtered representative $C_{\tau, W} F$ is supplied;
- (xi) the typed Type IV status and, if invoked, the tower comparison artifact, terminal-generic inputs, monitored degree set, and canonical μ/ν diagnostics;
- (xii) the repair record documenting the replacement of global non-expansiveness by threshold-gap-safe stability, the repair of the former finite-only crop condition, and the migration from legacy u to canonical ν notation.

Definition 2.79 (Metric and crop repair record). A Chapter 2 repair record contains:

- (i) the deprecated claim being avoided: global bottleneck non-expansiveness of T_τ^{bar} ;
- (ii) the replacement theorem: Theorem 2.33;
- (iii) the counterexample reference: Example 2.22;
- (iv) the relevant pre-threshold profiles and two-sided threshold-gap certificate;
- (v) the metric ledger, scalarization, and typed inequality

$$\text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau;$$

- (vi) any crop budget required to obtain the windowed pre-threshold profile comparison;

- (vii) if crop–collapse commutation is used, the exact criterion of Proposition 2.46;
- (viii) explicit confirmation that globally retained infinite bars were included in that criterion;
- (ix) the Type IV scope status and artifact reference when tower semantics are invoked;
- (x) confirmation that $\nu_{i,W,\tau}$ and ν_{Collapse} are used canonically and that legacy u -notation is confined to deprecation metadata.

Remark 2.80 (Audit consequence). A proof object that simply states “apply T_τ ” is incomplete under v19. It must state whether T_τ means barcode-level hard threshold deletion, a τ -split functorial model, or an exact-admissible collapse regime. If a metric discrepancy is transported through hard deletion, it must cite a two-sided scalarized threshold-gap comparison record rather than global non-expansiveness. A separately proved exact comparison is recorded in `exact` mode with its proof and is not misrepresented as a positive-radius gap certificate. If crop–collapse commutation is used, it must verify all globally retained bars, including infinite bars clipped by bounded windows. If Type IV is invoked, it must provide an actual comparison morphism and the typed terminal-generic diagnostic input of Chapter 5.

2.18 Operating summary

For each monitored object F , degree i , window W , and threshold τ , the repaired Chapter 2 pipeline is

$$F \longmapsto \mathbf{P}_i(F) \longmapsto B_{i,W}(F) = W_W \mathbf{P}_i(F) \longmapsto T_\tau^{\text{bar}}(B_{i,W}(F)) = \text{Eval}_{i,W,\tau}(F).$$

Threshold clearance is measured on $B_{i,W}(F)$, not on $\text{Eval}_{i,W,\tau}(F)$. The stability theorem consumes a real pre-threshold bottleneck bound

$$d_B(B, C) \leq \varepsilon.$$

When the bound comes from an abstract ledger, the certified metric chain is

$$\mathfrak{d}_{\text{met}} \xrightarrow{\text{val}_B} \varepsilon_B \geq d_B(B, C), \quad \varepsilon_B < \text{gap}_\tau(B, C).$$

This chain is two-sided and comparison-specific. A single-profile clearance is only readiness information. A separately proved exact equality or profile isomorphism uses `exact` mode instead. A source-level perturbation does not automatically yield the windowed bound; crop and endpoint effects require their own theorem or ledger component.

The default evaluation order is window first, threshold second. The two operations commute exactly under Proposition 2.46, whose criterion tests every globally retained bar, including infinite bars clipped to bounded windows.

Filtered or derived arguments require an admissible representative $C_{\tau,W}F$. Tower diagnostics require a repaired tower-profile system, an actual comparison morphism, terminal-tameness data, and a typed Type IV status. Exactness or strict functoriality requires a declared τ -split or exact-admissible regime.

2.19 Explicit non-claims

The following are not asserted in this chapter.

- (i) No global non-expansiveness of hard positive-threshold deletion under the standard bottleneck metric is asserted.
- (ii) No claim is made that positive-threshold short-bar objects form a Serre subcategory in general.

- (iii) No claim is made that T_τ^{bar} is a global exact endofunctor on all of $\text{Pers}_k^{\text{cons}}$.
- (iv) No claim is made that T_τ^{bar} is a global abelian reflector or Serre localization.
- (v) No unconditional commutation

$$W_W T_\tau^{\text{bar}} = T_\tau^{\text{bar}} W_W$$
 is asserted.
- (vi) No finite-only window-adapted condition is sufficient: globally retained infinite bars must also be checked after clipping.
- (vii) No raw source-level perturbation implies a windowed pre-threshold profile bound without a crop-stability argument or crop budget.
- (viii) No unscaled quantale element is compared directly with the real-valued clearance $\text{gap}_\tau = \text{clear}_\tau$.
- (ix) No numerical budget repairs a missing representative, comparison morphism, eligibility certificate, terminal-tameness certificate, claim status, or manifest record.
- (x) No threshold-gap certificate is one-sided.
- (xi) No single-profile clearance or gap-readiness record is treated as a two-sided metric comparison certificate.
- (xii) No finite bar of length exactly τ is treated as gap-safe at threshold τ .
- (xiii) No 2ε -band is claimed to be sharp for the two-sided theorem, and no factor convention $\text{gap}_\tau = \frac{1}{2} \text{clear}_\tau$ is used.
- (xiv) No global strict filtered lift $C_\tau : \text{FiltCh}(()k) \rightarrow \text{FiltCh}(()k)$ is asserted by default.
- (xv) No tower comparison map after threshold deletion is automatic unless an actual artifact or exact-admissible regime is declared.
- (xvi) No barcode matching by itself defines the kernel or cokernel of a tower comparison.
- (xvii) No undefined Type IV diagnostic is interpreted as zero, and no no-tower claim fabricates a numerical zero instead of recording `not_invoked`.
- (xviii) No equality $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ implies full comparison-map isomorphism without a no-interior-defect theorem.
- (xix) No legacy symbol $u_{i,W,\tau}$ or u_{Collapse} has normative diagnostic meaning in v19.0.0.
- (xx) No bounded-window cofinal bar is called a global infinite bar.
- (xxi) No Ext-clause is well-typed without an admissible filtered representative and an eligible realization condition.
- (xxii) No Search artifact, advisory invariant, spectral indicator, or AI-generated ranking replaces the windowed collapsed persistence evaluation or any typed proof obligation.

2.20 Provenance and status

The constructible persistence setting, interval decomposition, barcode uniqueness, bar-length convention, and bottleneck/interleaving isometry are standard persistence-theoretic facts in the one-parameter constructible regime over a field, interpreted here under Definition 2.12.

The v19-specific Core content of this chapter is the metric and crop repair: hard positive-threshold deletion is not globally non-expansive under the standard bottleneck metric; its admissible stability use is governed by sharp two-sided threshold-gap-safe stability; and crop–collapse commutation is governed by the exact all-retained-bars criterion of Proposition 2.46, including infinite bars clipped by bounded windows. Ledger use is typed through a bottleneck scalarization and a comparison-specific discrepancy proof. Type IV input is forwarded to the terminal-generic discipline of Chapter 5 using the canonical μ/ν notation and a declared monitored scope.

Status labels for this chapter. *Standard persistence theorem:* Theorem 2.9.

Metric-repair theorem: Theorem 2.33, Corollary 2.36.

Core correction results: Proposition 2.15, Corollary 2.16, Example 2.22, Corollary 2.24, Proposition 2.40, Example 2.44, Proposition 2.46.

Core definitions: Definitions 2.3, 2.5, 2.8, 2.12, 2.18, 2.25, 2.26, 2.27, 2.29, 2.45, 2.48, 2.63, 2.73, 2.75, 2.76.

Operational policy and records: Definitions 2.31, 2.41, 2.42, 2.50, 2.51, 2.78, 2.79; Remarks 2.49, 2.47, 2.74, 2.77.

Exact-admissible optional regime: Definitions 2.56, 2.57, 2.58, 2.60; Proposition 2.59.

Explicit exclusions: Subsection 2.19.

3 Chapter 3: Windowed Evaluation, Threshold-Gap Records, and Representative Discipline

Chapter 3 contract. This chapter is the operational middle layer between the metric repair of Chapter 2 and Appendix A and the eligible realization and Ext-edge of Chapter 4. It fixes how windowed barcode data are recorded, distinguishes single-profile threshold readiness from two-sided metric certification, and specifies when a filtered representative may be consumed by later layers.

The chapter does *not* prove the one-way bridge

$$\text{PH}_1 \implies \text{Ext}^1.$$

That bridge belongs to Chapter 4. The present chapter supplies the repaired evaluation, comparison, crop, and representative records that Chapter 4 may consume.

All numerical metric budgets are typed through a declared bottleneck scalarization

$$\text{val}_B : V \longrightarrow [0, +\infty].$$

Every metric transport through hard deletion uses either a two-sided threshold-gap comparison record or a separately proved exact comparison. A single-profile clearance record is readiness data only. Exact comparison is not automatically a positive-radius robustness certificate.

All non-scalar obligations—representative existence, realization data, actual comparison morphisms,

eligibility, claim status, manifest consistency, and artifact identity—remain independent proof obligations. The Chapter 3 record extension is subordinate to the canonical audit manifest of Chapter 7.

3.1 Scope and standing discipline

All statements in this chapter are made in the v19.0.0 Threshold-Gap Metric Repair regime fixed by Chapters 1–2 and Appendix A.

In particular:

- (i) persistence data live in constructible one-parameter persistence over a field k ;
- (ii) barcode-level threshold deletion T_τ^{bar} deletes precisely finite bars of length $\leq \tau$ and preserves infinite bars before bounded-window clipping;
- (iii) global non-expansiveness of T_τ^{bar} under the standard bottleneck metric is not asserted;
- (iv) the replacement stability principle is sharp two-sided threshold-gap-safe stability with closed band

$$[\tau - \varepsilon, \tau + \varepsilon];$$

- (v) the ledger margin is

$$\text{gap}_\tau = \text{clear}_\tau,$$

with gap_τ retained as the ledger-facing alias of clear_τ ;

- (vi) right-open windows $W = [u, u')$ are part of the certified data;
- (vii) no unconditional crop–collapse commutation is used;
- (viii) a filtered object $C_{\tau, W}F$ is an admissible representative of a repaired windowed barcode profile, not a global collapse functor;
- (ix) the metric ledger lives in a declared commutative quantale

$$(\mathbb{V}, \oplus, 0_{\mathbb{V}}, \preceq),$$

and is compared with real-valued clearance only after application of a declared monotone subadditive bottleneck scalarization $\text{val}_{\mathbb{B}}$;

- (x) no numerical budget repairs a missing representative, realization, comparison morphism, eligibility certificate, claim-status entry, manifest field, or artifact identity;
- (xi) if crop–collapse commutation is invoked, the Chapter 2 / Appendix A condition is checked on every globally retained bar, including infinite bars clipped by bounded windows;
- (xii) each comparison is explicitly typed as

$$\text{exact}, \quad \text{metric_gap_certified}, \quad \text{not_compared}.$$

Remark 3.1 (Why Chapter 3 moved before the Ext-edge). Earlier drafts placed eligibility and the Ext-edge in Chapter 3. In v19.0.0 the metric repair makes the record discipline logically prior: one must first know which windowed pre-threshold profiles, repaired evaluations, clearance values, crop budgets, comparison modes, and representative records are certified. Only then can an eligible realization regime be invoked. For this reason the Ext-edge is deferred to Chapter 4.

3.2 Pre-threshold windowed profiles

Definition 3.2 (Pre-threshold windowed profile). Let $F \in \text{FiltCh}(()k)$, let i be a monitored degree, and let

$$W = [u, u')$$

be a declared right-open window. The *pre-threshold windowed profile* is the barcode profile

$$B_{i,W}(F) := W_W \mathbf{P}_i(F).$$

It is the object on which threshold clearance is measured and to which any metric discrepancy consumed by hard deletion must apply.

Definition 3.3 (Repaired windowed evaluation). For a threshold $\tau > 0$, the repaired windowed evaluation is

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(B_{i,W}(F)) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Remark 3.4 (The two profiles are different). The pre-threshold profile $B_{i,W}(F)$ and the repaired evaluation $\text{Eval}_{i,W,\tau}(F)$ are not interchangeable. The former is used for threshold clearance and metric-gap records. The latter is used for post-collapse gate clauses and repaired-profile comparison. Thus a valid record keeps both:

$$\text{Pre}_{i,W}(F) := B_{i,W}(F), \quad \text{Post}_{i,W,\tau}(F) := \text{Eval}_{i,W,\tau}(F).$$

Remark 3.5 (Fixed order). The Core order is

$$F \mapsto \mathbf{P}_i(F) \mapsto W_W \mathbf{P}_i(F) \mapsto T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

No identity

$$W_W T_\tau^{\text{bar}} \cong T_\tau^{\text{bar}} W_W$$

is used without a separately declared window-adapted or exact-admissible hypothesis. The window-adapted condition is the necessary-and-sufficient condition of Chapter 2 and Appendix A: every globally retained bar, including an infinite bar, must be disjoint from the window or remain retained after nonempty clipping. A finite-only check is insufficient.

3.3 Threshold clearance and ledger gap

Definition 3.6 (Threshold clearance on a windowed profile). Let B be a barcode profile with finitely many bars in the monitored record, possibly including infinite bars. For $\tau > 0$, define

$$\text{clear}_\tau(B) := \inf_{I \in B_{\text{fin}}} |\ell(I) - \tau|,$$

where B_{fin} denotes the finite bars of B . If $B_{\text{fin}} = \emptyset$, set

$$\text{clear}_\tau(B) := +\infty.$$

For an AK evaluation record, write

$$\text{clear}_\tau(F; i, W) := \text{clear}_\tau(B_{i,W}(F)).$$

Definition 3.7 (Ledger gap). The ledger gap is the clearance itself:

$$\text{gap}_\tau(B) := \text{clear}_\tau(B).$$

For a finite family of monitored pre-threshold profiles $B_1, \dots, B_m$, define

$$\text{gap}_\tau(B_1, \dots, B_m) := \min_{1 \leq j \leq m} \text{clear}_\tau(B_j).$$

Thus gap_τ is not a half-clearance in v19.0.0; it is a ledger-facing alias for the actual threshold clearance.

Remark 3.8 (Why $\text{gap}_\tau = \text{clear}_\tau$). The sharp two-sided threshold-gap-safe theorem uses the closed band

$$[\tau - \varepsilon, \tau + \varepsilon].$$

Let $\mathfrak{d}_{\text{met}} \in \mathbb{V}$ be a gate-scoped metric ledger and let

$$\varepsilon_B(\mathfrak{d}_{\text{met}}) := \text{val}_B(\mathfrak{d}_{\text{met}}).$$

The strict readiness condition

$$\varepsilon_B(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau(B) = \text{clear}_\tau(B)$$

ensures that no finite bar of B lies in

$$[\tau - \varepsilon_B(\mathfrak{d}_{\text{met}}), \tau + \varepsilon_B(\mathfrak{d}_{\text{met}})].$$

For metric transport, the same budget must also upper-bound the bottleneck discrepancy between two declared profiles, and the condition must hold on both. The shorthand $\Sigma\delta < \text{gap}_\tau$ is permitted only when the run manifest identifies $\Sigma\delta$ with this scalarized real budget and records the discrepancy theorem or artifact.

Remark 3.9 (Bars of length exactly τ). If B contains a finite bar of length exactly τ , then

$$\text{clear}_\tau(B) = \text{gap}_\tau(B) = 0.$$

Such a profile is not positively gap-ready. It may still be evaluated by T_τ^{bar} , and the bar is deleted because the convention is $\ell(I) \leq \tau$. A separately proved exact identity may still be used in `exact` mode, but it is not a positive-radius robustness certificate.

3.4 Typed scalarized metric ledger

Definition 3.10 (Bottleneck scalarization and metric ledger). Let

$$(\mathbb{V}, \oplus, 0_{\mathbb{V}}, \preceq)$$

be the declared commutative ledger quantale used by the run. A *bottleneck scalarization* is a monotone subadditive map

$$\text{val}_B : \mathbb{V} \longrightarrow [0, +\infty]$$

satisfying

$$\text{val}_B(0_{\mathbb{V}}) = 0, \quad \text{val}_B(q \oplus q') \leq \text{val}_B(q) + \text{val}_B(q').$$

For a gate-scoped metric ledger $\mathfrak{d}_{\text{met}} \in \mathbb{V}$, define its real bottleneck budget by

$$\varepsilon_B(\mathfrak{d}_{\text{met}}) := \text{val}_B(\mathfrak{d}_{\text{met}}).$$

A scalarized ledger may be consumed only when a theorem, verified computation, or reconstructible artifact proves the relevant pre-threshold profile bound

$$d_B(B, C) \leq \varepsilon_B(\mathfrak{d}_{\text{met}}).$$

Monotonicity is understood relative to $\preceq$ on $\mathbb{V}$ and the usual order on $[0, +\infty]$.

Definition 3.11 (Non-scalar record obligations). The following are non-scalar proof obligations whenever they are in scope: representative existence, profile-isomorphism data, realization existence and compatibility, actual comparison morphisms, terminal-tameness certificates, eligibility, claim-status entries, manifest completeness and consistency, and artifact provenance and identity. They are not elements of $\mathcal{V}$ and cannot be discharged by increasing $\varepsilon_B(\mathfrak{d}_{\text{met}})$.

Remark 3.12 (Sharp band versus scalarization tightness). The adjective *sharp* refers to the optimal half-width ε of the forbidden threshold band for a given true bottleneck bound. It does not assert that a chosen scalarization val_B is numerically tight. A conservative scalarization may enlarge the certified band, but it does not change the sharp constant in the metric theorem.

3.5 Repaired evaluation and comparison records

Definition 3.13 (Repaired evaluation record). A *repaired evaluation record* for $(F; i, W, \tau)$ contains at least:

- (i) an immutable identifier for the filtered object or persistence source F ;
- (ii) the monitored degree i ;
- (iii) the right-open window $W = [u, u')$, including whether $u' = +\infty$;
- (iv) the threshold $\tau > 0$;
- (v) the pre-threshold profile

$$B_{i,W}(F) = W_W \mathbf{P}_i(F);$$

- (vi) the repaired post-threshold profile

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(B_{i,W}(F));$$

- (vii) the finite-bar deletion convention $\ell(I) \leq \tau$;
- (viii) the infinite-bar and endpoint conventions used by the barcode metric;
- (ix) artifact references sufficient to reconstruct the data.

Remark 3.14 (Definition, not certification). A repaired evaluation record is only a typed record. It does not by itself certify stability, transport, overlap compatibility, representative compatibility, or B-Gate⁺ pass. Those uses require a two-sided metric comparison record, a separately proved exact comparison, and every applicable non-scalar obligation.

Definition 3.15 (Single-profile threshold-gap readiness record). A *single-profile threshold-gap readiness record* for $(F; i, W, \tau)$, relative to a real test radius $\varepsilon_B \geq 0$, contains:

- (i) the immutable identifier of $B_{i,W}(F)$;
- (ii) the clearance

$$\text{clear}_\tau(B_{i,W}(F));$$

- (iii) the equality

$$\text{gap}_\tau(B_{i,W}(F)) = \text{clear}_\tau(B_{i,W}(F));$$

(iv) the strict check

$$\varepsilon_B < \text{gap}_\tau(B_{i,W}(F));$$

(v) the resulting closed-band exclusion

$$\ell(I) \notin [\tau - \varepsilon_B, \tau + \varepsilon_B]$$

for every finite bar $I \in B_{i,W}(F)$;

(vi) reconstructible evidence for the finite-bar summary and clearance.

For $\varepsilon_B = 0$, the strict check means $\text{clear}_\tau(B_{i,W}(F)) > 0$.

This record certifies only that one profile avoids the threshold band at a declared radius. It is *not* a metric comparison certificate, does not supply a second profile, and does not establish transport through hard deletion. The historical label `def:ch3-gap-certified-record` is retained for reference stability, but the normative role is readiness rather than comparison certification.

Definition 3.16 (Comparison record and two-sided metric certificate). Let F and G be two filtered objects or certified profile sources. A comparison record on (i, W, τ) declares exactly one mode:

$$\text{exact}, \quad \text{metric_gap_certified}, \quad \text{not_compared}.$$

In mode `exact`, the record contains a separately proved equality or profile isomorphism between the relevant pre-threshold or repaired profiles, its exact source and target, and reconstructible proof or artifact evidence. No positive-radius robustness follows merely from exactness.

In mode `metric_gap_certified`, the record is a two-sided threshold-gap comparison certificate and contains:

(i) immutable identifiers for

$$B := B_{i,W}(F), \quad C := B_{i,W}(G);$$

(ii) one metric ledger $\mathfrak{d}_{\text{met}}$, a declared scalarization val_B , and

$$\varepsilon_B := \text{val}_B(\mathfrak{d}_{\text{met}});$$

(iii) theorem-level, verified-computation, or artifact evidence that

$$d_B(B, C) \leq \varepsilon_B;$$

(iv) $\text{clear}_\tau(B)$, $\text{clear}_\tau(C)$, and

$$\text{gap}_\tau(B, C) = \min\{\text{clear}_\tau(B), \text{clear}_\tau(C)\};$$

(v) the strict two-sided reserve

$$\varepsilon_B < \text{gap}_\tau(B, C),$$

equivalently confirmation that neither finite-bar family meets $[\tau - \varepsilon_B, \tau + \varepsilon_B]$;

(vi) every declared metric component used to establish the windowed comparison bound, including crop, endpoint, measurement, realization-metric, computation, and threshold-reconciliation terms;

(vii) reconstructible artifact references;

(viii) one specialized status

`gap_certified, gap_failed, gap_pending,`

mapping respectively to pass, reject, and undefined.

The metric certificate is two-sided by definition. Two single-profile readiness records without the evidenced discrepancy bound do not constitute a comparison certificate. In mode `not_compared`, no comparison inference is permitted.

Proposition 3.17 (Scalarized gap-certified stability transfer). *A comparison record on $(F, G; i, W, \tau)$ whose mode is `metric_gap_certified`, with*

$$\varepsilon_B = \text{val}_B(\mathbf{d}_{\text{met}}),$$

implies

$$d_B(\text{Eval}_{i,W,\tau}(F), \text{Eval}_{i,W,\tau}(G)) \leq \varepsilon_B.$$

Proof. Apply the sharp threshold-gap-safe stability theorem of Chapter 2 and Appendix A to the pre-threshold profiles

$$B_{i,W}(F), \quad B_{i,W}(G).$$

The comparison record supplies the real bottleneck bound and the two-sided closed-band exclusion at that same budget. The theorem therefore yields the asserted post-threshold bound. $\square$

Remark 3.18 (Isometry convention). In the constructible one-parameter regime, d_B and d_{int} are identified by the barcode isometry theorem under the same infinite-bar cost convention fixed in Chapter 2 and Appendix A. Infinite bars are compared by birth-time displacement and are not finitely matched to finite bars or to the diagonal. A certificate may record the comparison using d_B or d_{int} only when the modules and barcodes are precisely those named in the comparison record and the metric convention is declared.

3.6 Crop budget and window-edge clearance

Definition 3.19 (Window-edge clearance record). *A window-edge clearance record for a comparison of F and G on (i, W) records the stability of passage from the source-level data to*

$$B_{i,W}(F) = W_W \mathbf{P}_i(F), \quad B_{i,W}(G) = W_W \mathbf{P}_i(G)$$

near the window endpoints. It contains:

- (i) the window endpoints u, u' and the bounded/terminal policy;
- (ii) immutable identifiers for the source and resulting windowed profiles;
- (iii) the relevant bar endpoints or critical parameters near u and u' , when required;
- (iv) a declared metric crop component

$$\delta_{\text{crop}}(i, W) \in V;$$

- (v) the theorem, verified computation, or artifact supporting that component and its use in the windowed bottleneck bound.

Remark 3.20 (Threshold gap does not replace crop control). Threshold clearance controls movement across the length threshold τ after windowing. It does not prove that a perturbation of F and G produces a controlled perturbation of their windowed profiles. The latter is a crop-stability question and must be supported separately through a crop component or a declared window-edge theorem. This is the Chapter 3 record-level expansion of the crop-budget discipline fixed in Chapter 2 and Appendix A; it is not an independent replacement definition.

Definition 3.21 (Profile-level metric-ledger decomposition). A profile comparison ledger may be decomposed in $\mathbb{V}$ as

$$\mathfrak{d}_{\text{met}} := \delta_{\text{source,met}} \oplus \delta_{\text{crop}} \oplus \delta_{\text{measurement}} \oplus \delta_{\text{realization,met}} \oplus \delta_{\text{computation}} \oplus \delta_{\text{other,met}}.$$

Every displayed component is a declared *metric discrepancy* component; in particular, $\delta_{\text{realization,met}}$ measures only an evidenced metric distortion induced by a realization step. Realization existence, compatibility, and eligibility remain non-scalar obligations.

The real bottleneck budget is

$$\varepsilon_B := \text{val}_B(\mathfrak{d}_{\text{met}}).$$

When all components are nonnegative real upper bounds and $\oplus$ is ordinary addition, this specializes to the usual sum. The threshold-gap comparison record consumes ε_B only after the run establishes

$$d_B(B_{i,W}(F), B_{i,W}(G)) \leq \varepsilon_B.$$

Undeclared or non-metric quantities do not belong to this decomposition.

Remark 3.22 (No shortcut from source perturbation). A source-level perturbation estimate is not automatically a threshold-gap comparison certificate. The certified chain is

$$\begin{aligned} \text{source comparison} &\implies (\mathfrak{d}_{\text{met}}, \text{val}_B) \\ &\implies d_B(B_{i,W}(F), B_{i,W}(G)) \leq \varepsilon_B \\ &\implies d_B(\text{Eval}_{i,W,\tau}(F), \text{Eval}_{i,W,\tau}(G)) \leq \varepsilon_B, \end{aligned}$$

where $\varepsilon_B = \text{val}_B(\mathfrak{d}_{\text{met}})$, and the last implication additionally consumes the two-sided gap reserve. Each arrow requires its own theorem or artifact.

3.7 Infinite bars and window policy

Definition 3.23 (Infinite-bar policy for windowed evaluation). Infinite bars are treated as follows.

- (i) Before windowing, an infinite bar has length $+\infty$ for thresholding.
- (ii) Under the standard infinite-bar bottleneck convention,
$$[a, \infty) \text{ matches } [a', \infty) \text{ with cost } |a - a'|.$$
- (iii) A finite bar and an infinite bar are not matched at finite bottleneck cost.
- (iv) If $W = [u, u']$ is bounded, window restriction may clip an infinite bar to a finite bar.
- (v) If $W = [u, \infty)$, an infinite bar intersecting W remains infinite after windowing.

Remark 3.24 (Where infinite bars affect the gate). Hard finite-threshold deletion never deletes an unwindowed infinite bar. However, bounded windowing may turn it into a finite clipped bar before thresholding. Therefore the infinite-bar convention must be read together with the declared window policy. The Core never lets an unwindowed infinite-bar rule override the actual windowed profile $B_{i,W}(F)$. If crop-collapse commutation is claimed, every globally retained infinite bar must be checked after clipping; on a bounded window it may become a finite bar of length at most τ , which destroys commutation.

3.8 Representative discipline

Definition 3.25 (Admissible filtered representative). Let $F \in \text{FiltCh}(\cdot)k$, let W be a declared window, let $\tau > 0$, and let I_{mon} be the declared set of monitored degrees. A filtered object

$$C_{\tau,W}F \in \text{FiltCh}(\cdot)k$$

is an *admissible filtered representative* of the repaired windowed profile family if, for every $i \in I_{\text{mon}}$, it is equipped with a declared profile isomorphism

$$\rho_i : \mathbf{P}_i(C_{\tau,W}F) \xrightarrow{\cong} \text{Eval}_{i,W,\tau}(F).$$

The isomorphisms, their source and target identities, and their equivalence strength are part of the representative data. Profile admissibility alone does not imply chain isomorphism, filtered chain-homotopy equivalence, filtered quasi-isomorphism, or derived equivalence.

Definition 3.26 (Representative record). A representative record for $C_{\tau,W}F$ contains:

- (i) the repaired evaluation records it realizes;
- (ii) an immutable identifier for $C_{\tau,W}F$;
- (iii) the monitored set I_{mon} ;
- (iv) the actual profile isomorphisms

$$\rho_i : \mathbf{P}_i(C_{\tau,W}F) \xrightarrow{\cong} \text{Eval}_{i,W,\tau}(F), \quad i \in I_{\text{mon}};$$

- (v) the declared equivalence strength;
- (vi) artifact references for the representative construction and the isomorphisms;
- (vii) whether filtered, derived, realization, or Ext-level use is requested;
- (viii) the representative-record status

not_invoked, undefined, certified;

- (ix) the claim-status, dependency, manifest, and artifact-identity entries governing the assertion.

A missing in-scope representative or profile-isomorphism artifact yields `undefined`; it is never repaired by metric slack.

Remark 3.27 (Representative is not a global collapse functor). The notation $C_{\tau,W}F$ does not assert a global functor

$$C_{\tau} : \text{FiltCh}(\cdot)k \longrightarrow \text{FiltCh}(\cdot)k.$$

It also does not assert

$$C_{\tau,W}F = (C_{\tau}F)|_W.$$

It means only that a filtered representative has been supplied for the repaired windowed profile family in the declared monitored degrees.

Definition 3.28 (Realization-compatible representative). A representative $C_{\tau,W}F$ is *realization-compatible* for a declared later use if it is supplied with:

- (i) a typed realization route $\mathcal{R}(C_{\tau,W}F)$;
- (ii) source and target categories and functors;
- (iii) the comparison or compatibility morphisms required by that route;
- (iv) artifact evidence and the additional hypotheses required by the eligible realization regime of Chapter 4.

Realization compatibility is not automatic from persistence-profile admissibility.

Remark 3.29 (Eligibility deferred). This chapter does not define or certify the full Ext-eligible regime. It only records whether a representative is intended for such use and whether the required realization data are present. The amplitude, edge-realization, and Ext-edge conditions are stated in Chapter 4.

3.9 Representative invariance at the profile level

Proposition 3.30 (Profile-level representative invariance). *Let $C_{\tau,W}F$ and $C'_{\tau,W}F$ be admissible representatives of the same repaired windowed profile family, and let i be a common monitored degree. Then*

$$\mathbf{P}_i(C_{\tau,W}F) = 0 \iff \mathbf{P}_i(C'_{\tau,W}F) = 0.$$

Moreover, each condition is equivalent to

$$\text{Eval}_{i,W,\tau}(F) = 0.$$

Thus the monitored topological vanishing clause is independent of the choice of admissible representative at profile-isomorphism strength.

Proof. Both monitored persistence profiles are isomorphic to $\text{Eval}_{i,W,\tau}(F)$. The zero object is preserved and reflected by an isomorphism, so the three vanishing statements are equivalent. $\square$

Remark 3.31 (Derived invariance is stronger). Profile-level representative invariance does not imply chain-level, derived, or Ext-level representative invariance. If a certificate invokes such constructions, it must supply realization-compatible representative data and, when required, a filtered quasi-isomorphism, derived isomorphism, or explicit comparison theorem at the consumed strength.

3.10 Evaluation use in B-Gate⁺

Definition 3.32 (Topological clause input). The topological input for B-Gate⁺ on $(F; W, \tau)$ is

$$\text{Eval}_{1,W,\tau}(F) = T_{\tau}^{\text{bar}}(B_{1,W}(F)).$$

The topological clause passes when

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

If an admissible representative $C_{\tau,W}F$ and its degree-one profile isomorphism are supplied, the same clause may be read as

$$\text{PH}_1(C_{\tau,W}F) = 0.$$

Definition 3.33 (Scalarized gap-ready topological input). The topological input is *gap-ready at radius* $\varepsilon_B \geq 0$ if its pre-threshold profile satisfies

$$\varepsilon_B < \text{gap}_\tau(B_{1,W}(F)).$$

Equivalently, every finite bar of $B_{1,W}(F)$ avoids

$$[\tau - \varepsilon_B, \tau + \varepsilon_B].$$

This is a single-profile readiness property. A stable perturbation claim also requires a second profile and a comparison record in mode `metric_gap_certified`, including evidence that the same ε_B upper-bounds their pre-threshold bottleneck discrepancy. The historical label is retained for reference compatibility.

Remark 3.34 (Gate value versus gate safety). The equality

$$\text{Eval}_{1,W,\tau}(F) = 0$$

is a post-threshold value statement. Single-profile gap readiness is a pre-threshold band-separation statement. A robustness claim is stronger: it requires the full two-sided comparison record. None of these numerical or profile statements supplies missing representative, realization, claim-status, manifest, or artifact evidence.

Remark 3.35 (Ext-clause not discharged here). This chapter does not discharge the Ext^1 -clause. It prepares the representative and evaluation records needed for the later one-way bridge. No implication

$$\text{Eval}_{1,W,\tau}(F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0$$

is asserted in Chapter 3.

3.11 Overlap comparison discipline

Definition 3.36 (Overlap pre-threshold profile). Let W_α and W_β be overlapping windows and write $W_{\alpha\beta} := W_\alpha \cap W_\beta$. The pre-threshold overlap profiles are

$$B_{i,W_{\alpha\beta}}^\alpha := \mathbb{W}_{W_{\alpha\beta}} \mathbf{P}_i(F_\alpha), \quad B_{i,W_{\alpha\beta}}^\beta := \mathbb{W}_{W_{\alpha\beta}} \mathbf{P}_i(F_\beta).$$

They are evaluated directly on the common overlap window, not obtained by silently reusing already-deleted local outputs.

Definition 3.37 (Overlap repaired evaluation). At one declared common threshold τ , the repaired overlap evaluations are

$$\begin{aligned} \text{Eval}_{i,W_{\alpha\beta},\tau}(F_\alpha) &:= T_\tau^{\text{bar}}(B_{i,W_{\alpha\beta}}^\alpha), \\ \text{Eval}_{i,W_{\alpha\beta},\tau}(F_\beta) &:= T_\tau^{\text{bar}}(B_{i,W_{\alpha\beta}}^\beta). \end{aligned}$$

Remark 3.38 (Two-layer structure of the Overlap Gate). In metric-gap mode, the Overlap Gate records one scalarized budget

$$\varepsilon_{\alpha\beta,i} := \text{val}_B(\mathfrak{d}_{\alpha\beta,i}),$$

together with the pre-threshold statements

$$\begin{aligned} d_B(B_{i,W_{\alpha\beta}}^\alpha, B_{i,W_{\alpha\beta}}^\beta) &\leq \varepsilon_{\alpha\beta,i}, \\ \varepsilon_{\alpha\beta,i} &< \text{gap}_\tau(B_{i,W_{\alpha\beta}}^\alpha, B_{i,W_{\alpha\beta}}^\beta). \end{aligned}$$

The metric-repair theorem then yields the post-threshold bound

$$d_{\text{int}}(\text{Eval}_{i, W_{\alpha\beta}, \tau}(F_\alpha), \text{Eval}_{i, W_{\alpha\beta}, \tau}(F_\beta)) \leq \varepsilon_{\alpha\beta, i}$$

in the declared constructible isometry regime. Thus the gap and consumed discrepancy are pre-threshold, while the resulting agreement is post-threshold. In exact mode, a separately proved equality or isomorphism is recorded instead and no positive-radius robustness is inferred.

Higher overlap coherence, simplex records, typed Type IV status, and countable-atlas conditions belong to Chapter 6 and are not created by a pairwise comparison record.

Proposition 3.39 (Scalarized overlap threshold safety). *Let $\mathfrak{d}_{\alpha\beta, i}$ be a declared overlap metric ledger and set*

$$\varepsilon_{\alpha\beta, i} := \text{val}_B(\mathfrak{d}_{\alpha\beta, i}).$$

Suppose

$$d_B(B_{i, W_{\alpha\beta}}^\alpha, B_{i, W_{\alpha\beta}}^\beta) \leq \varepsilon_{\alpha\beta, i}$$

and

$$\varepsilon_{\alpha\beta, i} < \text{gap}_\tau(B_{i, W_{\alpha\beta}}^\alpha, B_{i, W_{\alpha\beta}}^\beta).$$

Then

$$d_B(\text{Eval}_{i, W_{\alpha\beta}, \tau}(F_\alpha), \text{Eval}_{i, W_{\alpha\beta}, \tau}(F_\beta)) \leq \varepsilon_{\alpha\beta, i}.$$

Proof. The two assumptions supply a comparison record in mode `metric_gap_certified`. Apply Proposition 3.17 on the common overlap window. $\square$

3.12 Minimal E_1 -detector discipline

Definition 3.40 (Residual degree-one bar mass). Let $\mathcal{B}_{1, W, \tau}^{\text{res}}(F)$ denote the residual degree-one barcode of $\text{Eval}_{1, W, \tau}(F)$, counted with multiplicity. Define

$$E_1(F; W, \tau) := \sum_{I \in (\mathcal{B}_{1, W, \tau}^{\text{res}}(F))_{\text{fin}}} \ell(I).$$

If an infinite residual degree-one bar is present, set

$$E_1(F; W, \tau) := +\infty,$$

unless a separately declared truncated detector is being used. Empty or zero-length intervals are not barcode summands in the monitored constructible profile.

Proposition 3.41 (Sound E_1 -vanishing detector). *If*

$$E_1(F; W, \tau) = 0,$$

then

$$\text{Eval}_{1, W, \tau}(F) = 0.$$

Proof. By definition, the presence of an infinite residual bar would give $E_1 = +\infty$. Every nonempty finite interval summand has strictly positive length and contributes positively, with multiplicity, to the displayed sum. Therefore $E_1 = 0$ implies that no residual degree-one bar is present, so the repaired degree-one profile is zero. $\square$

Remark 3.42 (No replacement of Eval). The scalar E_1 is an operational vanishing detector. It does not replace the exact profile statement

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

It is not asserted to be bottleneck-Lipschitz, is not a threshold-gap budget, and is not a substitute for a two-sided comparison record. Length-sum functionals can be unstable when multiplicities grow or bars move across the threshold. A truncated variant must carry its own policy and cannot silently inherit the conclusion above.

3.13 Manifest consequences

Definition 3.43 (Chapter 3 evaluation and representative manifest extension). The Chapter 3 manifest is an extension of the foundational Chapter 1 Minimal Manifest Axiom. A proof object invoking Chapter 3 data records:

- (i) repaired evaluation records and immutable profile identities;
- (ii) pre-threshold profiles $B_{i,W}(F)$ and repaired evaluations $\text{Eval}_{i,W,\tau}(F)$;
- (iii) clearance and gap values and any single-profile readiness records;
- (iv) the comparison mode for every comparison;
- (v) for metric-gap mode, every metric ledger, its aggregation law, scalarization, real budget, evidenced discrepancy bound, two-sided gap reserve, specialized status, and artifact reference;
- (vi) for exact mode, the exact equality or profile-isomorphism proof and its precise source and target;
- (vii) crop budgets or window-edge records when source-level comparisons are transported to windowed profiles;
- (viii) any claimed τ -window-adapted commutation certificate, including checks for globally retained infinite bars on bounded windows;
- (ix) infinite-bar and bounded/terminal-window policies;
- (x) representative records, actual profile isomorphisms, monitored degrees, equivalence strengths, and typed statuses whenever $C_{\tau,W}F$ is in scope;
- (xi) realization-compatibility data when filtered, derived, or Ext-level use is intended;
- (xii) all in-scope non-scalar obligations, dependency states, atomic statuses, and artifact identities;
- (xiii) claim-status references, schema/version identifiers, repair ancestry, and artifact hashes sufficient for audit and replay.

Remark 3.44 (Canonical manifest hierarchy). Chapter 1 supplies the foundational manifest axiom; this chapter expands it for repaired evaluation, comparison mode, metric-ledger, crop, window, and representative records; Chapter 4 adds eligibility records. Chapter 7 integrates these extensions into the canonical v19 audit manifest with immutable run identity, completeness/consistency status, typed clause verdicts, replayability, and repair-as-new-record semantics. For final serialization and verdict reconstruction, the Chapter 7 contract is canonical.

Remark 3.45 (Record completeness and failure semantics). A record containing only $\text{Eval}_{i,W,\tau}(F)$ can state a post-threshold value, but cannot support stability, overlap transfer, representative compatibility, or Ext eligibility. Missing required Chapter 3 data yield an undefined audit clause, not a passing clause and not a mathematical counterexample. A failed evidenced inequality yields reject. An internally inconsistent record is routed to Type V rejection under Chapter 7.

3.14 Explicit non-claims of Chapter 3

The following are not asserted in this chapter.

- (i) No global non-expansiveness of T_τ^{bar} is asserted.
- (ii) No band of half-width 2ε is claimed to be sharp for the two-sided theorem; the sharp half-width is ε .
- (iii) No half-clearance convention

$$\text{gap}_\tau = \frac{1}{2} \text{clear}_\tau$$
 is used.
- (iv) No single-profile readiness record is a metric comparison certificate.
- (v) No exact equality or profile isomorphism is automatically a positive-radius robustness certificate.
- (vi) No unconditional crop-collapse commutation is asserted.
- (vii) No unscalarized quantale element is compared directly with the real-valued clearance $\text{gap}_\tau = \text{clear}_\tau$.
- (viii) No numerical budget repairs a missing representative, profile isomorphism, realization, comparison morphism, eligibility certificate, claim-status entry, manifest field, or artifact identity.
- (ix) No realization existence or compatibility obligation is hidden inside $\delta_{\text{realization,met}}$.
- (x) No finite-only window-adapted certificate is sufficient; globally retained infinite bars must also be checked after clipping.
- (xi) No pairwise overlap record supplies higher Čech coherence or countable-atlas gluing without Chapter 6 hypotheses.
- (xii) No Chapter 3 manifest fragment supersedes the canonical Chapter 7 audit manifest.
- (xiii) No source-level perturbation is automatically converted into a windowed-profile perturbation.
- (xiv) No global strict filtered collapse functor C_τ is asserted.
- (xv) No identity $C_{\tau,W}F = (C_\tau F)|_W$ is asserted.
- (xvi) Persistence-profile isomorphism does not automatically imply chain isomorphism, homotopy equivalence, filtered quasi-isomorphism, or derived equivalence.
- (xvii) No derived or Ext-level conclusion follows from an admissible representative without the eligible realization regime of Chapter 4.
- (xviii) No reverse implication $\text{Ext}^1 \Rightarrow \text{PH}_1$ is asserted.

(xix) No undefined or not-invoked Type IV state is represented by a fabricated numerical pair:

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0).$$

(xx) No problem-specific arithmetic, PDE, geometric, categorical, or cryptographic theorem is proved.

(xxi) No Search artifact, advisory invariant, scalar potential, ranking, or AI confidence value affects a Core repaired-evaluation verdict.

3.15 Provenance and status

This chapter depends on:

- (i) Chapter 1 for v19.0.0 claim discipline, B-Gate⁺, ledger discipline, Type IV status semantics, and manifest discipline;
- (ii) Chapter 2 for repaired windowed evaluation, comparison modes, crop control, and sharp threshold-gap-safe stability;
- (iii) Appendix A for the detailed proof of the metric repair and the exact crop-collapse criterion;
- (iv) Appendix B for fuller representative discipline when filtered or derived constructions are used;
- (v) Chapter 4 for eligible realization and the one-way $\text{PH}_1 \Rightarrow \text{Ext}^1$ bridge;
- (vi) Chapter 6 for higher overlap coherence and countable-atlas gluing;
- (vii) Chapter 7 for the canonical audit manifest and typed verdict semantics.

Status labels for this chapter. *Core definitions.* Definitions [3.2](#), [3.3](#), [3.6](#), [3.7](#), [3.10](#), [3.11](#), [3.13](#), [3.15](#), [3.16](#), [3.23](#), [3.25](#), [3.26](#), [3.32](#), and [3.33](#).

Core propositions. Propositions [3.17](#), [3.30](#), and [3.39](#).

Operational detector. Definition [3.40](#) and Proposition [3.41](#).

Operational policies and manifest records. Definitions [3.19](#), [3.21](#), [3.28](#), [3.36](#), [3.37](#), and [3.43](#).

Core doctrine and integration remarks. Remarks [3.12](#), [3.44](#), and [3.45](#).

Explicit exclusions. Subsection [3.14](#).

4 Chapter 4: Eligibility, Ext-edge, and the One-Way Bridge $\text{PH}_1 \Rightarrow \text{Ext}^1$

Chapter 4 contract. Chapter 3 fixes repaired windowed evaluation, comparison mode, and representative records. This chapter consumes those records and proves the conditional categorical discharge used by B-Gate⁺: once a scope-level realization functor, a residual edge policy, an actual edge-realization certificate, and the two-term amplitude condition have been fixed, repaired degree-one vanishing implies vanishing of the corresponding derived Ext¹-edge.

The derived-category identification

$$\mathrm{Ext}^1(A, k) \cong \mathrm{Hom}_k(H^{-1}(A), k)$$

is proved canonically from the standard truncation triangle. By contrast, the identification

$$H^{-1}(\mathcal{R}(C_{\tau, W}F)) \cong H_1^{\tau, W}(F)$$

with the fixed residual edge of the repaired persistence profile is an explicit eligibility interface, not a theorem manufactured from barcode data.

The only bridge proved here is therefore the conditional implication

$$\mathrm{Eval}_{1, W, \tau}(F) = 0 \implies \mathrm{Ext}^1(\mathcal{R}(C_{\tau, W}F), k) = 0.$$

The reverse implication is not derivable from the stated eligibility axioms without an additional zero-reflection or faithfulness theorem. Missing eligibility is *undefined*, not categorical vanishing, and no local equivalence $\mathrm{PH}_1 \Leftrightarrow \mathrm{Ext}^1$ belongs to the v19.0.0 Core.

4.1 Scope and placement in the v19 Core

All statements in this chapter are made in the v19.0.0 Threshold-Gap Metric Repair regime fixed by Chapters 1–3 and Appendix A. Thus:

- (i) persistence data live in the constructible one-parameter barcode regime over a field k ;
- (ii) the repaired evaluation is the post-threshold profile

$$\mathrm{Eval}_{i, W, \tau}(F) = T_{\tau}^{\mathrm{bar}}(W_W \mathbf{P}_i(F));$$

- (iii) threshold clearance is measured on the pre-threshold profile

$$B_{i, W}(F) := W_W \mathbf{P}_i(F), \quad \mathrm{gap}_{\tau} = \mathrm{clear}_{\tau};$$

- (iv) every comparison declares exactly one Chapter 3 mode

$$\mathrm{exact}, \quad \mathrm{metric_gap_certified}, \quad \mathrm{not_compared};$$

a metric comparison transported through hard deletion requires the typed condition

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_{\tau}(\mathfrak{B})$$

for the declared finite family $\mathfrak{B}$, whereas an exact comparison carries its own equality or profile-isomorphism proof and is not automatically a positive-radius robustness certificate;

- (v) an object $C_{\tau, W}F$ is an admissible filtered representative of the repaired profile, not a global strict collapse functor;

- (vi) the categorical Ext^1 -clause may be invoked only after an eligible realization record has been declared complete and consistent;
- (vii) realization and edge-extraction policies are fixed at declared scope level and may not be selected after inspecting the desired gate verdict;
- (viii) audited B-Gate⁺ use retains the independent threshold-gap, metric-ledger, Type IV, manifest, and claim-status clauses of Chapters 1, 5, and 7.

Remark 4.1 (Relocation of the previous failure-taxonomy chapter). Earlier drafts placed failure types, tower comparison artifacts, and Type IV diagnostics in Chapter 4. After the v19 Chapter 3 repair, that material is logically downstream of the one-way bridge and should be read as the next failure-diagnostics chapter. The present Chapter 4 is therefore reserved for eligibility and the Ext^1 -edge.

Remark 4.2 (Metric repair is consumed, not reproved). This chapter does not reprove threshold-gap-safe stability. It assumes that the repaired evaluation records consumed here have already been formed according to Chapter 3, with any required threshold-gap certification supplied separately. The bridge below is a categorical discharge result, not a stability theorem for threshold deletion.

4.2 Repaired degree-one input

Definition 4.3 (Degree-one repaired profile). For $F \in \text{FiltCh}(()k)$, a right-open window $W = [u, u')$, and a threshold $\tau > 0$, the degree-one repaired profile consumed by this chapter is

$$\text{Post}_{1,W,\tau}(F) := \text{Eval}_{1,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_1(F)).$$

The statement $\text{Eval}_{1,W,\tau}(F) = 0$ means that this post-threshold barcode profile is the zero persistence profile.

Definition 4.4 (Degree-one admissible representative). An admissible representative $C_{\tau,W}F$ is *degree-one compatible* if its Chapter 3 representative record contains an actual profile isomorphism

$$\rho_{1,F,W,\tau} : \mathbf{P}_1(C_{\tau,W}F) \xrightarrow{\sim} \text{Eval}_{1,W,\tau}(F).$$

The source, target, equivalence strength, and reconstructible artifact for $\rho_{1,F,W,\tau}$ are part of the record. A bare symbol $\cong$ without such evidence is not an eligibility certificate. When this holds, the notation

$$\text{PH}_1(C_{\tau,W}F) = 0$$

is used only as shorthand for the vanishing of that realized repaired profile.

Remark 4.5 (No global collapse interpretation). The notation $C_{\tau,W}F$ does not mean $(C_\tau F)|_W$, and no global exact functor C_τ is used. The only profile entering this chapter is the repaired windowed profile

$$T_\tau^{\text{bar}}(W_W \mathbf{P}_1(F)).$$

4.3 Eligible realization records

Definition 4.6 (Realization datum). A *realization datum* for a degree-one compatible representative $C_{\tau,W}F$ consists of:

- (i) a declared eligible domain

$$\mathcal{E} \subseteq \text{FiltCh}(()k)$$

containing $C_{\tau,W}F$;

(ii) a genuine functor, fixed at the declared scope level,

$$\mathcal{R} : \mathcal{E} \longrightarrow D^b(k\text{-mod});$$

(iii) the realized object

$$A_{F,W,\tau} := \mathcal{R}(C_{\tau,W}F);$$

(iv) immutable policy, version, and artifact identifiers for $\mathcal{E}$ and $\mathcal{R}$.

The functor may be partial through the declared domain $\mathcal{E}$, but it is not an objectwise rule chosen after the desired Ext verdict is known.

Definition 4.7 (Residual edge extractor). A *residual degree-one edge extractor* is a scope-level, isomorphism-invariant vector-space-valued policy

$$\text{Edge}_1 : \text{Prof}_1^{\text{rep}} \longrightarrow \text{Vect}_k$$

on the declared groupoid or category of repaired degree-one profiles and admissible profile isomorphisms, satisfying

$$\text{Edge}_1(0) = 0.$$

If no profile morphism category has been declared, this is read as an assignment invariant under barcode-profile isomorphism. The extractor is fixed independently of the objectwise gate verdict. It need not be faithful or zero-reflecting unless a stronger bridge theorem explicitly proves that property.

Definition 4.8 (Edge-realization certificate). An *edge-realization certificate* is attached to the declared data

$$(F; W, \tau, C_{\tau,W}F, \mathcal{R}, \text{Edge}_1).$$

It is an actual declared isomorphism

$$\epsilon_{F,W,\tau} : H^{-1}(\mathcal{R}(C_{\tau,W}F)) \xrightarrow{\sim} \text{Edge}_1(\text{Eval}_{1,W,\tau}(F)).$$

Its source, target, dependence on the representative isomorphism $\rho_{1,F,W,\tau}$, and reconstructible proof artifact are recorded. The only objectwise implication consumed by the Core bridge is

$$\text{Eval}_{1,W,\tau}(F) = 0 \implies H^{-1}(\mathcal{R}(C_{\tau,W}F)) = 0.$$

An objectwise certificate does not by itself establish naturality in F . Whenever naturality or transport across morphisms is claimed, the certificates must form a compatible natural family on the declared eligible domain.

Definition 4.9 (Eligible realization record). An *eligible realization record* for $(F; W, \tau)$ consists of:

- (i) a repaired evaluation record for $\text{Eval}_{1,W,\tau}(F)$;
- (ii) a degree-one compatible admissible representative $C_{\tau,W}F$, its Chapter 3 representative record, and the actual isomorphism $\rho_{1,F,W,\tau}$;
- (iii) a realization datum $(\mathcal{E}, \mathcal{R})$, membership $C_{\tau,W}F \in \mathcal{E}$, and the realized object $A_{F,W,\tau}$;
- (iv) the amplitude certificate

$$A_{F,W,\tau} \in D^{[-1,0]}(k\text{-mod});$$

- (v) the fixed residual-edge policy Edge_1 and its policy identifier;
- (vi) an edge-realization certificate $\epsilon_{F,W,\tau}$, with objectwise or natural-family scope explicitly declared;
- (vii) a field coefficient declaration for k ;
- (viii) a non-circularity entry confirming that $\mathcal{E}$, $\mathcal{R}$, and Edge_1 were fixed independently of the desired pass/reject verdict;
- (ix) representative-equivalence and realization-equivalence audit data;
- (x) immutable artifact references sufficient to reconstruct the representative, realization, amplitude, and edge certificates;
- (xi) the applicable claim-status and manifest entries under the canonical Chapter 7 audit contract.

A windowed object $(F; W, \tau)$ is called *eligible for the Ext-edge* only when the required record is complete and consistent. An invoked but incomplete, circular, or ill-typed record yields an undefined categorical clause, not a passing clause and not a mathematical counterexample.

Remark 4.10 (Eligibility is stronger than having a representative). An admissible representative alone is not enough to invoke Ext^1 . The Ext-edge also needs a scope-level realization functor, an amplitude certificate, a fixed residual-edge policy, and an actual edge-realization certificate. The canonical derived-category edge proved below does not construct that interface certificate. Consequently, the one-way bridge is a conditional Core/interface theorem, not an unconditional identification of persistent homology with derived Ext.

Remark 4.11 (Gap certification and non-scalar obligations are separate). The eligible realization record does not replace the Chapter 3 comparison record. For audited B-Gate⁺ use, every `metric_gap_certified` comparison still requires an evidenced pre-threshold discrepancy bound and the typed inequality

$$\text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau(\mathfrak{B})$$

measured on the relevant two-sided pre-threshold profile family $\mathfrak{B}$. A comparison in `exact` mode instead carries its exact proof and is not automatically a robustness certificate.

Conversely, numerical clearance cannot manufacture an admissible representative, a realization functor, an amplitude certificate, an edge-realization certificate, or a complete manifest. Those are non-scalar proof obligations. The present chapter discharges only the categorical Ext^1 -edge once exact repaired topological vanishing and full eligibility have been established.

4.4 The two-term Ext^1 -edge

In this chapter, $k\text{-mod}$ denotes finite-dimensional k -vector spaces. The superscript in $D^{[-1,0]}$ refers to the standard cohomological t -structure.

Lemma 4.12 (Formality over a field; noncanonical splitting). *Let $A \in D^b(k\text{-mod})$, where k is a field. Then there exists an isomorphism*

$$A \cong \bigoplus_{j \in \mathbb{Z}} H^j(A)[-j]$$

in $D^b(k\text{-mod})$. In general this direct-sum splitting depends on choices and is not used to prove the naturality of the Ext-edge identification below.

Proof. Every short exact sequence of k -vector spaces splits, so every bounded complex is quasi-isomorphic to its cohomology complex with zero differential. Choosing splittings gives the stated derived isomorphism. The choice-dependence is precisely why the natural edge theorem is proved instead from the canonical truncation triangle. $\square$

Theorem 4.13 (Canonical two-term Ext^1 -edge identification). *Let $A \in D^{[-1,0]}(k\text{-mod})$. Then there is a canonical isomorphism, natural in A ,*

$$\text{Ext}^1(A, k) := \text{Hom}_{D^b(k\text{-mod})}(A, k[1]) \xrightarrow{\sim} \text{Hom}_k(H^{-1}(A), k).$$

Proof. The standard t -structure gives the canonical truncation triangle

$$H^{-1}(A)[1] \longrightarrow A \longrightarrow H^0(A) \xrightarrow{+1} H^{-1}(A)[2].$$

Apply the contravariant cohomological functor $\text{Hom}_{D^b(k\text{-mod})}(-, k[1])$. The resulting long exact sequence contains the natural exact segment

$$\begin{aligned} \text{Hom}_{D^b}(H^0(A), k[1]) &\longrightarrow \text{Hom}_{D^b}(A, k[1]) \\ &\longrightarrow \text{Hom}_{D^b}(H^{-1}(A)[1], k[1]) \longrightarrow \text{Hom}_{D^b}(H^0(A), k[2]). \end{aligned}$$

Because $k\text{-mod}$ is semisimple,

$$\text{Hom}_{D^b}(H^0(A), k[1]) = \text{Ext}_k^1(H^0(A), k) = 0$$

and

$$\text{Hom}_{D^b}(H^0(A), k[2]) = \text{Ext}_k^2(H^0(A), k) = 0.$$

Hence the middle arrow is an isomorphism. Finally,

$$\text{Hom}_{D^b}(H^{-1}(A)[1], k[1]) \cong \text{Hom}_k(H^{-1}(A), k).$$

The truncation triangle and the long exact sequence are functorial in A , so the resulting isomorphism is natural. $\square$

Corollary 4.14 (Naturality square). *For every morphism $f : A \rightarrow A'$ in $D^{[-1,0]}(k\text{-mod})$, the square*

$$\begin{array}{ccc} \text{Ext}^1(A', k) & \longrightarrow & \text{Ext}^1(A, k) \\ \downarrow \sim & & \downarrow \sim \\ \text{Hom}_k(H^{-1}(A'), k) & \longrightarrow & \text{Hom}_k(H^{-1}(A), k) \end{array}$$

commutes, where the horizontal maps are induced contravariantly by f and by $H^{-1}(f)$, respectively.

Proof. This is the naturality assertion of Theorem 4.13 written as a commutative square. $\square$

Corollary 4.15 (Edge vanishing criterion). *If $A \in D^{[-1,0]}(k\text{-mod})$ and $H^{-1}(A) = 0$, then*

$$\text{Ext}^1(A, k) = 0.$$

Proof. Apply Theorem 4.13. $\square$

Remark 4.16 (Why the field hypothesis matters). Over a general coefficient ring, the truncation argument yields an exact sequence whose outer terms include $\text{Ext}^1(H^0(A), k)$ and $\text{Ext}^2(H^0(A), k)$. These groups need not vanish, so the displayed natural isomorphism is not a theorem in that generality without additional projectivity, injectivity, or homological-dimension hypotheses.

Remark 4.17 (Splitting versus naturality). Lemma 4.12 explains the semisimple shape of the answer, but a chosen direct-sum decomposition of A need not be natural. Theorem 4.13 obtains the same group canonically from truncation and therefore supports representative and realization functoriality.

4.5 The one-way bridge $\text{PH}_1 \Rightarrow \text{Ext}^1$

Theorem 4.18 (Conditional one-way repaired bridge). *Let $(F; W, \tau)$ be eligible for the Ext-edge in the sense of Definition 4.9. If*

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

then

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Equivalently, when the representative realizes the repaired degree-one profile,

$$\text{PH}_1(C_{\tau,W}F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Proof. Set

$$A_{F,W,\tau} := \mathcal{R}(C_{\tau,W}F).$$

By eligibility,

$$A_{F,W,\tau} \in D^{[-1,0]}(k\text{-mod}).$$

The edge-realization certificate gives

$$H^{-1}(A_{F,W,\tau}) \cong \text{Edge}_1(\text{Eval}_{1,W,\tau}(F)).$$

If $\text{Eval}_{1,W,\tau}(F) = 0$, then by zero-compatibility of the edge extractor,

$$\text{Edge}_1(\text{Eval}_{1,W,\tau}(F)) = \text{Edge}_1(0) = 0.$$

Hence

$$H^{-1}(A_{F,W,\tau}) = 0.$$

Corollary 4.15 now gives

$$\text{Ext}^1(A_{F,W,\tau}, k) = 0,$$

which is the asserted conclusion. $\square$

Remark 4.19 (Discharge theorem, not equivalence). Theorem 4.18 is a conditional discharge theorem for the categorical clause of B-Gate⁺. Its unconditional mathematical input is the canonical two-term Ext-edge; its interface hypothesis is the eligible realization record. It states that repaired topological vanishing is sufficient to discharge the eligible Ext¹-edge. It does not state that categorical vanishing reconstructs the repaired degree-one barcode profile, nor does it prove the edge-realization certificate from persistence data.

Remark 4.20 (No use of search, spectral, or CCC reverse principles). The bridge uses only the eligible realization record and the two-term field-coefficient Ext-edge. The realization and residual-edge policies must be fixed before the objectwise verdict is evaluated. The proof does not use CCC authorization, spectral-gap hypotheses, Bridge Programs, validity maps, advisory invariants, Search-layer outputs, or AI-generated confidence values.

4.6 Gate integration

Definition 4.21 (Ext-edge discharge record). An *Ext-edge discharge record* for $(F; W, \tau)$ contains:

- (i) the repaired evaluation artifact establishing the exact post-threshold statement $\text{Eval}_{1,W,\tau}(F) = 0$;
- (ii) the comparison mode and, only when that exact vanishing was transported from another object or run, the corresponding exact or two-sided metric comparison record;
- (iii) the complete eligible realization record of Definition 4.9;
- (iv) the theorem identifier for Theorem 4.18;
- (v) the canonical Ext-edge identification and its naturality data;
- (vi) the resulting categorical conclusion

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0;$$

- (vii) immutable artifact and claim-register references sufficient for replay.

If repaired topological vanishing is asserted under perturbation, the record also cites the relevant two-sided threshold-gap certificate, evidenced pre-threshold discrepancy bound, and scalarized metric budget. Exact mode instead cites the exact equality or profile-isomorphism proof.

Definition 4.22 (Typed Ext-edge clause status). The categorical clause attached to $(F; W, \tau)$ has one of the following audit meanings:

- (i) `not_invoked`: the run manifest explicitly places the Ext-clause outside scope;
- (ii) `undefined`: the clause is invoked but a required representative, eligibility, amplitude, edge, artifact, claim-status, or manifest record is absent or ill-typed;
- (iii) `pass`: the clause is well-typed and $\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0$, whether by Theorem 4.18 or by a separately verified direct computation;
- (iv) `reject`: the clause is well-typed and the eligible Ext^1 -group is evidenced to be nonzero.

The statuses `undefined` and `not_invoked` are distinct; neither is a fabricated zero group.

Proposition 4.23 (B-Gate⁺ categorical discharge). *Let $(F; W, \tau)$ be eligible for the Ext-edge. If the repaired topological clause*

$$\text{Eval}_{1,W,\tau}(F) = 0$$

holds, then the categorical Ext^1 -clause of B-Gate⁺ is satisfied:

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Proof. This is exactly Theorem 4.18 read as a gate-discharge statement. □

Remark 4.24 (What remains after categorical discharge). The Ext-edge discharge is only one atomic B-Gate⁺ clause. A full audited pass still requires, independently, the exact topological statement

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

complete and consistent manifest/claim-status data, and every comparison obligation in its declared mode. In particular, a metric transport through hard deletion requires

$$\text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau(\mathfrak{B})$$

together with an evidenced discrepancy bound, while exact mode requires its exact proof. The pass also requires a Type IV scope status equal to

$$\text{not_invoked} \quad \text{or} \quad \text{certified_zero}.$$

If tower semantics are invoked, the comparison morphism, terminal-tameness, and defect diagnostics must be present; undefined and obstructed fail the gate. No passing categorical clause cancels another rejected or required undefined clause.

4.7 Representative and realization invariance

Definition 4.25 (Realization-equivalent representatives). Two eligible representatives $C_{\tau,W}F$ and $C'_{\tau,W}F$ are called *realization-equivalent* if their realized objects are isomorphic in the derived category:

$$\mathcal{R}(C_{\tau,W}F) \cong \mathcal{R}(C'_{\tau,W}F) \quad \text{in } D^b(k\text{-mod}),$$

and the edge-realization certificates are compatible with that isomorphism.

Proposition 4.26 (Representative invariance of Ext discharge). *Let the realization-equivalent eligible representatives be*

$$C_{\tau,W}F \quad \text{and} \quad C'_{\tau,W}F.$$

Then the declared realization isomorphism induces an identification of the categorical groups, and the truth value of

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0$$

is independent of the representative choice.

Proof. The declared isomorphism

$$\alpha : \mathcal{R}(C_{\tau,W}F) \xrightarrow{\sim} \mathcal{R}(C'_{\tau,W}F)$$

induces an isomorphism under the contravariant functor $\text{Hom}_{D^b}(-, k[1])$. This identification is canonical relative to the recorded α ; no independence from an unrecorded choice of realization is asserted. By Corollary 4.14, it agrees with dualizing the induced map on H^{-1} . Compatibility of the edge-realization certificates therefore transports the discharge data, and vanishing is invariant. $\square$

Remark 4.27 (Profile invariance is weaker). Profile-level representative invariance from Chapter 3 does not by itself imply realization-level invariance. For Ext-use, the proof object must record realization-equivalence or must commit to a specific representative. This prevents the Ext¹-verdict from depending on an untracked representative choice.

4.8 Interaction with the E_1 detector

Proposition 4.28 (Operational E_1 -detector discharge). *Suppose $(F; W, \tau)$ is eligible for the Ext-edge and the canonical, non-truncated operational detector of Chapter 3 satisfies*

$$E_1(F; W, \tau) = 0.$$

Then

$$\text{Ext}^1(\mathcal{R}(C_{\tau, W}F), k) = 0.$$

Proof. By the canonical detector rule of Chapter 3, $E_1 = 0$ already excludes both finite and infinite residual bars and therefore implies

$$\text{Eval}_{1, W, \tau}(F) = 0.$$

The conclusion follows from Theorem 4.18. $\square$

Remark 4.29 (No stability or budget role for E_1). The preceding proposition uses the canonical E_1 only as a vanishing detector. It does not make E_1 a bottleneck-stable quantity, a threshold-gap certificate, a metric-ledger component, or a replacement for

$$\text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_{\tau}(\mathfrak{B}).$$

A truncated detector has its own policy and does not inherit the proposition without a separate proof.

4.9 Non-converse discipline

Proposition 4.30 (Failure of the reverse bridge for a natural nonfaithful edge). *Under Definitions 4.6–4.9, the implication*

$$\text{Ext}^1(\mathcal{R}(C_{\tau, W}F), k) = 0 \implies \text{Eval}_{1, W, \tau}(F) = 0$$

is false in general, even when the realization and edge extractor are fixed functorially at scope level.

Proof. Choose $0 < \tau < L$, choose $\delta > 0$, set

$$W = [0, L + \delta), \quad t_* = L + \frac{\delta}{2},$$

and let F be a finite filtered complex with a degree-one generator born at 0 and a degree-two generator born at L whose boundary kills the degree-one generator. Its degree-one persistence barcode on W is the single interval $[0, L)$. Since $L > \tau$, hard threshold deletion retains that interval, so

$$\text{Eval}_{1, W, \tau}(F) \cong k_{[0, L)} \neq 0.$$

Take this filtered model as the degree-one compatible representative.

Fix, independently of the verdict, the point-evaluation edge policy and realization functor

$$\text{Edge}_1(P) := P(t_*), \quad \mathcal{R}_{t_*}(C) := H_1(C^{t_*})[1].$$

Both are functorial on the evident declared domain, and $\mathcal{R}_{t_*}(C) \in D^{[-1, -1]}(k\text{-mod}) \subset D^{[-1, 0]}(k\text{-mod})$. The representative isomorphism induces the natural edge certificate

$$H^{-1}(\mathcal{R}_{t_*}(C_{\tau, W}F)) = H_1((C_{\tau, W}F)^{t_*}) \xrightarrow{\sim} \text{Eval}_{1, W, \tau}(F)(t_*) = \text{Edge}_1(\text{Eval}_{1, W, \tau}(F)).$$

Because $t_* > L$, both sides are zero. Therefore

$$\text{Ext}^1(\mathcal{R}_{t_*}(C_{\tau, W}F), k) = 0,$$

while the repaired persistence profile itself remains nonzero. Thus a point-evaluation edge can miss a transient bar, and reverse recovery requires a separate zero-reflection or faithfulness theorem. $\square$

Remark 4.31 (What a reverse theorem would require). A reverse bridge would need a separately proved zero-reflection condition, for example

$$\text{Edge}_1(P) = 0 \implies P = 0$$

on the declared repaired-profile class, together with compatibility showing that the realization edge is exactly that extractor. Such faithfulness is a new bridge theorem with new hypotheses; it cannot be inferred from Ext vanishing, amplitude, or eligibility alone.

4.10 Canonical audit-manifest extension

Definition 4.32 (Eligibility manifest extension). The Chapter 4 eligibility entry is an extension of the foundational Chapter 1 manifest and the Chapter 3 evaluation/representative extension, and is governed canonically by the Chapter 7 audit manifest. A proof object invoking the Ext-edge records at least:

- (i) the repaired evaluation and pre-threshold profile identifiers from Chapter 3;
- (ii) the representative identifier, the actual profile isomorphism $\rho_{1,F,W,\tau}$, and immutable construction artifacts for $C_{\tau,W}F$;
- (iii) the monitored degree, window, and threshold $(1, W, \tau)$;
- (iv) the eligible domain $\mathcal{E}$, the realization functor $\mathcal{R}$, their versions, and artifact identifiers;
- (v) the amplitude certificate $\mathcal{R}(C_{\tau,W}F) \in D^{[-1,0]}(k\text{-mod})$;
- (vi) the fixed edge extractor Edge_1 , its policy identifier, the edge-realization certificate $\epsilon_{F,W,\tau}$, and whether its scope is objectwise or natural-family;
- (vii) the coefficient field declaration;
- (viii) the non-circularity record and representative/realization equivalence policy;
- (ix) the typed Ext-clause status of Definition 4.22;
- (x) the theorem or direct-computation identifier supporting a passing categorical clause;
- (xi) the Chapter 3 comparison mode and, when topological vanishing is transported under perturbation, the two-sided threshold-gap certificate, evidenced discrepancy bound, and scalarized metric budget;
- (xii) all claim-register entries, hashes, and provenance links needed for independent replay.

Remark 4.33 (Manifest status and failure routing). An invoked Ext-clause with missing eligibility data is `undefined`; it is not a mathematical counterexample and not `not_invoked` unless scope exclusion was explicitly declared. Under Chapter 7, an incomplete manifest yields audit undefinedness unless another evidenced clause already forces rejection, while an inconsistent manifest yields Type V rejection. Only a complete and consistent manifest can support a categorical pass.

4.11 Bridge package

Theorem 4.34 (Eligible Ext-edge bridge package). *In the v19 repaired Core, the following package is available.*

- (i) *An eligible realization record makes the object $\mathcal{R}(C_{\tau, W}F)$ well-typed in $D^{[-1, 0]}(k\text{-mod})$.*
- (ii) *In that two-term field-coefficient regime,*

$$\text{Ext}^1(A, k) \cong \text{Hom}_k(H^{-1}(A), k).$$

- (iii) *The eligible interface supplies, rather than derives, the edge-realization certificate*

$$H^{-1}(\mathcal{R}(C_{\tau, W}F)) \cong H_1^{\tau, W}(F),$$

identifying the fixed residual degree-one edge of $\text{Eval}_{1, W, \tau}(F)$.

- (iv) *Therefore repaired degree-one vanishing conditionally discharges the categorical edge:*

$$\text{Eval}_{1, W, \tau}(F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau, W}F), k) = 0.$$

- (v) *This implication is one-way and does not replace the threshold-gap, budget, or Type IV clauses of $B\text{-Gate}^+$.*

Proof. Item (i) is the eligibility record of Definition 4.9. Item (ii) is Theorem 4.13. Item (iii) is Definition 4.8. Item (iv) is Theorem 4.18. Item (v) follows from the explicit scope and non-converse remarks of this chapter and from the $B\text{-Gate}^+$ contract of Chapter 1. $\square$

Remark 4.35 (Package status). Theorem 4.34 is a conditional closure package. It collects the proved derived-category edge together with explicitly assumed, typed interface prerequisites. It does not promote an edge-realization certificate, eligibility record, artifact, manifest entry, or $B\text{-Gate}$ clause into an independently proved mathematical theorem.

4.12 Explicit non-claims

The following are not asserted in this chapter.

- (i) No converse implication

$$\text{Ext}^1(\mathcal{R}(C_{\tau, W}F), k) = 0 \implies \text{Eval}_{1, W, \tau}(F) = 0$$

is asserted.

- (ii) No local equivalence

$$\text{PH}_1 \Leftrightarrow \text{Ext}^1$$

is asserted.

- (iii) No unconditional persistence-to-derived bridge is asserted without a fixed realization functor, a fixed residual-edge policy, and an actual edge-realization certificate.
- (iv) No objectwise edge-realization certificate is read as a natural transformation unless natural-family compatibility has been proved and recorded.

- (v) No realization or edge extractor may be selected adaptively after inspecting the desired pass/reject verdict.
- (vi) No Ext-clause is invoked without an admissible representative, a complete eligibility record, an amplitude certificate, and an edge-realization certificate.
- (vii) No missing eligibility or artifact is interpreted as zero; its required-clause status is `undefined`.
- (viii) No explicitly out-of-scope clause is confused with missing evidence. The statuses

`not_invoked` `and` `undefined`

are distinct.

- (ix) No global collapse functor C_τ or identity $C_{\tau,W}F = (C_\tau F)|_W$ is used.
- (x) No exactness of positive-threshold deletion is used.
- (xi) No threshold-gap-safe stability theorem is reproved here; it is imported from Chapter 2 / Appendix A.
- (xii) No scalar detector, including E_1 , replaces the repaired barcode profile $\text{Eval}_{1,W,\tau}(F)$.
- (xiii) No numerical ledger slack repairs missing realization, eligibility, morphism, claim-status, or manifest data.
- (xiv) No categorical discharge cancels a failed threshold-gap, scalarized metric-ledger, Type IV, or manifest clause.
- (xv) No Search artifact, AI confidence value, spectral warning flag, CCC authorization, or advisory invariant discharges the Ext^1 -clause.
- (xvi) No problem-specific arithmetic, PDE, geometric, or categorical conclusion is proved here.

4.13 Provenance and status

This chapter uses the repaired windowed evaluation and representative discipline of Chapter 3, the metric repair of Chapter 2 and Appendix A, and the B-Gate⁺ contract of Chapter 1. Its only unconditional derived-category input is the canonical two-term field-coefficient Ext^1 -edge obtained from the standard truncation triangle. The persistence-to- H^{-1} edge realization remains a declared and auditable interface hypothesis.

Status labels for this chapter. *Core definitions:* Definitions [4.3](#), [4.4](#), [4.6](#), [4.7](#), [4.8](#), [4.9](#), [4.21](#), [4.22](#), [4.25](#), [4.32](#).

Core derived-category results: Lemma [4.12](#), Theorem [4.13](#), Corollary [4.14](#), Corollary [4.15](#).

Conditional Core/interface results: Theorem [4.18](#), Proposition [4.23](#), Proposition [4.26](#), Proposition [4.28](#), Proposition [4.30](#), Theorem [4.34](#).

Operational cautions / precision remarks: Remarks [4.1](#), [4.2](#), [4.5](#), [4.10](#), [4.11](#), [4.16](#), [4.17](#), [4.19](#), [4.27](#), [4.29](#), [4.33](#), [4.35](#).

Explicit exclusions: Subsection [4.12](#).

5 Chapter 5: Failure Taxonomy and Terminal-Generic Type IV Diagnostics

Chapter 5 contract. Type IV is a typed comparison-morphism diagnostic, not a scalar proxy. A no-tower claim records `not_invoked`; an invoked but incomplete tower claim records `undefined`; only a valid terminal-generic diagnostic may be `certified_zero` or `obstructed`. Metric budgets enter this chapter only through the declared bottleneck scalarization

$$\text{val}_B : V \longrightarrow [0, +\infty],$$

while missing morphisms, terminal-tameness, claim status, and manifest completeness remain non-scalar obligations.

5.1 Scope, relocation, and standing regime

This chapter closes the repaired Core failure taxonomy after the metric, record, representative, and Ext-edge repairs of Chapters 1–4. It consumes the following previously established data:

- (i) the repaired windowed evaluation

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F));$$

- (ii) the distinction between the pre-threshold profile

$$B_{i,W}(F) := W_W \mathbf{P}_i(F)$$

and the post-threshold repaired profile $\text{Eval}_{i,W,\tau}(F)$;

- (iii) single-profile threshold-gap readiness records and, whenever metric transport through hard deletion is used, two-sided threshold-gap comparison certificates from Chapter 3; exact comparisons instead carry their exact equality or profile-isomorphism evidence;
- (iv) admissible representative and realization records from Chapters 3–4;
- (v) the one-way categorical discharge

$$\text{Eval}_{1,W,\tau}(F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0$$

in the eligible regime of Chapter 4.

The purpose of Chapter 5 is:

- (i) to give a typed, nonexclusive failure taxonomy of Types I–V;
- (ii) to specify when a tower comparison is a mathematical morphism rather than a barcode-level suggestion;
- (iii) to define terminal-generic kernel/cokernel diagnostics on bounded and right-unbounded windows;
- (iv) to prevent undefined or inapplicable Type IV diagnostics from being reported as zero;
- (v) to integrate the resulting diagnostics with B-Gate⁺ without upgrading zero diagnostics to full comparison-map isomorphism.

Remark 5.1 (Relocation of the previous Chapter 5 material). The previous Chapter 5 material on Window Logic, Overlap Gate, Restart/Summability, and local-to-global certificate assembly is not discarded. In the v19 ordering it belongs after the present failure-and-diagnostics chapter and is therefore reserved for Chapter 6. The present chapter does not assert Restart or gluing theorems.

Remark 5.2 (Metric repair is consumed, not reproved). The sharp threshold-gap-safe stability theorem is not reproved here. Whenever a tower artifact or comparison record is built from perturbed repaired profiles, the relevant two-sided threshold-gap-certified records must already have been supplied under Chapters 1–3 and Appendix A. If the metric ledger is $\mathfrak{d}_{\text{met}} \in \mathbf{V}$, the gate-scoped real safety condition is

$$\varepsilon_{\mathbf{B}}(\mathfrak{d}_{\text{met}}) := \text{val}_{\mathbf{B}}(\mathfrak{d}_{\text{met}}) < \text{gap}_{\tau}(\mathfrak{B}) = \min_{B \in \mathfrak{B}} \text{clear}_{\tau}(B),$$

for the actual finite comparison family $\mathfrak{B}$ of pre-threshold profiles. The older notation $\Sigma\delta < \text{gap}_{\tau}$ is permitted only as an explicitly declared abbreviation for this scalarized real inequality.

5.2 Typed failure landscape

Definition 5.3 (v19 failure types). Let a Core claim be represented by a declared run record on a window W , threshold τ , and finite set of monitored degrees. The following failure types are nonexclusive.

Type I. Topological failure. The repaired degree-one profile is nonzero:

$$\text{Eval}_{1,W,\tau}(F) \neq 0.$$

If an admissible representative is supplied, this is equivalently recorded as

$$\text{PH}_1(C_{\tau,W}F) \neq 0$$

at the monitored profile level.

Type II. Categorical failure in the eligible regime. The Ext-clause is invoked, an eligible realization record exists, and

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) \neq 0.$$

Failure to supply an invoked eligibility record does not establish Type II failure; it makes the categorical clause undefined. The missing operational evidence is routed through Type III. Type V is added only when the manifest conceals the missing evidence, falsely reports pass, or is otherwise inconsistent.

Type III. Operational, metric, or functorial instability. A required finite-level operation or safety certificate is absent, undefined, or fails. This includes failure of a repaired evaluation record, a representative compatibility record, a crop budget, a window-edge clearance record, a two-sided threshold-gap certificate, an eligibility record, a profile-system morphism, a terminal-tameness condition, or a declared comparison budget. In particular, when positive-radius metric transport through hard deletion is invoked, either

$$\text{clear}_{\tau}(B) = 0 \quad \text{for a protected profile } B, \quad \text{or} \quad \text{val}_{\mathbf{B}}(\mathfrak{d}_{\text{met}}) \geq \text{gap}_{\tau}(\mathfrak{B})$$

is a Type III metric-safety failure. Likewise,

$$\text{diagnostic_status} = \text{undefined}$$

is a Type III diagnostic failure when Type IV is required by scope. By contrast, an exact comparison may remain meaningful when $\text{clear}_{\tau} = 0$; it simply carries no positive-radius threshold-gap robustness certificate.

Type IV. Tower-level terminal-generic obstruction. A tower comparison artifact valid for the claim’s declared source semantics exists and its monitored terminal-generic kernel/cokernel diagnostic is nonzero:

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0).$$

On a right-unbounded window this is an infinite-bar defect. On a bounded window it is a defect that persists cofinally to the declared right endpoint of that window.

Type V. Claim-status or manifest failure. A run cites a rejected or deprecated claim, suppresses a required missing-artifact marker, falsely reports an undefined clause as pass or zero, misstates source provenance, uses a Search/AI/advisory artifact as proof, treats a v19.1 candidate as a v19.0.0 theorem, or otherwise makes the canonical manifest incomplete or inconsistent. Mere absence of in-scope mathematical evidence is first recorded as an undefined Type III obligation; Type V concerns the governance or manifest misrepresentation of that state.

Remark 5.4 (The taxonomy is nonexclusive). A single run may exhibit several failure types simultaneously. For example, a missing two-sided gap record is Type III; if the manifest nevertheless reports a certified pass, the same run also has Type V failure. The type number is not a severity ranking.

Definition 5.5 (Failure-check order). The Core audit order is:

- (1) claim and manifest well-formedness (Type V);
- (2) metric, operational, representative, and artifact well-formedness (Type III);
- (3) topological and eligible categorical verdicts (Types I–II);
- (4) invoked tower diagnostics (Type IV);
- (5) final B-Gate⁺ conjunction and budget decision.

A later numerical zero may not repair an earlier undefined or ill-typed record.

Remark 5.6 (Undefined is not zero). The absence of a tower comparison artifact, the failure of terminal tameness, or the absence of an actual morphism between the declared source and target profiles does not imply $\mu_{\text{Collapse}} = \nu_{\text{Collapse}} = 0$. It means that the Type IV diagnostic is undefined. Reporting that state as $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ is a Type V manifest error.

Remark 5.7 (No scalar repair of non-scalar obligations). A larger metric ledger cannot manufacture a repaired tower-profile system, an actual comparison morphism, a terminal-tameness certificate, a kernel/cokernel artifact, a valid claim status, or a complete and consistent manifest. These are typed proof obligations, not numerical error terms. Consequently $\text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau$ is necessary for metric safety when invoked, but it cannot turn undefined into `certified_zero`.

5.3 Structural and audited collapse admissibility

Definition 5.8 (Type IV scope status). Every Core run records exactly one of the following Type IV scope statuses:

`not_invoked`, `undefined`, `certified_zero`, `obstructed`.

They mean respectively:

- (i) no tower, apex, limit, or colimit comparison is part of the claim;

- (ii) tower semantics are invoked, but the diagnostic input is incomplete or ill-typed;
- (iii) an artifact valid for the claim's declared source semantics exists and $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$;
- (iv) an artifact valid for the claim's declared source semantics exists and $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0)$.

Definition 5.9 (Structural collapse admissibility). A declared object $(F; W, \tau)$ is *structurally collapse-admissible* if:

- (i) $\text{Eval}_{1,W,\tau}(F) = 0$;
- (ii) if the Ext-clause is invoked, an eligible realization record exists and the categorical clause is discharged;
- (iii) the Type IV scope status is either `not_invoked` or `certified_zero`;
- (iv) no Type V claim-status defect is present in the structural record.

Definition 5.10 (Audited collapse admissibility). A structurally collapse-admissible run is *audited collapse-admissible* if, in addition:

- (i) every required repaired comparison declares exactly one comparison mode

`exact,      metric_gap_certified,      not_compared;`

every metric comparison transported through hard deletion has a two-sided threshold-gap certificate, while every exact comparison carries its exact equality or profile-isomorphism proof; `not_compared` is admissible only when no comparison inference is required by the claim;

- (ii) every crop and window-edge effect used by an exact comparison is controlled by an exact theorem or artifact, while every such metric contribution used in `metric_gap_certified` mode is represented in the metric ledger $\mathfrak{d}_{\text{met}}$;
- (iii) whenever metric mode is invoked, the ledger aggregation law and bottleneck scalarization $\text{val}_{\mathfrak{B}}$ are declared;
- (iv) for every gate-relevant finite family $\mathfrak{B}$ protected by metric transport, the strict safety inequality

$$\text{val}_{\mathfrak{B}}(\mathfrak{d}_{\text{met}}) < \text{gap}_{\tau}(\mathfrak{B})$$

holds;

- (v) every in-scope non-scalar proof obligation is present and well-typed;
- (vi) the canonical Chapter 7 manifest is complete and consistent for every clause used by the run;
- (vii) no Type III operational or metric failure remains.

Proposition 5.11 (Eligible topological vanishing discharges Type II). *Assume an eligible realization record in the sense of Chapter 4. If*

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

then

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Consequently, in the eligible regime, a Type I pass discharges the invoked Type II clause, but does not discharge Type III, Type IV, Type V, or the budget clause.

Proof. This is Theorem 4.18 together with the separation of gate clauses fixed in Chapter 1. □

5.4 Windowed tower-profile systems

Let

$$F_0 \longrightarrow F_1 \longrightarrow F_2 \longrightarrow \cdots$$

be a declared tower of filtered or persistence objects and let F_∞ be a declared apex, terminal object, or comparison target when one is supplied.

Definition 5.12 (Windowed repaired tower objects). For a monitored degree i , window W , and threshold τ , write

$$M_{n;i,W,\tau} := \text{Eval}_{i,W,\tau}(F_n).$$

These objects alone do not form a directed system, because barcode-level threshold deletion is not a global functor on arbitrary persistence morphisms.

Definition 5.13 (Repaired tower-profile system). A *repaired tower-profile system* for (i, W, τ) consists of:

- (i) the repaired objects $M_{n;i,W,\tau}$;
- (ii) actual persistence-profile morphisms

$$\psi_{n,m} : M_{n;i,W,\tau} \longrightarrow M_{m;i,W,\tau} \quad (n \leq m);$$

- (iii) the identities $\psi_{n,n} = \text{id}$ and composition law

$$\psi_{m,r} \circ \psi_{n,m} = \psi_{n,r};$$

- (iv) provenance showing that the morphisms are either explicitly constructed or supplied canonically by a declared τ -split/exact-admissible regime.

A sequence of barcode multisets or bottleneck matchings without these morphisms is not a repaired tower-profile system.

Definition 5.14 (Collapsed source profile). Given a repaired tower-profile system, a source profile

$$\text{ColimProfile}_{i,W,\tau}(F_\bullet)$$

is declared with exactly one provenance status:

$$\text{genuine_colimit}, \quad \text{verified_stabilization}, \quad \text{implementation_artifact}.$$

In the first case it is a genuine colimit in the declared constructible profile category. In the second it is a certified stabilized profile; it is identified with a categorical colimit only when a separate stabilization-to-colimit theorem is recorded. In the third it is an implementation object whose relation to the tower is part of the artifact and is not silently promoted either to a categorical colimit or to a verified stabilization.

Every claim also declares the source semantics it requires. A source artifact is admissible for that claim only when its provenance has sufficient semantic authority. In particular, an `implementation_artifact` may support an implementation-scoped comparison, but it cannot discharge a claim about a genuine tower colimit merely by being named `ColimProfile`.

Definition 5.15 (Collapsed apex profile). The repaired apex profile is

$$\text{ApexProfile}_{i,W,\tau}(F_\infty) := \text{Eval}_{i,W,\tau}(F_\infty),$$

provided the repaired evaluation record for F_∞ is declared.

Remark 5.16 (No automatic colimit–collapse interchange). No identity of the form

$$\lim_{\rightarrow n} \text{Eval}_{i,W,\tau}(F_n) \cong \text{Eval}_{i,W,\tau}(F_\infty)$$

or

$$\lim_{\rightarrow n} T_\tau^{\text{bar}}(M_n) \cong T_\tau^{\text{bar}}\left(\lim_{\rightarrow n} M_n\right)$$

is available in the general Core. Any such comparison is exactly the data that the tower artifact must supply or justify.

5.5 Tower comparison artifacts

Definition 5.17 (Tower comparison artifact). A *tower comparison artifact* for (i, W, τ) consists of the following declared data:

- (i) a repaired tower-profile system;
- (ii) a source profile $\text{ColimProfile}_{i,W,\tau}(F_\bullet)$ with provenance status;
- (iii) a repaired apex profile $\text{ApexProfile}_{i,W,\tau}(F_\infty)$;
- (iv) an actual morphism in the declared constructible profile category on W ,

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty);$$

- (v) the construction or proof establishing that $\phi_{i,W,\tau}$ is well-typed;
- (vi) the claim-required source semantics and evidence that the source provenance has sufficient semantic authority for that claim;
- (vii) the repaired evaluation, comparison-mode, crop-budget, and representative records used in constructing the source, target, or comparison; every metric comparison transported through hard deletion carries a two-sided threshold-gap certificate, whereas an exact comparison carries its exact proof;
- (viii) the terminal mode and terminal-tameness certificate used by the diagnostic;
- (ix) an explicitly declared abelian constructible profile category on W , closed under the kernels and cokernels used here, together with kernel and cokernel computation artifacts or their categorical proof.

Remark 5.18 (A bottleneck matching is not the comparison morphism). A bottleneck matching controls metric discrepancy between barcodes. It is not, by itself, a natural transformation of persistence modules and therefore does not define the kernel or cokernel needed by Type IV. The tower artifact must contain an actual morphism $\phi_{i,W,\tau}$ or a canonical construction of one. Moreover, the morphism must live in the declared abelian profile category and its source provenance must match the semantic strength required by the claim.

Definition 5.19 (Exact-admissible tower artifact). A tower comparison artifact is *exact-admissible* if its profile-system morphisms, source construction, comparison map, kernels, and cokernels are supplied by a declared regime in which the retained-long-bar assignment is functorial and preserves every exact sequence used by the diagnostic.

Remark 5.20 (Explicit artifacts remain allowed). Exact admissibility is sufficient but not necessary. An explicitly constructed morphism between declared repaired source and apex profiles is also admissible, provided all typing, source-semantics, provenance, ambient-abelian-category, kernel/cokernel, and terminal-tameness obligations are met. Artifact validity is always relative to the claim’s declared source semantics; a lower-authority implementation source does not become a categorical colimit by successful kernel/cokernel computation.

5.6 Terminal-generic dimension

The old shorthand “generic dimension equals infinite-bar multiplicity” is correct only on a right-unbounded window. On a bounded window, zero extension outside the window makes the ordinary $t \rightarrow +\infty$ dimension identically zero and would render the diagnostic vacuous. The following window-relative definition removes that ambiguity.

Definition 5.21 (Terminal-tame profile). Let $W = [u, v)$, where $v \in \mathbb{R} \cup \{+\infty\}$, and let M be a constructible persistence profile on W . We call M *terminal-tame on W* if:

- (i) every fiber near the terminal end is finite-dimensional;
- (ii) if $v < +\infty$, there exists $s < v$ such that every structure map $M(t) \rightarrow M(t')$ with $s \leq t \leq t' < v$ is an isomorphism;
- (iii) if $v = +\infty$, there exists $s \in \mathbb{R}$ such that every structure map $M(t) \rightarrow M(t')$ with $s \leq t \leq t'$ is an isomorphism.

Definition 5.22 (Terminal-generic dimension). For a terminal-tame profile M on $W = [u, v)$, define

$$\text{tgdim}_W(M) := \begin{cases} \lim_{t \uparrow v} \dim_k M(t), & v < +\infty, \\ \lim_{t \rightarrow +\infty} \dim_k M(t), & v = +\infty. \end{cases}$$

When $v = +\infty$, we may also write $\text{gdim}(M)$.

Proposition 5.23 (Well-definedness of terminal-generic dimension). *For every terminal-tame profile M on $W = [u, v)$, the limit in Definition 5.22 exists and is a finite nonnegative integer.*

Proof. By terminal tameness, there is a terminal subinterval on which every structure map is an isomorphism. Hence the finite-dimensional fibers on that subinterval all have the same dimension. The displayed dimension function is therefore eventually constant as $t \uparrow v$ when $v < +\infty$, or as $t \rightarrow +\infty$ when $v = +\infty$. Its eventual value is finite and is exactly the asserted limit. $\square$

Definition 5.24 (Cofinal bar in a window). A bar I in a barcode profile on $W = [u, v)$ is *cofinal in W* if:

- (i) $\sup I = v$ when $v < +\infty$; or
- (ii) I is unbounded above when $v = +\infty$.

For bounded W , a cofinal bar is a boundary-reaching bar relative to W , not a global infinite bar.

Remark 5.25 (Interval decomposition and terminal tameness are distinct inputs). In the declared constructible one-parameter regime, the monitored profile is interval-decomposable. Terminal tameness is an additional right-end condition; constructibility on bounded subintervals alone does not silently supply a common terminal tail. Proposition 5.26 uses both facts.

Proposition 5.26 (Terminal dimension counts cofinal bars). *For a terminal-tame, interval-decomposable object M on W , the terminal dimension equals the multiplicity of cofinal bars:*

$$\mathrm{tgdim}_W(M) = \#\{\text{bars cofinal in } W\},$$

where multiplicity is counted. On a right-unbounded window, the right-hand side counts the infinite bars.

Proof. Choose a terminal tail on which all structure maps are isomorphisms. A bar contributes one dimension to every fiber on that tail exactly when it is cofinal in W . Every noncofinal bar dies before the terminal tail and contributes zero there. Summing the surviving interval summands gives the stated equality. $\square$

Remark 5.27 (Interpretation of earlier gdim notation). The gdim -notation used schematically in Chapters 1–2 is interpreted by this chapter as the terminal-generic dimension appropriate to the declared window. For $W = [u, +\infty)$ it is the ordinary eventual dimension and counts infinite bars. For bounded $W = [u, v)$ the canonical local diagnostic is tgdim_W , which counts bars reaching v from the left.

Proposition 5.28 (Terminal tameness passes to defects). *Let $\phi : M \rightarrow N$ be a morphism in a declared abelian constructible profile category on W , closed under the pointwise kernels and cokernels used by the diagnostic. If M and N are terminal-tame, then $\ker \phi$ and $\mathrm{coker} \phi$ are terminal-tame.*

Proof. On a common terminal tail, the structure maps of M and N are isomorphisms and the naturality squares for ϕ commute. The induced structure maps on pointwise kernels and cokernels are therefore isomorphisms on the same tail. Finite-dimensionality is inherited pointwise. $\square$

5.7 Defect profiles and Type IV diagnostics

Definition 5.29 (Defect profiles). For a valid tower comparison artifact, define

$$\mathrm{Defect}_{i,W,\tau}^{\ker} := \ker(\phi_{i,W,\tau}), \quad \mathrm{Defect}_{i,W,\tau}^{\mathrm{coker}} := \mathrm{coker}(\phi_{i,W,\tau}).$$

These are objects of the declared constructible profile category on W .

Definition 5.30 (Degreewise Type IV diagnostics). Assume the defect profiles are terminal-tame. Define

$$\mu_{i,W,\tau} := \mathrm{tgdim}_W(\mathrm{Defect}_{i,W,\tau}^{\ker}), \quad \nu_{i,W,\tau} := \mathrm{tgdim}_W(\mathrm{Defect}_{i,W,\tau}^{\mathrm{coker}}).$$

Thus μ measures terminal-generic kernel defect and ν measures terminal-generic cokernel defect.

Definition 5.31 (Total collapse diagnostics). For a fixed declared window–threshold pair (W, τ) and a finite declared monitored-degree set I_{mon} , define

$$\mu_{\mathrm{Collapse}} := \sum_{i \in I_{\mathrm{mon}}} \mu_{i,W,\tau}, \quad \nu_{\mathrm{Collapse}} := \sum_{i \in I_{\mathrm{mon}}} \nu_{i,W,\tau}.$$

The legacy notations $u_{i,W,\tau}$ and u_{Collapse} are deprecated and have no normative use in v19. If the monitored-degree scope is infinite, neither total is defined without a separately proved aggregation or summability theorem and an explicit aggregation policy.

Definition 5.32 (Type IV diagnostic record). A Type IV diagnostic record contains the window, threshold, monitored degrees, declared tower scope, and exactly one status

$$\text{not_invoked}, \quad \text{undefined}, \quad \text{certified_zero}, \quad \text{obstructed}.$$

Its remaining required fields depend on that status.

- (i) For `not_invoked`, the record contains an explicit scope rationale showing that no tower, apex, limit, or colimit comparison belongs to the claim. No numerical μ, ν values are asserted.
- (ii) For `undefined`, the record contains either the identifier of a partial or invalid artifact, or an explicit missing-artifact marker, together with the list of missing or ill-typed obligations. No numerical μ, ν values are asserted as Core diagnostics.
- (iii) For `certified_zero` or `obstructed`, the record contains:
 - (a) a valid tower comparison artifact identifier, source-profile provenance status, claim-required source semantics, and semantic-authority check;
 - (b) the terminal mode `bounded_terminal` or `right_unbounded`;
 - (c) terminal-tameness certificates for the defect profiles;
 - (d) kernel and cokernel defect-profile artifacts;
 - (e) all degree-wise and total μ, ν values over the fixed (W, τ) and declared finite set I_{mon} ;
 - (f) the interior-defect status, recorded separately from terminal-generic vanishing, with exactly one value

`not_checked,      certified_absent,      certified_present.`

For the interior-defect field, `certified_absent` requires proof that both full defect profiles vanish. An explicit nonzero interior-defect witness is required for `certified_present`.

The status is `certified_zero` exactly when all invoked inputs are valid for the claim's declared source semantics and $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$. It is `obstructed` exactly when those inputs are valid and $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0)$. Both verdicts are relative to the declared source semantics and monitored scope. An absent artifact, insufficient-provenance source, or source-semantics mismatch is represented by a missing/invalid-obligation marker and yields `undefined`, never zero.

Definition 5.33 (Type IV obstruction and Type IV-free status). Relative to a valid tower comparison artifact whose source semantics match the claim:

- (i) the run has a *Type IV obstruction* if $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0)$;
- (ii) the run is *Type IV-free in the monitored terminal-generic sense* if $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$. Both statements are relative to the fixed (W, τ) , the finite monitored-degree set, and the source semantics recorded by the artifact.

Proposition 5.34 (Characterization of certified Type IV-free status). *A tower record is certified Type IV-free if and only if:*

- (i) *the tower comparison artifact is valid;*
- (ii) *its kernel and cokernel defect profiles are terminal-tame;*
- (iii) $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$.

Proof. This is the defining conjunction in Definitions 5.32 and 5.33. The validity and terminal-tameness clauses are necessary to ensure that the displayed zero is a defined Core diagnostic rather than a missing value. $\square$

Remark 5.35 (Zero diagnostics are not a full isomorphism theorem). The equality

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$$

says that no defect bar reaches the terminal end of the declared window. It does not imply

$$\ker \phi_{i,W,\tau} = 0 \quad \text{or} \quad \text{coker } \phi_{i,W,\tau} = 0.$$

Finite interior defect bars may remain. A full isomorphism conclusion requires a separate no-interior-defect proof establishing that the entire kernel and cokernel profiles vanish, not merely their terminal-generic parts.

Example 5.36 (Zero terminal diagnostics with a nonisomorphism). Let $W = [0, 10)$ and consider the morphism

$$0 \longrightarrow k_{[2,3)}.$$

Its cokernel is $k_{[2,3)}$, so the morphism is not an isomorphism. However, the cokernel has no bar cofinal in W , hence

$$\text{tgdim}_W(k_{[2,3)}) = 0.$$

Thus the terminal-generic diagnostic is zero while a finite interior defect remains.

Example 5.37 (Pure kernel, pure cokernel, and mixed Type IV modes). On a window $W = [u, v)$, let K_W denote one cofinal interval bar: $k_{[u,v)}$ when $v < +\infty$ and $k_{[u,+\infty)}$ when $v = +\infty$. Then:

$$\begin{aligned} K_W \rightarrow 0 & \text{ has } (\mu, \nu) = (1, 0), \\ 0 \rightarrow K_W & \text{ has } (\mu, \nu) = (0, 1), \\ K_W \oplus 0 \rightarrow 0 \oplus K_W & \text{ has } (\mu, \nu) = (1, 1) \end{aligned}$$

for the evident zero maps. These are the pure-kernel, pure-cokernel, and mixed terminal-generic Type IV modes. The mixed map is explicitly the zero morphism from $K_W \oplus 0$ to $0 \oplus K_W$.

Remark 5.38 (Finite-layer checks do not determine Type IV). Finite-layer repaired evaluations do not logically determine the kernel or cokernel of an undeclared source-to-apex comparison. For example, a zero source profile together with a declared apex containing a cofinal bar produces a pure cokernel defect once the comparison $0 \rightarrow K_W$ is declared. This is an artifact-level mismatch witness; it is not a claim that an arbitrary apex is a categorical colimit.

5.8 Artifact-coherent stable bands

Definition 5.39 (Artifact-coherent Type IV stable band). Fix a window W , monitored degrees, a tower-profile construction, and a terminal mode. An interval $J \subset (0, +\infty)$ is an *artifact-coherent Type IV stable band* if:

- (i) for every $\tau \in J$, a valid tower comparison artifact $\phi_{i,W,\tau}$ is supplied by one declared construction rule;
- (ii) the claim-required source semantics, source provenance, semantic authority, and terminal mode do not change silently across J ;
- (iii) all defect profiles are terminal-tame;
- (iv) every repaired comparison used by the artifact declares a comparison mode. In metric mode, the record carries the ledger $\mathfrak{d}_{\text{met},\tau}$, scalarization val_B , evidenced discrepancy bound, and strict inequality

$$\text{val}_B(\mathfrak{d}_{\text{met},\tau}) < \text{gap}_\tau(\mathfrak{B}_\tau);$$

in exact mode it carries the exact equality or profile-isomorphism proof and makes no automatic positive-radius robustness claim;

(v) the associated Type IV record has status `certified_zero` for every $\tau \in J$.

Remark 5.40 (Stable band is an artifact family, not pointwise wishful thinking). Pointwise numerical zeros obtained from unrelated comparison maps do not define a stable band. The artifact family, source provenance, terminal convention, and comparison policy must remain auditable throughout the band. If a single budgeted perturbation is claimed uniformly on J , the manifest must provide a common ledger $\mathbf{d}_{\text{met},J}$ and verify the uniform reserve condition

$$\text{val}_B(\mathbf{d}_{\text{met},J}) < \inf_{\tau \in J} \text{gap}_\tau(\mathfrak{B}_\tau).$$

A pointwise family of positive reserves does not by itself imply that this infimum is positive.

Proposition 5.41 (No terminal-generic Type IV obstruction on a stable band). *If J is an artifact-coherent Type IV stable band, then every $\tau \in J$ is Type IV-free in the monitored terminal-generic sense relative to its declared artifact.*

Proof. By the stable-band definition, every threshold in J has a valid diagnostic record whose status is

`certified_zero`.

The certified-zero characterization then applies pointwise. □

5.9 Integration with B-Gate⁺

Definition 5.42 (Type IV gate discharge). The Type IV clause of B-Gate⁺ is discharged exactly as follows:

- (i) if the claim has no tower/apex/limit semantics, the manifest records `not_invoked`;
- (ii) if tower semantics are invoked, a valid Type IV record with status `certified_zero` is required, and the artifact's source provenance must have sufficient authority for the claim's declared source semantics.

The statuses `undefined` and `obstructed` fail the gate.

Remark 5.43 (Meaning of $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ in the gate). The shorthand

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$$

is used only in the invoked-tower case after the diagnostic has been shown to be defined. A no-tower claim uses `not_invoked`, not a fabricated numerical zero.

Proposition 5.44 (Audited admissibility implies the structural and safety clauses of B-Gate⁺). *If a run is audited collapse-admissible in the sense of Definition 5.10, then:*

- (i) *its topological clause passes;*
- (ii) *its invoked categorical clause passes;*
- (iii) *its Type IV clause is either correctly out of scope or certified zero;*
- (iv) *every invoked metric threshold-gap and strict budget clause passes, and every exact comparison carries its exact proof;*
- (v) *no Type III or Type V defect remains in those records.*

Hence the corresponding $B\text{-Gate}^+$ record passes for every clause marked required by the run's declared Core scope. This does not by itself yield any domain-specific external conclusion; such a conclusion still requires its separately proved Interface or Bridge theorem.

Proof. Each item is one of the defining clauses of structural or audited collapse admissibility. The final statement is the $B\text{-Gate}^+$ conjunction fixed in Chapter 1, with the Type IV scope refinement of Definition 5.42. $\square$

Definition 5.45 (Failure routing record). A failure routing record contains:

- (i) every applicable failure type among I–V;
- (ii) the first failed audit stage in Definition 5.5;
- (iii) the exact failed clause and either the relevant artifact identifier or an explicit missing-artifact marker;
- (iv) whether the failure is mathematical, operational, diagnostic, or manifest-level;
- (v) the permitted repair action, if any, without changing the claim status.

No scalar score replaces this typed routing record.

5.10 Failure-and-diagnostics contract

Definition 5.46 (v19 failure-and-diagnostics contract). The normative Chapter 5 contract is the following.

- (i) Types I–V are nonexclusive and are checked in the declared audit order.
- (ii) Type II is meaningful only in the eligible regime of Chapter 4.
- (iii) Type III includes threshold-gap, crop-budget, representative, profile-system, artifact-typing, and terminal-tameness failures.
- (iv) Type IV is defined only relative to a valid comparison morphism and terminal-generic defect profiles.
- (v) Type V includes false pass declarations, deprecated claims, suppressed or misrepresented mandatory records, provenance overstatement, and undefined-as-zero substitutions.
- (vi) On bounded windows, Type IV counts defects cofinal to the right endpoint; on right-unbounded windows, it counts infinite-bar defects.
- (vii) $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ is weaker than full comparison-map isomorphism.
- (viii) $u_{i,W,\tau}$ and u_{Collapse} are deprecated; $\nu_{i,W,\tau}$ and ν_{Collapse} are canonical.
- (ix) A no-tower claim records `not_invoked`; an invoked tower claim requires a valid `certified_zero` record to pass.
- (x) Search, AI, spectral, or advisory proxies cannot replace the actual tower comparison artifact and its kernel/cokernel diagnostics.

Remark 5.47 (Contract, not a mixed theorem bundle). Definition 5.46 is a typing and audit contract. Its mathematical content is supplied by the preceding definitions and propositions; it is not presented as a theorem combining definitions, policies, and proved implications into one undifferentiated statement.

5.11 Minimal manifest consequences

Remark 5.48 (Manifest hierarchy and canonical audit schema). The entries below extend the Chapter 1 Minimal Manifest and the Chapter 3 evaluation/representative records with failure-routing and Type IV data. They are not a competing manifest definition. Chapter 7 supplies the canonical v19.0.0 audit manifest into which these entries are embedded. An incomplete invoked Type IV entry yields `undefined`; an inconsistent entry is a Type V rejection.

Definition 5.49 (Type IV manifest entry). Every run records the Type IV scope status. The remaining manifest fields are status-dependent.

- (i) For `not_invoked`: the scope rationale and the claim clause declaring tower semantics out of scope.
- (ii) For `undefined`: the tower-profile, source, apex, or comparison identifiers that exist, together with an explicit missing/invalid-obligation marker for every absent mandatory object, source-semantics mismatch, insufficient-provenance source, absent ambient abelian category, failed typing, or failed terminal-tameness obligation. No Core numerical diagnostic is serialized as zero.
- (iii) For `certified_zero` or `obstructed`: the tower-profile system identifier; source and apex identifiers; source provenance; claim-required source semantics and semantic-authority verdict; valid comparison morphism artifact identifier; ambient abelian profile category; window, threshold, monitored degrees, and evaluation order; terminal mode and terminal-tameness artifacts; defect-profile artifacts; all degree-wise and total μ, ν values; diagnostic status; and the separate interior-defect status.

If metric perturbation enters the artifact construction, the entry also records $\mathfrak{d}_{\text{met}}$, $\text{val}_{\mathfrak{B}}$, the real budget $\varepsilon_{\mathfrak{B}} = \text{val}_{\mathfrak{B}}(\mathfrak{d}_{\text{met}})$, and the gate-scoped comparison family $\mathfrak{B}$.

Definition 5.50 (Failure manifest entry). Every failed or conditionally accepted run records:

- (i) the applicable failure types;
- (ii) the first failed audit stage;
- (iii) the failed claim, theorem, gate, or artifact reference;
- (iv) whether the result is rejected, repairable, conditional, or out of scope;
- (v) the version boundary, including any v19.1-only candidate used in analysis but excluded from proof.

5.12 Explicit non-claims

The following are not asserted in Chapter 5.

- (i) No sequence of repaired barcode objects is automatically a directed system.
- (ii) No bottleneck matching is automatically a persistence-module morphism.
- (iii) No automatic comparison

$$\lim_{\overrightarrow{n}} \text{Eval}_{i,W,\tau}(F_n) \longrightarrow \text{Eval}_{i,W,\tau}(F_{\infty})$$

is asserted.

- (iv) No exactness or global functoriality of T_{τ}^{bar} is used.

- (v) No undefined diagnostic is interpreted as zero.
- (vi) No bounded-window terminal bar is called a global infinite bar.
- (vii) No equality $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ is promoted to full comparison-map isomorphism without a separate no-interior-defect theorem.
- (viii) No Type IV result is accepted without an actual comparison morphism, a source whose provenance has sufficient authority for the claim, a declared abelian profile category, terminal-tameness data, and defect-profile artifacts; their absence or mismatch is recorded explicitly and yields `undefined`.
- (ix) No scalar ledger slack repairs a missing morphism, missing terminal-tameness certificate, invalid claim status, or incomplete manifest.
- (x) No Type II verdict is issued without eligibility.
- (xi) No scalar potential, Search artifact, AI confidence, spectral warning, or advisory invariant replaces Types I–V, B-Gate⁺, or the Type IV artifact.
- (xii) No Restart/Summability, Čech gluing, or local-to-global certificate theorem is proved here; those are deferred to Chapter 6.
- (xiii) No v19.1 candidate, including any stronger NBT transport package, is used as a v19.0.0 theorem.

5.13 Provenance and status

Chapter 5 depends on Chapter 1 for the five-type failure landscape and B-Gate⁺, Chapter 2 and Appendix A for repaired persistence and metric safety, Chapter 3 for repaired evaluation and representative records, and Chapter 4 for the conditional eligible one-way Ext discharge. Its new mathematical clarification is the window-relative terminal-generic diagnostic, which prevents bounded-window zero extension from making Type IV vacuous. Its record definitions extend the Chapter 1 and Chapter 3 manifests and are serialized through the canonical Chapter 7 audit manifest.

Core mathematical propositions. Propositions [5.11](#), [5.23](#), [5.26](#), [5.28](#), [5.34](#), [5.41](#), and [5.44](#).

Core definitions and contracts. Definitions [5.3](#), [5.5](#), [5.8](#), [5.9](#), [5.10](#), [5.13](#), [5.14](#), [5.17](#), [5.21](#), [5.22](#), [5.30](#), [5.31](#), [5.32](#), [5.33](#), [5.39](#), [5.42](#), and [5.46](#).

Operational examples and cautions. Examples [5.36](#) and [5.37](#); Remarks [5.6](#), [5.7](#), [5.25](#), [5.48](#), [5.18](#), [5.16](#), [5.27](#), [5.35](#), and [5.43](#).

Explicit exclusions. Section [5.12](#).

6 Chapter 6: Window Logic, Overlap, κ -Restart, and Record-Level Gluing

6.1 Scope, dependencies, and relocation discipline

This chapter works in the v19 repaired Core established by Chapters 1–5 and Appendix A. Its inputs are:

- (i) the pre-threshold window profile

$$\mathcal{B}_{i,w}(F) := W_w \mathbf{P}_i(F);$$

- (ii) the repaired post-threshold evaluation

$$\text{Eval}_{i,w,\tau}(F) := T_\tau^{\text{bar}}(\mathcal{B}_{i,w}(F));$$

- (iii) the sharp two-sided threshold-gap theorem with closed band

$$[\tau - \varepsilon, \tau + \varepsilon];$$

- (iv) the convention

$$\text{gap}_\tau = \text{clear}_\tau$$

as a ledger-facing alias;

- (v) the representative and eligible-realization disciplines of Chapters 3–4;
- (vi) the typed Type IV status and terminal-generic diagnostics of Chapter 5.

The purpose of this chapter is to define exactly when local certificate records may be compared, continued, and assembled without reintroducing an unsupported global theorem. It develops:

- (i) right-open partition and overlap-cover logic;
- (ii) sharp two-sided Overlap Gate certificates;
- (iii) finite Čech-depth coherence at the *record* level;
- (iv) conditional κ -admissible Restart;
- (v) the correct weighted summability condition for countable restart;
- (vi) countable certificate atlases under explicit local-finiteness and coherence hypotheses.

Remark 6.1 (Relocation of the former Chapter 6). The former v18.1 Chapter 6 material on the δ -ledger, Audit Mode, proposer–verifier separation, and Core sovereignty is not discarded. In the v19 main-text order it is relocated to Chapter 7, after the present chapter has fixed the window, overlap, restart, and gluing records that the audit chapter must consume. The present chapter uses ledger values but does not redefine the final audit-verdict semantics.

Remark 6.2 (No automatic local-to-global theorem). Neither a family of local B-Gate⁺ passes nor pairwise small bottleneck distance automatically produces a global persistence object, a global filtered representative, or a global theorem. This chapter certifies only the compatibility and propagation of declared certificate records unless an independent object-level gluing theorem is supplied.

6.2 Right-open domains, partitions, and overlap covers

Definition 6.3 (Right-open Core window). A *Core window* is a right-open interval

$$W = [u, v), \quad -\infty < u < v \leq +\infty.$$

It is *bounded* if $v < +\infty$, and *right-unbounded* if $v = +\infty$. All local profiles in this chapter are evaluated directly on the declared window before threshold deletion.

Definition 6.4 (Right-open partition cover). A family

$$\mathcal{W}_{\text{part}} = \{W_k = [u_k, u_{k+1})\}_{k \in K}$$

is a *right-open partition cover* of a declared range $X \subseteq \mathbb{R}$ if:

- (i) the windows are pairwise disjoint under the right-open endpoint convention;
- (ii) $\bigcup_{k \in K} W_k = X$;
- (iii) the indexing order is part of the data whenever sequential continuation or Restart is used.

The former term “MECE windowing” may be retained as an operational alias, but the mathematical object is a right-open partition cover.

Definition 6.5 (Overlap-admitting cover). A family

$$\mathcal{W}_{\text{ov}} = \{W_\alpha\}_{\alpha \in A}$$

of right-open windows is an *overlap-admitting cover* of X if

$$X = \bigcup_{\alpha \in A} W_\alpha$$

and every nonempty finite intersection used by the certificate logic is explicitly indexed and recorded. For a finite index set $\sigma = \{\alpha_0, \dots, \alpha_p\}$, write

$$W_\sigma := W_{\alpha_0} \cap \dots \cap W_{\alpha_p}.$$

Remark 6.6 (Partition and overlap covers have different jobs). A right-open partition cover is used for sequential Restart and accumulated-budget control. An overlap-admitting cover is used for compatibility and Čech-type coherence. A disjoint partition has no nontrivial overlap on which an Overlap Gate could be evaluated; conversely, an overlap cover does not by itself provide an ordered continuation path.

Definition 6.7 (Direct common-window profile). Let local objects F_α and F_β be declared on windows W_α and W_β , and suppose $W_{\alpha\beta} := W_\alpha \cap W_\beta \neq \emptyset$. Their pre-threshold common-window profiles are

$$\mathcal{B}_{i,\alpha\beta}^\alpha := W_{W_{\alpha\beta}} \mathbf{P}_i(F_\alpha), \quad \mathcal{B}_{i,\alpha\beta}^\beta := W_{W_{\alpha\beta}} \mathbf{P}_i(F_\beta).$$

For a declared common threshold $\tau_{\alpha\beta}$, the corresponding post-threshold profiles are

$$\text{Eval}_{i,\alpha\beta}^\alpha := T_{\tau_{\alpha\beta}}^{\text{bar}}(\mathcal{B}_{i,\alpha\beta}^\alpha), \quad \text{Eval}_{i,\alpha\beta}^\beta := T_{\tau_{\alpha\beta}}^{\text{bar}}(\mathcal{B}_{i,\alpha\beta}^\beta).$$

Remark 6.8 (No crop-after-collapse substitution). The profiles in Definition 6.7 are computed directly on the common window. They are not replaced by cropping a profile already collapsed on W_α or W_β , unless a separate window-adapted commutation certificate is supplied. This prevents the false identity

$$W_{W_{\alpha\beta}} T_\tau^{\text{bar}} \cong T_\tau^{\text{bar}} W_{W_{\alpha\beta}}$$

from entering the overlap proof through notation. When such a certificate is invoked, it must use the necessary-and-sufficient condition of Chapter 2 and Appendix A: every bar retained before cropping, including every infinite bar, is either disjoint from the common window or remains retained after nonempty clipping. A finite-bar-only check is insufficient.

6.3 Metric scalarization of the ledger

The threshold-gap theorem is stated in the real-valued bottleneck metric. Consequently, an abstract quantale-valued ledger cannot be compared directly with clear_τ unless a metric scalarization has been declared.

Definition 6.9 (Bottleneck scalarization). Let

$$(\mathbb{V}, \oplus, 0_{\mathbb{V}}, \preceq)$$

be the declared commutative ledger quantale. A *bottleneck scalarization* is a monotone, subadditive map

$$\text{val}_{\mathbb{B}} : \mathbb{V} \longrightarrow [0, +\infty]$$

satisfying

$$\text{val}_{\mathbb{B}}(0_{\mathbb{V}}) = 0, \quad \text{val}_{\mathbb{B}}(q \oplus q') \leq \text{val}_{\mathbb{B}}(q) + \text{val}_{\mathbb{B}}(q').$$

It converts a metric-ledger element into a real bottleneck-discrepancy upper bound. The manifest must also identify the actual profile comparison to which that upper bound applies.

Definition 6.10 (Scalarized metric ledger). For a gate-scoped metric ledger record

$$\begin{aligned} \mathfrak{d}_{\text{met}} = & \delta^{\text{alg}} \oplus \delta^{\text{disc}} \oplus \delta^{\text{meas}} \oplus \delta^{\text{rep,met}} \oplus \delta^{\text{real,met}} \oplus \delta^{\text{cmp}} \\ & \oplus \delta^{\text{crop}} \oplus \delta^{\text{cont}} \oplus \delta^{\text{ov}} \oplus \delta^{\text{rst}}, \end{aligned}$$

define its scalarized bottleneck budget by

$$\varepsilon_{\mathbb{B}}(\mathfrak{d}_{\text{met}}) := \text{val}_{\mathbb{B}}(\mathfrak{d}_{\text{met}}).$$

Every displayed component is a metric-discrepancy contribution only. In particular, $\delta^{\text{rep,met}}$ and $\delta^{\text{real,met}}$ may measure proved profile distortion introduced by representative or realization transport; they do not encode the existence, typing, or compatibility of a representative or realization. Only components actually consumed by the audited comparison are included. Each component retains its provenance and may not be silently reused by an unrelated gate. Crop, overlap, continuation, and restart contributions remain separately addressable even when $\text{val}_{\mathbb{B}}$ eventually combines them into one real upper bound.

Definition 6.11 (Non-scalar continuation and coherence obligations). The following are typed proof obligations rather than metric-ledger components:

- (i) existence and typing of local representatives and realization data;
- (ii) actual overlap, face, continuation, and tower comparison morphisms when such morphisms are invoked;
- (iii) threshold reconciliation, κ_k -admissibility, terminal tameness, and Type IV certificates;
- (iv) higher-simplex compatibility and face coherence;
- (v) claim status, manifest completeness, consistency, and artifact readability.

No increase in $\varepsilon_{\mathbb{B}}$, no larger threshold gap, and no summability estimate can discharge a missing obligation in this list.

Remark 6.12 (Meaning of $\Sigma\delta < \text{gap}_\tau$). Whenever this chapter writes

$$\Sigma\delta < \text{gap}_\tau,$$

it means the gate-scoped typed inequality

$$\varepsilon_B(\mathbf{d}_{\text{met}}) = \text{val}_B(\mathbf{d}_{\text{met}}) < \text{gap}_\tau(\mathfrak{B}).$$

An unscalarized quantale element is not compared directly with the real number clear_τ , and a budget certified for one profile family $\mathfrak{B}$ is not silently spent again on another family.

Remark 6.13 (Sharp threshold band versus scalarization tightness). The word *sharp* in this chapter refers to the optimal half-width ε of the two-sided forbidden band

$$[\tau - \varepsilon, \tau + \varepsilon]$$

when ε is a valid bottleneck-distance upper bound. It does not assert that the chosen scalarization val_B is numerically tight. If $\text{val}_B(\mathbf{d}_{\text{met}})$ overestimates the true bottleneck error, the resulting certificate is conservative even though the underlying threshold theorem remains sharp.

Definition 6.14 (Gate-scoped threshold reserve). Let $\mathfrak{B} = \{B_j\}_{j \in J}$ be the finite family of pre-threshold profiles consumed by one declared local, overlap, or restart gate. Define

$$\text{gap}_\tau(\mathfrak{B}) := \min_{j \in J} \text{clear}_\tau(B_j),$$

and, for its scalarized metric budget ε , define the *threshold reserve*

$$\text{res}_\tau(\mathfrak{B}; \varepsilon) := \text{gap}_\tau(\mathfrak{B}) - \varepsilon.$$

The gate has positive reserve precisely when

$$\varepsilon < \text{gap}_\tau(\mathfrak{B}).$$

This reserve belongs to the declared gate and profile family; it is not a globally reusable balance.

Remark 6.15 (The gap is measured before thresholding). The family $\mathfrak{B}$ in Definition 6.14 consists of pre-threshold profiles. The post-threshold evaluations are used for the compatibility verdict, but they do not define the threshold clearance.

6.4 Sharp two-sided Overlap Gate

Definition 6.16 (Threshold reconciliation record). If two local records use different thresholds τ_α and τ_β , an overlap comparison is well-typed only after declaring a common overlap threshold $\tau_{\alpha\beta}$ and directly re-evaluating both local objects on $W_{\alpha\beta}$ at that threshold. The data establishing this choice are called a *threshold reconciliation record*. No comparison between differently thresholded outputs is silently interpreted as an Overlap Gate.

Definition 6.17 (Overlap comparison mode). Every overlap inference records exactly one comparison mode:

$$\text{exact}, \quad \text{metric_gap_certified}, \quad \text{not_compared}.$$

The modes mean:

- (i) `exact`: a direct equality or persistence-profile isomorphism on the directly evaluated common window is proved and artifact-backed;
- (ii) `metric_gap_certified`: a real pre-threshold distance bound and a two-sided threshold-gap certificate are supplied at the same budget;
- (iii) `not_compared`: no overlap-comparison conclusion is consumed by the claim.

An `exact` comparison proves the declared relation for the specified profiles. It does not, by itself, certify positive-radius robustness. The mode `not_compared` cannot discharge an overlap obligation. In particular, it is unavailable when the governing claim consumes the comparison.

Definition 6.18 (Exact overlap certificate). An *exact overlap certificate* on $(i, W_{\alpha\beta}, \tau_{\alpha\beta})$ contains:

- (i) the direct common-window pre-threshold and repaired profiles;
- (ii) the common threshold and threshold-reconciliation record;
- (iii) an actual equality or persistence-profile isomorphism

$$\text{Eval}_{i,\alpha\beta}^\alpha \xrightarrow{\sim} \text{Eval}_{i,\alpha\beta}^\beta;$$

- (iv) immutable source, target, proof, and artifact identifiers.

No threshold-gap reserve is inferred merely from this exact certificate.

Definition 6.19 (Two-sided overlap metric certificate). Fix a monitored degree i , an overlap $W_{\alpha\beta}$, a common threshold $\tau_{\alpha\beta}$, and a scalarized budget $\varepsilon_{\alpha\beta,i} \geq 0$. A *two-sided overlap metric certificate* consists of:

- (i) a pre-threshold bottleneck estimate

$$d_B(\mathcal{B}_{i,\alpha\beta}^\alpha, \mathcal{B}_{i,\alpha\beta}^\beta) \leq \varepsilon_{\alpha\beta,i};$$

- (ii) two-sided sharp clearance

$$\text{clear}_{\tau_{\alpha\beta}}(\mathcal{B}_{i,\alpha\beta}^\alpha) > \varepsilon_{\alpha\beta,i}, \quad \text{clear}_{\tau_{\alpha\beta}}(\mathcal{B}_{i,\alpha\beta}^\beta) > \varepsilon_{\alpha\beta,i};$$

- (iii) the ledger provenance whose bottleneck scalarization yields $\varepsilon_{\alpha\beta,i}$;
- (iv) the direct common-window and threshold-reconciliation records required above.

Proposition 6.20 (Sharp overlap transfer). Under a two-sided overlap metric certificate,

$$d_B(\text{Eval}_{i,\alpha\beta}^\alpha, \text{Eval}_{i,\alpha\beta}^\beta) \leq \varepsilon_{\alpha\beta,i}.$$

In the one-parameter constructible regime and under the infinite-bar convention of Appendix A, the same bound may be read using d_{int} .

Proof. Clause (i) of Definition 6.19 supplies the input distance bound. Clause (ii) says that neither pre-threshold barcode contains a finite bar whose length lies in the closed band

$$[\tau_{\alpha\beta} - \varepsilon_{\alpha\beta,i}, \tau_{\alpha\beta} + \varepsilon_{\alpha\beta,i}].$$

The sharp threshold-gap-safe stability theorem of Chapter 2 and Appendix A therefore applies and yields the post-threshold bottleneck bound. The bottleneck–interleaving isometry then gives the interleaving formulation in the declared constructible regime. $\square$

Definition 6.21 (Pairwise Overlap Gate). The *pairwise Overlap Gate* on $W_{\alpha\beta}$ passes if, for every monitored degree i :

- (i) exactly one overlap comparison mode is declared;
- (ii) in mode `exact`, a valid exact overlap certificate exists;
- (iii) in mode `metric_gap_certified`, a two-sided overlap metric certificate exists and the post-threshold discrepancy bound of Proposition 6.20 holds;
- (iv) mode `not_compared` is used only when the governing claim consumes no overlap inference;
- (v) all local representative or eligible-realization clauses used on the overlap have compatible provenance records;
- (vi) Type IV is either explicitly `not_invoked` by scope or, when invoked, is `certified_zero`;
- (vii) `undefined` is not treated as zero, and `obstructed` is a failure;
- (viii) every required crop, comparison, exact-proof, and threshold-reconciliation artifact is present.

Remark 6.22 (Two layers of the Overlap Gate). The Overlap Gate has two mathematically different layers:

- (i) the discrepancy verdict compares the *post-threshold* evaluations;
- (ii) threshold safety is certified on the *pre-threshold* profiles.

The second layer is not recoverable from the first. In particular, small post-threshold distance alone does not certify that threshold classification was stable. An exact overlap certificate may bypass metric transport for the declared exact comparison, but it does not supply positive-radius threshold robustness.

Remark 6.23 (Pairwise compatibility is not higher coherence). Passing every pairwise Overlap Gate does not by itself establish compatibility on triple or higher overlaps. Higher coherence must be declared through the nerve records below whenever the cover has nontrivial higher intersections.

6.5 Finite Čech depth and record-level coherence

Definition 6.24 (Čech depth). An overlap-admitting cover has *Čech depth at most K* if every intersection of $K + 2$ distinct windows is empty. Equivalently, its Čech nerve has simplicial dimension at most K . Thus all nonempty overlap records occur on at most $(K + 1)$ -fold intersections.

Definition 6.25 (Simplex compatibility record). Let $\sigma = \{\alpha_0, \dots, \alpha_p\}$ index a nonempty overlap W_σ . A *simplex compatibility record* consists of:

- (i) direct pre-threshold profiles of every participating local object on W_σ ;
- (ii) a common threshold and threshold-reconciliation record;
- (iii) an explicit comparison mode for every face comparison used by the record; any comparison required for atlas coherence must be `exact` or `metric_gap_certified`, not `not_compared`;
- (iv) for each `exact` face, an exact comparison proof and artifact; for each `metric_gap_certified` face, a real distance bound and sharp two-sided gap certificate;

- (v) a face-compatibility statement saying that restriction from σ to any face agrees with the already declared face record, under the same declared exact or metric equivalence notion;
- (vi) typed Type IV status and all required artifacts on W_σ .

Definition 6.26 (Čech-coherent certificate atlas). A family of local certificate records on $\mathcal{W}_{\text{ov}}$ is a *Čech-coherent certificate atlas* if every nonempty simplex of the nerve carries a simplex compatibility record and all face restrictions are mutually compatible.

Proposition 6.27 (Finite-depth verification bound). *If the cover has Čech depth at most K , then the coherence portion of the certificate atlas requires no records above simplicial degree K .*

Proof. By Definition 6.24, there are no nonempty intersections indexed by simplices of degree larger than K . Hence there are no higher overlap domains on which additional compatibility records could be evaluated. $\square$

Theorem 6.28 (Finite-cover record-level gluing). *Let $\mathcal{W}_{\text{ov}}$ be a finite overlap-admitting cover of finite Čech depth. Assume:*

- (i) *every window carries a valid local certificate record;*
- (ii) *every nonempty nerve simplex carries a simplex compatibility record, and every comparison required for coherence uses mode `exact` or `metric_gap_certified`;*
- (iii) *every required Type IV scope has one permitted status,*

`not_invoked` or `certified_zero`,

and is never `undefined`;

- (iv) *every metric-mode comparison satisfies its sharp two-sided threshold reserve, while every exact-mode comparison carries its exact proof artifact;*
- (v) *all face-compatibility statements and artifact references are audit-readable.*

Then the family is a Čech-coherent certificate atlas on the covered range.

Proof. The local records provide the vertices of the nerve. The simplex compatibility records provide all overlap data through the maximal nonempty nerve degree, and the face conditions ensure that these records agree under restriction. Finite depth and finiteness of the cover make the required record set finite. The threshold and Type IV clauses prevent undefined or unstable local comparisons from being reported as coherent. Therefore the family satisfies Definition 6.26. $\square$

Remark 6.29 (The theorem glues records, not mathematical objects). Theorem 6.28 does not assert that the filtered objects F_α , their representatives, or their derived realizations glue to a global object. It produces an auditable certificate atlas whose local verdicts and overlap records are coherent. Object-level descent requires an additional theorem in an explicitly declared category.

6.6 κ -admissible Restart

Let

$$\mathcal{W}_{\text{part}} = \{W_k = [u_k, u_{k+1})\}_{k \geq 0}$$

be an ordered right-open partition cover. For each k , let $\mathfrak{B}_k$ be the finite pre-threshold profile family used by the local gate and choose a finite working gap

$$0 < g_k \leq \text{gap}_{\tau_k}(\mathfrak{B}_k).$$

If the exact clearance is $+\infty$, the manifest chooses a finite audit cap g_k for Restart arithmetic. Let $\varepsilon_k < +\infty$ be the gate-scoped scalarized local budget and define the finite local reserve

$$r_k := g_k - \varepsilon_k \in \mathbb{R}.$$

A local gate has positive working reserve exactly when $r_k > 0$. The use of a finite certified lower bound avoids undefined extended-real expressions in the Restart recursion.

Definition 6.30 (Continuation artifact). A *continuation artifact* from W_k to W_{k+1} records:

- (i) the source and target local record identifiers;
- (ii) the source and target thresholds;
- (iii) any endpoint-state, initialization, representative, or comparison data used to pass from the source record to the target record;
- (iv) a finite scalarized continuation loss $\eta_k \geq 0$, not already counted in the local target budget;
- (v) a declared degradation factor $\kappa_k \in (0, 1]$;
- (vi) a proof object or independently checkable artifact for the reserve inequality below;
- (vii) explicit confirmation that missing morphisms, representatives, eligibility records, Type IV data, or manifest fields have not been encoded numerically inside η_k .

Definition 6.31 (κ_k -admissible continuation). A continuation artifact is κ_k -admissible if it certifies

$$r_{k+1} \geq \kappa_k r_k - \eta_k.$$

The number κ_k is part of the certified continuation data; its existence is not inferred from window adjacency or from threshold deletion alone.

Remark 6.32 (Restart is conditional, not automatic). There is no Core theorem asserting that every admissible-looking continuation has some universal $\kappa > 0$. Window restriction may change bar lengths, thresholds may change, and the next pre-threshold profile may be unrelated to the preceding one. A Restart claim is licensed only by an explicit κ_k -admissible continuation artifact. This is the v19 downgrade of the former unconditional Restart lemma.

Remark 6.33 (Restart is a reserve-transport claim). A κ -Restart assertion transports a finite positive threshold reserve and is therefore stronger than an isolated exact equality at one transition. An exact continuation may be recorded as exact comparison data, but it does not become a κ_k -admissible Restart step unless finite working gaps, local budgets, r_k, r_{k+1} , and the certified recurrence

$$r_{k+1} \geq \kappa_k r_k - \eta_k$$

are all supplied.

Definition 6.34 (Restart transition status). Each transition has exactly one of the statuses

`not_invoked`, `undefined`, `certified`, `failed`.

They mean:

- (i) `not_invoked`: the governing scope explicitly declares that no continuation or inherited reserve claim is being made;
- (ii) `undefined`: Restart is in scope, but a required continuation artifact, scalarized budget, finite working reserve, or κ_k -certificate is missing or ill-typed;
- (iii) `certified`: the artifact is complete and the κ_k -admissible inequality is verified;
- (iv) `failed`: the inputs are defined but the declared κ_k -inequality is false.

The transition status certifies the continuation law only. A successful Restart step additionally requires a passing target local record, in particular positive target reserve and no rejected or undefined non-reserve clause.

Theorem 6.35 (Finite-horizon Restart bound). *Assume the transitions $0 \rightarrow 1, \dots, n-1 \rightarrow n$ are respectively $\kappa_0, \dots, \kappa_{n-1}$ -admissible. Then*

$$r_n \geq \left(\prod_{j=0}^{n-1} \kappa_j \right) r_0 - \sum_{m=0}^{n-1} \left(\prod_{j=m+1}^{n-1} \kappa_j \right) \eta_m,$$

where an empty product is 1.

Proof. The case $n = 1$ is Definition 6.31. Assume the formula holds for n . Then

$$r_{n+1} \geq \kappa_n r_n - \eta_n.$$

Substituting the induction hypothesis and distributing κ_n gives

$$r_{n+1} \geq \left(\prod_{j=0}^n \kappa_j \right) r_0 - \sum_{m=0}^n \left(\prod_{j=m+1}^n \kappa_j \right) \eta_m.$$

Thus the formula follows by induction. □

Corollary 6.36 (Non-degrading continuation). *If $\kappa_k = 1$ for $0 \leq k < n$, then*

$$r_n \geq r_0 - \sum_{m=0}^{n-1} \eta_m.$$

Consequently, if

$$\sum_{m=0}^{n-1} \eta_m < r_0,$$

all reserves through level n are positive.

Definition 6.37 (Normalized Restart loss). Set

$$K_0 := 1, \quad K_n := \prod_{j=0}^{n-1} \kappa_j \quad (n \geq 1).$$

The normalized loss through level n is

$$L_n^{\text{norm}} := \sum_{m=0}^{n-1} \frac{\eta_m}{K_{m+1}}.$$

Corollary 6.38 (Normalized reserve criterion). *Under the hypotheses of Theorem 6.35,*

$$\frac{r_n}{K_n} \geq r_0 - L_n^{\text{norm}}.$$

Hence $L_n^{\text{norm}} < r_0$ implies $r_n > 0$.

Proof. Factor K_n from the lower bound in Theorem 6.35 and use

$$\prod_{j=m+1}^{n-1} \kappa_j = \frac{K_n}{K_{m+1}}.$$

□

Remark 6.39 (Ordinary summability is not enough). The condition $\sum_k \eta_k < \infty$ does not by itself guarantee positive reserves when the factors κ_k can contract. The correct inherited-reserve condition is normalized by the accumulated degradation:

$$\sum_{k=0}^{\infty} \frac{\eta_k}{K_{k+1}} < r_0.$$

If $K_n \rightarrow 0$, even a certified infinite run need not possess a positive uniform lower bound on r_n .

6.7 Countable Restart and summability

Definition 6.40 (κ -normalized summability). A countable certified Restart run is κ -normalized summable if

$$\sum_{k=0}^{\infty} \frac{\eta_k}{K_{k+1}} < \infty.$$

It is *reserve-safe* if, in addition,

$$\sum_{k=0}^{\infty} \frac{\eta_k}{K_{k+1}} < r_0.$$

Theorem 6.41 (Conditional countable Restart principle). *Let $\{W_k\}_{k \geq 0}$ be an ordered right-open partition cover. Assume:*

- (i) *the initial local record has $r_0 > 0$;*
- (ii) *every invoked transition has status *certified*;*
- (iii) *the run is reserve-safe in the sense of Definition 6.40;*

- (iv) every target window has a complete local record whose required non-reserve clauses pass, including representative, eligibility, and Type IV clauses when invoked;
- (v) no transition or target record is *undefined*.

Then every inherited reserve r_n is positive, and the sequence of local records forms an auditable sequential certificate record on the partition cover.

Proof. For every finite n , reserve-safety implies

$$L_n^{\text{norm}} \leq \sum_{k=0}^{\infty} \frac{\eta_k}{K_{k+1}} < r_0.$$

Corollary 6.38 therefore gives $r_n > 0$. The declared continuation artifacts and complete target records make every transition reconstructible. Thus the ordered family is an auditable sequential certificate record. $\square$

Corollary 6.42 (Pointwise reserve and uniform reserve are distinct). *Under the hypotheses of Theorem 6.41, set*

$$L_{\infty}^{\text{norm}} := \sum_{k=0}^{\infty} \frac{\eta_k}{K_{k+1}}, \quad c := r_0 - L_{\infty}^{\text{norm}} > 0.$$

Then every finite stage satisfies

$$r_n \geq K_n c > 0.$$

Consequently, pointwise positivity of all r_n follows from reserve-safety. A positive uniform lower bound follows if $\inf_n K_n > 0$; it does not follow merely from reserve-safety when $K_n \rightarrow 0$.

Proof. Corollary 6.38 gives

$$\frac{r_n}{K_n} \geq r_0 - L_n^{\text{norm}} \geq r_0 - L_{\infty}^{\text{norm}} = c.$$

Since every $K_n > 0$, the pointwise bound follows. Taking infima gives a uniform positive bound only when $\inf_n K_n > 0$. $\square$

Remark 6.43 (What the countable theorem does not say). Theorem 6.41 does not construct a single global filtered object or assert convergence of the local objects. It certifies the sequential inheritance of positive threshold reserve and the audibility of the record chain.

6.8 Countable overlap atlases

Definition 6.44 (Audit-locally finite cover). A countable overlap cover $\{W_n\}_{n \geq 1}$ is *audit-locally finite* if every bounded subinterval of the declared range meets only finitely many windows and only finitely many recorded nerve simplices.

Definition 6.45 (Summable simplex budget). Let N_{met} be the countable set of recorded nerve simplices whose comparison mode is `metric_gap_certified`, and let $\varepsilon_{\sigma} \in [0, +\infty)$ denote the corresponding scalarized metric budget. Define the unordered nonnegative sum by

$$\sum_{\sigma \in N_{\text{met}}} \varepsilon_{\sigma} := \sup_{S \subseteq N_{\text{met}}, S \text{ finite}} \sum_{\sigma \in S} \varepsilon_{\sigma}.$$

The atlas has a *summable simplex budget* when this quantity is finite. This definition is independent of enumeration. Exact-mode simplices require exact proof artifacts and are not assigned fictitious metric exposure

merely to enter this sum. The sum controls cumulative declared metric exposure but does not replace the separate sharp inequality

$$\varepsilon_\sigma < \text{gap}_{\tau_\sigma}(\mathfrak{B}_\sigma)$$

on every metric-mode simplex, nor any non-scalar coherence obligation.

Theorem 6.46 (Countable record-atlas principle). *Let $\{W_n\}_{n \geq 1}$ be a countable, audit-locally finite overlap cover of finite Čech depth. Assume:*

- (i) *every local window carries a complete certificate record;*
- (ii) *every nonempty nerve simplex is recorded and carries a simplex compatibility record with a declared `exact` or `metric_gap_certified` comparison mode for every consumed comparison;*
- (iii) *every metric-mode simplex has positive threshold reserve, and every exact-mode simplex has an exact proof artifact;*
- (iv) *the metric-mode simplex budgets are summable;*
- (v) *every Type IV scope is either correctly `not_invoked` or `certified_zero`;*
- (vi) *all face restrictions and artifact references are audit-readable.*

Then the family forms an audit-locally finite Čech-coherent certificate atlas on the covered range, and its total declared scalarized metric-mode simplex exposure is finite.

Proof. Audit-local finiteness ensures that every bounded audit query touches only finitely many local and overlap records. Finite Čech depth bounds the order of intersections. The simplex compatibility records and face conditions provide coherence on every nonempty nerve simplex. Metric-mode positive threshold reserves license their thresholded transports, while exact-mode records are licensed by their exact proof artifacts. Definition 6.45 records finite total scalarized exposure across the metric-mode portion of the countable atlas. Therefore the resulting atlas is locally reconstructible and globally indexed as a coherent certificate record system with finite declared metric exposure. $\square$

Remark 6.47 (Summability is not coherence). Simplex-budget summability is an audit-exposure condition. It neither creates missing face maps nor proves higher compatibility. Conversely, a coherent atlas may fail the chosen global exposure policy if its scalarized simplex budgets are not summable. The two obligations are logically independent and both are required by Theorem 6.46.

Remark 6.48 (No global pass from summability alone). A summable atlas is not automatically a single global B-Gate⁺ pass. The conclusion is a coherent atlas of local and overlap certificates. A single global pass requires an independently declared global object, global window, and global gate record.

Remark 6.49 (Canonical Type IV notation in Chapter 6). Whenever Chapter 6 cites aggregate Type IV diagnostics, the canonical pair is

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}).$$

The legacy symbols $u_{i,W,\tau}$ and u_{Collapse} have no normative use. The coordinate u in a window $[u, v]$ is unrelated and is retained as a domain endpoint.

6.9 Integration with B-Gate⁺ and Type IV

Definition 6.50 (Local certificate status). A local window record has status

pass, reject, undefined

according to the B-Gate⁺, eligibility, Type IV, and manifest disciplines of Chapters 1–5. Undefined records cannot participate as successful vertices of an overlap atlas or as successful targets of Restart.

Proposition 6.51 (Compatibility does not repair a local failure). *Passing an Overlap Gate does not convert a locally rejected or undefined record into a local pass. Likewise, a certified Restart transition does not discharge a missing local B-Gate⁺, representative, or Type IV requirement on the target window.*

Proof. Overlap and Restart records certify compatibility or continuation between local records. They do not alter the internal gate clauses of either endpoint record. Hence local validity remains a separate necessary condition. □

Proposition 6.52 (Typed Type IV participation). *Whenever Type IV is in scope on a local window, overlap, or continuation target:*

- (i) *certified_zero permits the record to proceed;*
- (ii) *obstructed forces rejection;*
- (iii) *undefined prevents certification and is routed as Type III operational undefinedness; it becomes Type V only if the missing state is concealed, serialized as zero or pass, or makes the manifest inconsistent;*
- (iv) *not_invoked is admissible only when the governing policy explicitly places Type IV outside the scope of that record.*

Proof. This is the typed Type IV discipline of Chapter 5 applied to the local, overlap, and restart records defined in this chapter. □

6.10 Failure routing

Definition 6.53 (Chapter 6 failure routing). Failures in this chapter are routed as follows; the types are not mutually exclusive.

- (i) an unstable threshold comparison or failed sharp gap condition is Type III;
- (ii) a false crop-commutation substitution, failed overlap comparison, or failed κ_k -inequality is Type III;
- (iii) a nonzero terminal-generic tower defect is Type IV;
- (iv) a missing required mathematical or operational artifact yields an undefined clause and is Type III at the operational level. It is additionally Type V only when the missing-artifact marker is omitted or concealed, an unsupported pass or zero is reported, or the manifest is incomplete or inconsistent;
- (v) a local topological obstruction remains Type I, and an eligible categorical obstruction remains Type II.

Remark 6.54 (Why missing κ is not evidence for $\kappa = 1$). Absence of a degradation certificate does not justify setting $\kappa_k = 1$, $\eta_k = 0$, or declaring the transition harmless. It yields status undefined and therefore no certified Restart.

6.11 Minimal manifest consequences

Proposition 6.55 (Window/overlap/Restart manifest extension). *An audit-readable manifest using this chapter extends the Chapter 1 base manifest and the Chapter 3 profile/representative extension by recording, at minimum:*

- (i) *the cover type: `partition` or `overlap`;*
- (ii) *every window and every nonempty recorded overlap;*
- (iii) *the direct common-window evaluation policy;*
- (iv) *the local and overlap thresholds, including threshold-reconciliation records;*
- (v) *the pre-threshold profiles or their immutable artifact identifiers;*
- (vi) *the clearance and gap values used by each comparison;*
- (vii) *the ledger quantale and bottleneck scalarization;*
- (viii) *the comparison mode of every consumed local, overlap, and simplex comparison, together with every exact proof artifact or scalarized metric budget as applicable;*
- (ix) *every scalarized local, overlap, crop, and continuation budget actually consumed by a metric-mode inference;*
- (x) *every pairwise and higher simplex compatibility record;*
- (xi) *the Čech depth and audit-local-finiteness declaration, when applicable;*
- (xii) *every continuation artifact, κ_k , η_k , reserve r_k , and Restart status;*
- (xiii) *all representative and eligible-realization records used by local clauses;*
- (xiv) *every typed Type IV scope status and required comparison artifact;*
- (xv) *all local, overlap, restart, and atlas verdicts;*
- (xvi) *the failure-routing record for every rejected or undefined clause.*

Proof. Each item is an input to, or a reconstructibility condition for, one of the definitions and theorems of this chapter. Omitting any invoked item prevents an independent verifier from reproducing the corresponding compatibility, restart, or gluing verdict. $\square$

Remark 6.56 (Canonical manifest location). Proposition 6.55 is an extension schema, not a competing canonical manifest. Chapter 7 integrates the Chapter 1 base fields, the Chapter 3 evaluation fields, the present window/overlap/Restart fields, and the remaining audit fields into the canonical v19 audit manifest with statuses complete, incomplete, and inconsistent.

6.12 Window, overlap, and Restart package

Theorem 6.57 (Conditional window/overlap/Restart closure package). *Within the v19 repaired Core, the following package is available.*

- (i) *Overlap profiles are evaluated directly on the common window before threshold deletion.*
- (ii) *Metric ledger data are compared with threshold clearance only after a declared bottleneck scalarization; the sharpness of the threshold band does not assert tightness of that scalarization.*
- (iii) *Overlap comparison is typed: exact mode consumes an exact profile proof, while metric-gap-certified mode uses a sharp two-sided certificate to transfer a pre-threshold ε -bound to the post-threshold evaluations.*
- (iv) *Pairwise Overlap Gates do not replace higher Čech coherence.*
- (v) *Finite-depth overlap data produce a coherent certificate atlas only at the record level.*
- (vi) *Restart is conditional on a declared κ_k -admissible continuation artifact.*
- (vii) *Finite-horizon reserves obey the product-weighted inequality of Theorem 6.35.*
- (viii) *Countable Restart requires κ -normalized reserve-safety; ordinary summability alone is insufficient, and pointwise positive reserves do not automatically have a positive uniform infimum.*
- (ix) *Countable overlap atlases additionally require audit-local finiteness, finite Čech depth, simplex coherence, exact proof artifacts on exact-mode simplices, and positive reserves plus summable exposure on metric-mode simplices.*
- (x) *Undefined artifacts or statuses are never promoted to successful compatibility, Restart, or gluing verdicts.*

Proof. Items (i)–(iii) follow from Definitions 6.7, 6.9, 6.17, 6.18, and Proposition 6.20. Items (iv)–(v) follow from Remark 6.23 and Theorem 6.28. Items (vi)–(viii) follow from Definitions 6.31, Theorem 6.35, and Theorem 6.41. Item (ix) is Theorem 6.46. Item (x) follows from the typed status disciplines of Definitions 6.34 and 6.50, together with Proposition 6.52. $\square$

Remark 6.58 (Closure package, not an additional mechanism). Theorem 6.57 is a closure package collecting the proved transfer bounds and the explicitly conditional record theorems of this chapter. It does not turn operational definitions into a new unconditional local-to-global theorem, and it does not supply any missing non-scalar artifact.

6.13 Explicit non-claims

The following are not asserted in this chapter.

- (i) No unconditional Restart lemma is asserted.
- (ii) No universal or automatically existing $\kappa > 0$ is asserted.
- (iii) No missing continuation artifact is interpreted as $\kappa = 1$ or zero loss.
- (iv) No ordinary summability condition $\sum_k \eta_k < \infty$ is claimed to preserve positive reserve under arbitrary contracting κ_k .

- (v) No pairwise Overlap Gate is claimed to imply higher Čech coherence.
- (vi) No exact overlap comparison is automatically a positive-radius robustness certificate, and no exact proof is forced into a fictitious metric budget merely for ledger summation.
- (vii) No comparison of differently thresholded outputs is made without threshold reconciliation and direct common-threshold re-evaluation.
- (viii) No crop-after-collapse comparison replaces direct common-window evaluation without a commutation certificate.
- (ix) No quantale-valued ledger is compared directly with a real threshold clearance without a declared scalarization.
- (x) No claim is made that the bottleneck scalarization is numerically tight merely because the two-sided threshold band is sharp.
- (xi) No metric reserve, large gap, or summability estimate repairs a missing representative, morphism, compatibility record, Type IV certificate, claim status, or manifest field.
- (xii) No pointwise conclusion $r_n > 0$ for every finite n is promoted to a uniform bound $\inf_n r_n > 0$ without an additional lower bound on K_n .
- (xiii) No coherent certificate atlas is automatically a global filtered object, global persistence module, or global B-Gate⁺ pass.
- (xiv) No undefined Type IV, Restart, representative, or overlap status is treated as zero or pass.
- (xv) No Search-side map, score, or AI proposal supplies a missing overlap, continuation, or gluing certificate.

6.14 Provenance and status

The sharp overlap-transfer step is an application of the threshold-gap-safe stability theorem of Chapter 2 and Appendix A. The representative and eligible-realization clauses are inherited from Chapters 3–4. The typed Type IV clauses are inherited from Chapter 5. The new contribution of this chapter is the typed record architecture joining these ingredients, together with the explicit downgrade of Restart to a κ -admissible conditional theory and the resulting product-weighted reserve bounds.

Status labels for this chapter. *Core mathematical propositions/theorems:* Proposition 6.20, Proposition 6.27, Theorem 6.35, Corollary 6.36, Corollary 6.38, and Corollary 6.42.

Conditional record theorems: Theorem 6.28, Theorem 6.41, Theorem 6.46, and Theorem 6.57.

Core record definitions: Definitions 6.3, 6.4, 6.5, 6.7, 6.9, 6.10, 6.11, 6.14, 6.16, 6.17, 6.18, 6.19, 6.21, 6.24, 6.25, 6.26, 6.30, 6.31, 6.34, 6.37, 6.40, 6.44, 6.45, 6.50, and 6.53.

Manifest and auditability: Proposition 6.55.

Explicit exclusions: Section 6.13.

7 Chapter 7: Typed Audit Semantics and Core Sovereignty

Chapter 7 contract. This chapter is the canonical audit layer of the v19.0.0 Core. It integrates the foundational manifest fields of Chapter 1, the evaluation and representative extensions of Chapter 3, the eligibility extension of Chapter 4, the failure and Type IV extensions of Chapter 5, and the window/overlap/Restart extensions of Chapter 6.

A Core verdict is a typed conjunction of independently evidenced clauses. Every consumed comparison is classified as

`exact,` `metric_gap_certified,` `not_compared.`

Exact proof is not confused with positive-radius robustness. Numerical slack may bound a declared metric discrepancy, but it may not create a missing representative, morphism, eligibility certificate, Type IV artifact, Restart certificate, claim status, or manifest field. The statuses `reject`, `undefined`, and `justified` `not_invoked` remain distinct.

7.1 Scope, dependency order, and relocation

This chapter closes the audit semantics of the v19 Core after the mathematical and record-theoretic constructions of Chapters 1–6. It consumes, but does not replace, the following previously established layers:

- (i) the claim discipline, comparison-mode distinction, and B-Gate⁺ contract of Chapter 1;
- (ii) the standard bottleneck metric, hard threshold deletion, and sharp two-sided stability theorem of Chapter 2 and Appendix A;
- (iii) the repaired-evaluation, single-profile threshold-readiness, two-sided comparison-certificate, and representative disciplines of Chapter 3;
- (iv) the eligible realization and conditional one-way bridge of Chapter 4;
- (v) the typed failure taxonomy, source-semantics discipline, and terminal-generic Type IV diagnostics of Chapter 5;
- (vi) the typed ledger scalarization, comparison-mode Overlap Gate, Čech atlas, and κ -admissible Restart discipline of Chapter 6.

The purpose of the present chapter is to determine when those inputs yield an auditable Core verdict. It therefore specifies:

- (i) which deviations may enter a numerical ledger, which comparisons are exact or metric, and which proof obligations may not be scalarized;
- (ii) the typed meanings of `pass`, `reject`, `undefined`, and `not_invoked`;
- (iii) the aggregation rule for atomic clause statuses in finite and countable scopes;
- (iv) the audit reading of B-Gate⁺, Type IV, Overlap, Restart, and atlas records;
- (v) manifest completeness, inconsistency, artifact closure, replayability, and repair traceability;
- (vi) Core sovereignty and proposer–verifier separation.

Remark 7.1 (Relocation of the former Chapter 7). The former chapter entitled “Core Theorem Package and Explicit Non-Claims” is not discarded. In the v19 dependency order it follows the present audit chapter and is therefore relocated to Chapter 8. The present chapter precedes that closure package because claim status, auditability, and Core sovereignty are inputs to the final statement of what the Core does and does not assert.

Remark 7.2 (Normative rather than external-domain content). Most statements in this chapter are definitions, typing contracts, or consequences of those contracts. They govern admissible proof objects and verdicts; they are not new theorems about an external arithmetic, geometric, PDE, computational, or cryptographic domain.

7.2 Audit identity and immutable run scope

Definition 7.3 (Audit run identity). An *audit run identity* is a tuple

$$\mathfrak{R} = (\text{RunID}, \text{theory_version}, \text{schema_version}, \text{claim_register_version}, \text{artifact_root}).$$

It identifies the exact theory release, manifest schema, claim-register snapshot, and content-addressed artifact root under which all verdicts in the run are interpreted.

Definition 7.4 (Immutable audit scope). The *immutable audit scope* of a run records, at minimum:

- (i) the object or object family under audit;
- (ii) the monitored degrees and, for Type IV totals, the finite set I_{mon} ;
- (iii) the windows, thresholds, terminal modes, and evaluation order;
- (iv) the complete registry of invoked clauses and explicitly non-invoked clauses;
- (v) the comparison mode of every consumed comparison;
- (vi) the claim identifiers, invocation roles, theorem or policy locations, and registered strengths;
- (vii) the immutable identifiers and content hashes of all source, profile, representative, comparison, diagnostic, and verdict artifacts.

A change to any of these data defines a new audit run rather than a silent mutation of the existing run.

Remark 7.5 (Why immutable identity matters). A previously rejected or undefined run may be repaired, but the repaired object receives a new RunID and ManifestID, optionally linked to the former run by an explicit parent or repair relation. Later evidence does not rewrite the historical meaning of the earlier verdict.

7.3 The typed δ -ledger

Definition 7.6 (Declared ledger algebra). A run using numerical error budgets declares a commutative unital ordered monoid

$$(\mathbb{V}, \oplus, 0_{\mathbb{V}}, \preceq)$$

whose operation is monotone in each variable. If the implementation also uses arbitrary joins and $\oplus$ distributes over those joins, the ledger algebra may be declared a commutative quantale. The word “quantale” is not used merely for a four-tuple lacking completeness and distributivity. Every component carries a declared semantic type, provenance, gate scope, and aggregation rule.

Definition 7.7 (Metric-bearing ledger components). The *metric-bearing ledger* is the declared element

$$\begin{aligned} \mathfrak{d}_{\text{met}} = & \delta^{\text{alg}} \oplus \delta^{\text{disc}} \oplus \delta^{\text{meas}} \oplus \delta^{\text{rep,met}} \oplus \delta^{\text{real,met}} \oplus \delta^{\text{cmp}} \\ & \oplus \delta^{\text{crop}} \oplus \delta^{\text{cont}} \oplus \delta^{\text{ov}} \oplus \delta^{\text{rst}}, \end{aligned}$$

with only the components actually consumed by the audited metric comparison included. The terms represent implementation, discretization, measurement, comparison, crop, continuation, overlap, and Restart discrepancies, together with metric errors caused by representative choice or realization. A representative-related or realization-related term is included only when an actual comparison theorem supplies a quantitative bound. Existence, typing, and compatibility remain non-numerical obligations.

Definition 7.8 (Bottleneck scalarization). A bottleneck scalarization is a monotone subadditive map

$$\text{val}_B : V \longrightarrow [0, +\infty]$$

satisfying

$$\text{val}_B(0_V) = 0, \quad \text{val}_B(q \oplus q') \leq \text{val}_B(q) + \text{val}_B(q').$$

The scalarized metric budget is

$$\varepsilon_B(\mathfrak{d}_{\text{met}}) := \text{val}_B(\mathfrak{d}_{\text{met}}).$$

A metric gate requires this value to be finite and to be proved to dominate the actual bottleneck discrepancy consumed by that gate. A scalarization value without its metric upper-bound theorem is only a recorded number, not a certificate.

Remark 7.9 (Sharp threshold constant versus conservative scalarization). The adjective *sharp* refers to the optimal two-sided exclusion band $[\tau - \varepsilon, \tau + \varepsilon]$ for a valid bottleneck upper bound ε . It does not assert that a chosen scalarization is numerically tight. A conservative scalarization preserves soundness but may reduce the set of passing runs.

Definition 7.10 (Non-scalarizable proof obligations). The following are *non-scalarizable proof obligations*:

- (i) existence, typing, and equivalence strength of a required representative;
- (ii) eligibility, amplitude, edge-realization, and realization-equivalence certificates;
- (iii) existence and typing of an actual overlap, continuation, or tower comparison morphism;
- (iv) source semantics, terminal tameness, defect profiles, and interior-defect status;
- (v) threshold reconciliation, exact comparison, simplex coherence, continuation artifacts, and κ -admissibility certificates;
- (vi) local pass, target pass, completeness of a countable registry, and higher coherence;
- (vii) claim status, invocation role, repair records, and version boundaries;
- (viii) manifest completeness, consistency, artifact closure, and identity obligations.

No increase in a numerical budget discharges an item in this list.

Proposition 7.11 (No scalar repair of missing evidence). *Let a required non-scalarizable proof obligation be missing or ill-typed. Then no value of $\mathfrak{d}_{\text{met}}$, no scalarized tolerance, and no positive threshold clearance can convert the corresponding clause into a certified pass.*

Proof. A metric ledger bounds a declared discrepancy. It does not construct the missing mathematical or governance object. Hence the obligation remains undefined, or a contradictory governance declaration remains rejected, independently of numerical slack. $\square$

Definition 7.12 (Gate-scoped threshold reserve). Let $\mathfrak{B} = \{B_j\}_{j \in J}$ be the finite family of pre-threshold profiles consumed by one comparison in `metric_gap_certified` mode. For finite $\varepsilon_B = \text{val}_B(\mathfrak{d}_{\text{met}})$, define

$$\text{gap}_\tau(\mathfrak{B}) := \min_{j \in J} \text{clear}_\tau(B_j)$$

and

$$\text{res}_\tau(\mathfrak{B}; \mathfrak{d}_{\text{met}}) := \text{gap}_\tau(\mathfrak{B}) - \varepsilon_B.$$

The metric clause has positive reserve precisely when

$$\varepsilon_B < \text{gap}_\tau(\mathfrak{B}).$$

If the scalarized budget is infinite, the metric clause does not pass. An exact comparison uses its exact proof artifact and receives no positive-radius robustness certificate merely from exactness.

Remark 7.13 (Meaning of $\Sigma\delta < \text{gap}_\tau$). The legacy notation $\Sigma\delta < \text{gap}_\tau$ denotes only the typed real inequality of Definition 7.12. An unscalarized ledger element is never compared directly with real clearance. A budget belongs to its declared gate and profile family and is not silently spent again elsewhere.

Definition 7.14 (Ledger completeness). A ledger is *complete and non-duplicative* for a metric clause if:

- (i) every metric perturbation affecting the clause is represented exactly once or by a documented joint bound;
- (ii) no perturbation is omitted merely because it is difficult to estimate;
- (iii) no component is counted twice under different names;
- (iv) every scalarized value is traceable to a proof or immutable artifact;
- (v) the aggregation law is the one declared by the run;
- (vi) the resulting real value is proved to bound the actual consumed discrepancy.

This definition is not invoked for a purely exact comparison unless a separate metric robustness claim is also made.

7.4 Claim-status and artifact preflight

Definition 7.15 (Claim-use status). Every registered item consumed by a run has exactly one status:

`proved`, `conditional`, `operational`, `spec`, `deprecated`, `rejected`, `unknown`.

The statuses distinguish theorem-level evidence, conditional evidence, procedural rules, specifications, historical claims, unavailable claims, and unclassified items.

Definition 7.16 (Claim invocation role). Every registered item consumed by a run is also assigned exactly one role:

`theorem_evidence`, `operational_rule`, `specification_target`, `provenance_only`.

The admissible combinations are:

- (i) `theorem_evidence`: status `proved`, or status `conditional` with every registered condition discharged;

- (ii) `operational_rule`: status `operational`, used only to govern a typed procedure;
- (iii) `specification_target`: status `spec`, used only as a schema, target, or acceptance criterion and never as proof of attainment;
- (iv) `provenance_only`: a deprecated, rejected, unknown, or superseded item cited only for history and contributing no passing theorem clause.

The role is fixed before verdict computation and may not be selected after seeing the desired result.

Definition 7.17 (Audit preflight). The *audit preflight* passes exactly when:

- (i) the run identity and immutable scope are complete;
- (ii) every consumed registered item has both a status and an invocation role;
- (iii) every status–role combination satisfies Definition 7.16;
- (iv) no `operational`, `spec`, `deprecated`, `rejected`, or `unknown` item is promoted to theorem evidence;
- (v) every relevant v18-to-v19 repair has an immutable repair record;
- (vi) every required artifact is present, typed, versioned, content-addressed, and linked to the run;
- (vii) no identifier, hash, version, role, or status makes incompatible assertions about the same input.

Definition 7.18 (Typed preflight verdict). The preflight has exactly one status:

`pass`, `reject`, `undefined`.

It is `pass` when Definition 7.17 is satisfied. It is `reject` when a well-typed record evidences forbidden theorem use, incompatible version or hash claims, an unauthorized scope exclusion, or another definite governance contradiction. It is `undefined` when a required identity, role, status, repair record, or artifact is missing, unresolved, or ill-typed but no evidenced contradiction yet forces rejection.

Remark 7.19 (Claim registration is not proof). A Claim Register records permitted strength and use. It does not prove an item by listing it. A proved entry points to an actual theorem and proof location; a `conditional` entry records all conditions; `operational` and `specification` entries retain their non-theorem roles.

Proposition 7.20 (Preflight failure blocks pass). *If the preflight status is not `pass`, the run cannot receive a Core pass verdict.*

Proof. A nonpassing preflight means that admissible theorem strength, scope, version, or artifact identity is either unresolved or contradicted. In neither case can every required Core clause be certified. $\square$

7.5 Typed atomic clause statuses

Definition 7.21 (Atomic audit status). Every atomic clause in the declared audit scope has exactly one status:

`pass`, `reject`, `undefined`, `not_invoked`.

They mean:

- (i) `pass`: the clause is well-typed, fully evidenced, and true;

- (ii) **reject**: the clause is well-typed and evidenced but false or obstructed, or a governed consistency clause is contradicted by incompatible declared evidence;
- (iii) **undefined**: the clause is required but incomplete, ill-typed, unresolved, or not computable from the declared evidence, with no evidenced contradiction yet forcing rejection;
- (iv) **not_invoked**: the governing immutable scope explicitly excludes the clause.

Remark 7.22 (Not invoked requires an explicit scope decision). A missing clause is not automatically **not_invoked**. The scope policy must positively declare that the clause is irrelevant to the claim; otherwise it is **undefined**.

Definition 7.23 (Typed specializations). The specialized statuses of earlier chapters embed into the atomic status algebra as follows.

- (i) Type IV **certified_zero**, **obstructed**, and **undefined** map to **pass**, **reject**, and **undefined**; Type IV **not_invoked** remains so only under an explicit no-tower scope decision.
- (ii) Chapter 3 representative status **certified** maps to **pass**; missing or ill-typed representative data map to **undefined**; explicit exclusion maps to **not_invoked**.
- (iii) Chapter 4 Ext statuses map identically to the atomic statuses with the same names.
- (iv) Chapter 6 Restart-transition **failed** maps to **reject**, and **undefined** maps to **undefined**. Transition **certified** passes only the recurrence subclause and not the complete Restart step.
- (v) Manifest **complete**, **incomplete**, and **inconsistent** map to **pass**, **undefined**, and **reject**.

Definition 7.24 (Aggregate audit verdict). Let C_{all} be the finite or countable audit-indexed registry of potential clauses in one immutable scope, and let $C_{\text{req}} \subseteq C_{\text{all}}$ be the invoked clauses. Every clause outside C_{req} carries a justified **not_invoked** status. For a countable scope, the manifest certifies that the registry and its dependency closure are complete and audit-readable; a finite prefix cannot support a **pass**. The aggregate headline verdict is:

- (i) **reject** if at least one invoked clause is **reject**;
- (ii) **undefined** if no invoked clause is **reject** and at least one is **undefined**, or if a purported **not_invoked** clause lacks valid scope justification;
- (iii) **pass** if every invoked clause is **pass** and every excluded clause is justified.

The record retains all rejected, undefined, and non-invoked clause identifiers in addition to the headline.

Proposition 7.25 (Determinacy of complete records). *If every required clause is well-typed and evidenced and the scope registry is complete, then the aggregate verdict is either **pass** or **reject**; it is not **undefined**.*

Proof. Every required clause then has a truth-valued status **pass** or **reject**; apply Definition 7.24. $\square$

Proposition 7.26 (Undefined is neither success nor counterexample). *An **undefined** verdict is not a Core **pass** and does not, by itself, constitute a mathematical counterexample to the intended claim.*

Proof. Undefinedness records missing, ill-typed, unresolved, or unavailable evidence. It establishes neither the clause nor the negation of a well-typed mathematical statement. $\square$

Definition 7.27 (Comparison-mode audit record). Every consumed profile comparison declares exactly one mode:

`exact,      metric_gap_certified,      not_compared.`

The modes mean:

- (i) **exact**: an actual equality or persistence-profile isomorphism, together with every exact crop, restriction, source, target, and proof artifact used by the inference, is present;
- (ii) **metric_gap_certified**: an evidenced pre-threshold bottleneck upper bound, a complete scalarized metric ledger, and a two-sided threshold-gap certificate at the same budget are present;
- (iii) **not_compared**: the governing claim consumes no comparison inference between the profiles.

Exact mode proves only the declared exact relation and does not itself give positive-radius robustness. The mode `not_compared` cannot discharge a comparison, Overlap, Restart, or coherence obligation that is actually invoked.

7.6 Audit reading of B-Gate⁺

Definition 7.28 (B-Gate⁺ audit clause set). For a declared object, window, threshold, and claim scope, the B-Gate⁺ clause set contains the following clauses whenever they are in scope:

- (i) **preflight**: the typed preflight verdict is pass;
- (ii) **evaluation typing**:

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F));$$
- (iii) **topological clause**: $\text{Eval}_{1,W,\tau}(F) = 0$;
- (iv) **categorical clause**: when invoked, a complete eligible realization record exists and

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0;$$
- (v) **comparison-mode clause**: every consumed repaired comparison declares a mode from Definition 7.27; exact mode carries an exact proof, metric mode carries a two-sided threshold-gap comparison certificate, and `not_compared` is used only outside comparison scope;
- (vi) **metric budget clause**: for every metric comparison, the ledger is complete, scalarized, proved to dominate the actual discrepancy, and has strict positive reserve; no reserve is fabricated for exact mode;
- (vii) **Type IV clause**: the status is justified `not_invoked` or `certified_zero`;
- (viii) **representative and realization clause**: every representative-level or derived-level use has the required actual isomorphisms, equivalence strength, and realization artifacts;
- (ix) **overlap and atlas clause**: if gluing is invoked, every consumed pairwise and higher simplex comparison uses exact or metric mode and passes;
- (x) **Restart clause**: if sequential inheritance is invoked, each transition recurrence, target reserve, target local record, and non-reserve obligation passes;
- (xi) **manifest clause**: the canonical manifest is complete, consistent, and artifact-closed;

(xii) **failure-routing clause:** every non-pass atomic clause has a typed Type I–V route.

Remark 7.29 (One-way bridge discharge only). Under a complete eligible realization record, Theorem 4.18 may discharge the categorical clause from topological vanishing. It does not discharge metric, Type IV, overlap, Restart, manifest, or claim-status clauses and does not license the converse implication.

Definition 7.30 (Core audit pass). A run receives a *Core audit pass* exactly when the aggregate verdict of its required B-Gate⁺ clause set is pass.

Definition 7.31 (Core audit reject). A run receives a *Core audit reject* exactly when the aggregate verdict is reject. The record identifies every evidenced failed clause and its failure route.

Definition 7.32 (Core audit undefined). A run receives a *Core audit undefined* verdict exactly when no evidenced clause forces rejection but at least one required clause remains undefined. The record identifies the missing, ill-typed, unresolved, or unavailable evidence.

Proposition 7.33 (Soundness of the pass reading). *A Core audit pass implies, for every invoked clause, the exact structural, metric, artifact, scope, and manifest conditions listed in Definition 7.28.*

Proof. Every required atomic clause has status pass; unpack the definitions. □

Proposition 7.34 (No pass by cancellation). *A passing clause cannot cancel a rejected or undefined clause. In particular, large clearance, a zero Ext group, or a zero Type IV diagnostic cannot repair a missing representative, failed exact comparison, failed metric reserve, undefined Restart transition, or forbidden claim use.*

Proof. The aggregate verdict is a typed conjunction, not a signed numerical sum. □

7.7 Local, overlap, Restart, and atlas audit composition

Definition 7.35 (Local certificate verdict). A local window certificate inherits pass, reject, or undefined from its local B-Gate⁺ audit. Only a local pass may serve as a successful atlas vertex or Restart endpoint.

Definition 7.36 (Compatibility verdict). An overlap or higher-simplex compatibility record receives:

- (i) pass in exact mode when the direct common-window profiles, threshold reconciliation, exact equality or profile-isomorphism artifact, face provenance, and required Type IV record all pass;
- (ii) pass in metric_gap_certified mode when the direct profiles, reconciliation, evidenced distance bound, sharp two-sided gap certificate, scalarized budget, face provenance, and required Type IV record all pass;
- (iii) reject when a well-typed exact relation, metric bound, reserve, face condition, or other compatibility requirement is evidenced false;
- (iv) undefined when a required mode, profile, proof, scalarization, face record, or artifact is missing or ill-typed;
- (v) not_invoked only under an explicit scope exclusion of compatibility inference.

Exact compatibility does not inherit positive-radius robustness without a separate metric certificate.

Definition 7.37 (Restart verdict). A complete Restart step receives:

- (i) `pass` exactly when the Chapter 6 transition status is `certified`, the target reserve is positive, the target local record passes, and every non-reserve artifact is present;
- (ii) `reject` when the declared κ_k -inequality fails, the defined target reserve is nonpositive, or the target local record rejects;
- (iii) `undefined` when a continuation artifact, finite working reserve, κ_k , η_k , scalarized budget, endpoint certificate, target record, or other required input is missing or ill-typed;
- (iv) `not_invoked` only when sequential inheritance is explicitly outside scope.

Transition status `certified` is necessary but not sufficient for a complete Restart pass.

Proposition 7.38 (Atlas composition rule). *A finite or countable certificate atlas is auditable as coherent only if:*

- (i) *every local vertex record passes;*
- (ii) *every required nonempty nerve simplex is recorded and every consumed face comparison uses `exact` or `metric_gap_certified` mode and passes;*
- (iii) *the declared Čech depth and audit-local-finiteness conditions pass;*
- (iv) *simplex-budget summability is checked on exactly the metric-mode simplices when that global exposure policy is invoked;*
- (v) *every invoked Type IV and Restart record has a passing typed status;*
- (vi) *the manifest is complete, consistent, and artifact-closed.*

A rejected item rejects the atlas; if none rejects but one is undefined, the atlas is undefined.

Proof. Apply the aggregate verdict rule to the Chapter 6 atlas clause registry. □

Remark 7.39 (Atlas pass is not a global-object pass). A coherent atlas certifies compatibility of local records. It does not create a single global filtered object, persistence module, or global B-Gate⁺ pass. Object-level descent and global audit require separate theorems and records.

Remark 7.40 (Pointwise reserve is not uniform reserve). Countable Restart may give $r_n > 0$ at every finite stage while $r_n \rightarrow 0$ when $K_n = \prod_{j < n} \kappa_j \rightarrow 0$. A claim $\inf_n r_n > 0$ requires its own lower-bound hypothesis and artifact.

7.8 Manifest completeness, consistency, and replayability

Remark 7.41 (Canonical manifest integration). The earlier manifest definitions form an extension chain rather than competing schemas: Chapter 1 supplies the base; Chapters 3–6 add evaluation, representative, eligibility, Type IV, overlap, Restart, and atlas fields. Definition 7.43 is their canonical semantic superset for v19.0.0. A chapter-local fragment may add fields, but it may not change shared meanings or final status semantics.

Definition 7.42 (Manifest status). A manifest has exactly one status:

`complete,      incomplete,      inconsistent.`

Here `complete` means every required field and artifact is present, typed, dependency-closed, and mutually consistent; `incomplete` means at least one required field or artifact is absent or unresolved; and `inconsistent` means declared fields, versions, hashes, statuses, roles, or artifacts make incompatible assertions about the same run.

Definition 7.43 (Canonical minimal v19 audit manifest). The canonical minimal v19 audit manifest contains, at minimum:

- (i) ManifestID, RunID, theory, schema, and claim-register versions, and repair ancestry;
- (ii) source-object identifiers, content hashes, artifact root, and dependency graph;
- (iii) monitored degrees, finite I_{mon} , windows, terminal modes, thresholds, and evaluation order;
- (iv) the hard-deletion and infinite-bar conventions;
- (v) all pre-threshold and repaired-profile identifiers;
- (vi) single-profile threshold-readiness records and the comparison mode of every consumed comparison;
- (vii) in exact mode, actual equality or profile-isomorphism proofs and exact crop/restriction artifacts;
- (viii) in metric mode, the finite protected profile family, clear_τ , gap_τ , ledger algebra, scalarization, discrepancy upper-bound proof, and reserve;
- (ix) representative, realization, eligibility, amplitude, edge, and equivalence-strength records when used;
- (x) Type IV scope, tower system, required source semantics, source provenance authority, comparison morphism, terminal mode, and terminal-tameness data;
- (xi) all degree-wise and total μ, ν diagnostics at fixed $(W, \tau, I_{\text{mon}})$, together with the separate interior-defect status

`not_checked,      certified_absent,      certified_present;`
- (xii) every local, overlap, simplex, threshold-reconciliation, atlas, and face-coherence record when used;
- (xiii) every continuation artifact, working gap, κ_k, η_k, K_n , normalized loss, pointwise reserve, claimed uniform reserve, transition status, target status, and complete Restart verdict;
- (xiv) every atomic status, aggregate verdict, and Type I–V route;
- (xv) every ClaimID, registered status, invocation role, theorem or policy location, discharged condition, and repair record;
- (xvi) the complete finite or countable clause registry and every justified scope exclusion;
- (xvii) provenance links sufficient to replay or independently reconstruct every computed verdict.

Definition 7.44 (Artifact closure). A canonical manifest has *artifact closure* when:

- (i) every referenced artifact has a content hash, type declaration, provenance, and immutable identifier;

- (ii) every transitive dependency needed to interpret or verify a consumed artifact is referenced or belongs to the registered theory release;
- (iii) no identifier is dangling, ambiguous, mutable under the same hash claim, or resolved differently by two fragments;
- (iv) the dependency graph contains no undeclared artifact replacement after the run identity was fixed.

Artifact closure is required for replayability but is not itself mathematical proof.

Proposition 7.45 (Canonical manifest conformance). *A chapter-local manifest fragment is audit-admissible only if it embeds into the canonical manifest without changing the meaning of a shared field, status, role, identifier, comparison mode, or version boundary. Incompatible assignments make the canonical manifest inconsistent.*

Proof. The canonical manifest describes one immutable run; incompatible shared assignments violate that condition. $\square$

Definition 7.46 (Replayability). A manifest is *replayable* if it has artifact closure and an independent verifier, using only the declared immutable artifacts and registered theory/schema versions, can reconstruct every deterministic profile, ledger, diagnostic, dependency, and verdict computation used by the run.

Remark 7.47 (Replayability is not mathematical proof by itself). Replayability establishes reproducibility of the declared computation and record graph. It does not validate a false theorem, insufficient hypothesis, or semantically inadequate artifact.

Proposition 7.48 (Manifest effect on the headline verdict). *A manifest with status *incomplete* makes the manifest clause *undefined* unless another evidenced clause already forces rejection. A manifest with status *inconsistent* forces rejection as a Type V governance failure. Only a *complete*, artifact-closed manifest can support a Core pass.*

Proof. Incomplete data leave required evidence unresolved; inconsistent data contradict the single-run semantics. $\square$

7.9 Failure routing and preservation of diagnostic detail

Definition 7.49 (Audit failure-routing record). Every non-pass run records all applicable, nonexclusive failure types:

- (i) **Type I:** nonzero repaired topological profile;
- (ii) **Type II:** nonzero eligible categorical obstruction;
- (iii) **Type III:** a failed or undefined metric, crop, overlap, exact-comparison, continuation, Restart, representative, realization, comparison-morphism, terminal-tameness, or other mathematical/operational obligation;
- (iv) **Type IV:** a valid, semantically authorized tower comparison artifact with $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0)$;
- (v) **Type V:** forbidden theorem use, incomplete or inconsistent canonical manifest, missing repair/governance record, suppressed missing-artifact marker, unsupported zero or pass declaration, misrepresented scope/version/hash/provenance, unauthorized scope exclusion, or proposer output treated as proof.

Mere absence or ill-typing of mathematical evidence first yields an undefined Type III clause. Type V is added when the governance record is itself incomplete, inconsistent, concealed, or semantically false.

Remark 7.50 (Headline compression does not erase details). The precedence of `reject` over `undefined` is only a headline rule. If one clause rejects and another is undefined, the manifest retains both states and both failure routes.

Proposition 7.51 (Repair creates a new verdict record). *Resolving an undefined clause or correcting a rejected clause produces a new run and manifest identity, optionally linked to the former run by an explicit repair relation. The original immutable RunID and historical verdict are not reused or overwritten.*

Proof. This is the immutable-scope rule of Definition 7.4. □

7.10 Core sovereignty and proposer–verifier separation

Definition 7.52 (Core sovereignty). Only Core-typed evidence evaluated under the registered verifier discipline may determine a Core verdict. No external proposal mechanism may overwrite, waive, or reinterpret an atomic Core clause. A human maintainer may revise theory, schema, or scope only by creating a new version or run identity.

Definition 7.53 (Proposer and verifier). A *proposer* is any human, algorithmic, Search-side, Platform-side, Companion-side, or AI mechanism that produces candidate objects, windows, thresholds, representatives, morphisms, claims, bridge programs, rankings, maps, or proof sketches. The *verifier* is the typed Core audit discipline of this chapter and the mathematical chapters it consumes.

Definition 7.54 (Re-entry protocol). A proposer output may influence a Core verdict only after:

- (i) normalization into a declared Core type;
- (ii) claim-status and invocation-role classification;
- (iii) production of required mathematical and artifact evidence;
- (iv) comparison-mode, ledger, and threshold-gap typing where applicable;
- (v) manifest registration, artifact closure, and immutable identity;
- (vi) execution of every relevant verifier clause.

Proposition 7.55 (Search and AI outputs do not update verdicts). *A Search score, potential, terrain map, ranking, validity-map value, AI confidence, proof sketch, or agent recommendation does not alter a Core verdict merely by being produced.*

Proof. Such an output is proposer-side until it completes the re-entry protocol and passes the verifier. □

Proposition 7.56 (Verifier independence). *Replacing the proposer mechanism does not change the mathematical meaning of a fixed verifier verdict computed from the same immutable Core inputs and registered theory versions.*

Proof. The verdict depends on registered inputs and clause semantics, not on the identity of the proposer. □

7.11 Audit package

Proposition 7.57 (Typed audit-and-sovereignty contract consequences). *The v19 Core audit discipline has the following consequences:*

- (i) *metric discrepancies are controlled by a declared ledger, scalarization, and actual upper-bound proof;*
- (ii) *non-scalar obligations cannot be bought off by numerical slack;*
- (iii) *metric reserve is gate-scoped and pre-threshold only in `metric_gap_certified` mode, while exact mode is governed by its exact proof artifact;*
- (iv) *every atomic clause has one typed status;*
- (v) *pass requires every invoked clause to pass and every exclusion to be justified;*
- (vi) *false evidence yields rejection, while unresolved evidence yields undefinedness unless another clause rejects;*
- (vii) *Type IV and complete Restart verdicts embed into the same status algebra;*
- (viii) *local, compatibility, Restart, atlas, and global verdicts remain distinct;*
- (ix) *only a complete, consistent, artifact-closed manifest can support pass;*
- (x) *failure routing preserves Types I–V and all clause-level detail;*
- (xi) *proposer outputs remain non-authoritative until typed re-entry;*
- (xii) *the theorem-bearing verifier remains sovereign.*

Proof. Items (i)–(iii) follow from Definitions 7.7, 7.8, 7.10, 7.12, and 7.27. Items (iv)–(vi) follow from Definitions 7.21 and 7.24. Items (vii)–(viii) follow from Definitions 7.23, 7.36, and 7.37, together with Proposition 7.38. Items (ix)–(x) follow from Definitions 7.42, 7.44, and 7.49. Items (xi)–(xii) follow from Definitions 7.52, 7.53, and 7.54. $\square$

Remark 7.58 (Contract consequence list, not a new external theorem). Proposition 7.57 is a closure list for the typed audit contracts of this chapter. It is not a new theorem about persistence modules, external interfaces, or a problem-specific domain.

7.12 Explicit non-claims

The following are not asserted in this chapter.

- (i) No unscalarized ledger element is directly compared with clear_τ .
- (ii) No scalarized budget repairs a missing representative, morphism, realization, claim status, or manifest field.
- (iii) No equality $\varepsilon_B = \text{gap}_\tau$ is a metric pass.
- (iv) No budget on one clause is reused elsewhere without an explicit joint-bound theorem.
- (v) No exact comparison is required to fabricate a threshold-gap reserve, and no exact equality or profile isomorphism is promoted to positive-radius robustness without a separate metric certificate.

- (vi) No `not_compared` mode discharges a comparison, overlap, Restart, or coherence obligation.
- (vii) No passing clause cancels a rejected or undefined clause.
- (viii) No `undefined` verdict is reported as pass or as a mathematical counterexample.
- (ix) No missing Type IV artifact is interpreted as $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$.
- (x) No missing κ -certificate is interpreted as $\kappa = 1$ or zero loss.
- (xi) No pointwise-positive Restart reserve is interpreted as a uniform positive lower bound.
- (xii) No finite prefix of a countable audit scope is treated as a complete pass certificate.
- (xiii) No pairwise overlap pass alone proves higher Čech coherence.
- (xiv) No coherent certificate atlas creates an unrestricted global object or global B-Gate⁺ pass.
- (xv) No zero terminal-generic Type IV diagnostic implies full comparison-map isomorphism on interior bars.
- (xvi) No use of $u_{i,W,\tau}$ or u_{Collapse} is normative; the cokernel diagnostics are $\nu_{i,W,\tau}$ and ν_{Collapse} .
- (xvii) No total μ/ν diagnostic is aggregated across different windows, thresholds, or infinitely many monitored degrees without a separately proved aggregation or summability theorem.
- (xviii) No deprecated global non-expansiveness claim for hard threshold deletion is used.
- (xix) No band of half-width 2ε is called sharp for the two-sided threshold theorem; the sharp half-width is ε .
- (xx) No Claim Register entry proves itself by registration.
- (xxi) No `operational`, `spec`, `deprecated`, `rejected`, or `unknown` item is theorem evidence; its admissible non-theorem role remains explicitly typed.
- (xxii) No Search, Platform, Companion, or AI artifact updates a Core verdict outside the re-entry protocol.
- (xxiii) No NBT statement is treated as a v19.0.0 theorem.
- (xxiv) No audit pass is a problem-specific external theorem without a separately proved Interface theorem.

7.13 Provenance and status

This chapter introduces no new barcode stability theorem, Ext computation, or tower invariant. Its substantive contribution is the canonical typed verifier and manifest architecture that separates exact proof from metric robustness, prevents numerical budgets from absorbing missing obligations, prevents undefined evidence from being reported as success, and preserves the distinction between local, overlap, Restart, atlas, and global verdicts.

Status labels for this chapter. *Core audit definitions:* Definitions [7.3](#), [7.4](#), [7.6](#), [7.7](#), [7.8](#), [7.10](#), [7.12](#), [7.14](#), [7.15](#), [7.16](#), [7.17](#), [7.18](#), [7.21](#), [7.23](#), [7.24](#), [7.27](#), [7.28](#), [7.30](#), [7.31](#), [7.32](#), [7.35](#), [7.36](#), [7.37](#), [7.42](#), [7.43](#), [7.44](#), [7.46](#), [7.49](#), [7.52](#), [7.53](#), and [7.54](#).

Logical consequences of the audit definitions: Propositions [7.11](#), [7.20](#), [7.25](#), [7.26](#), [7.33](#), [7.34](#), [7.38](#), [7.45](#), [7.48](#), [7.51](#), [7.55](#), [7.56](#), and [7.57](#).

Precision and boundary remarks: Remarks [7.1](#), [7.2](#), [7.5](#), [7.9](#), [7.13](#), [7.19](#), [7.22](#), [7.29](#), [7.39](#), [7.40](#), [7.41](#), [7.47](#), [7.50](#), and [7.58](#).

Explicit non-claims: Section [7.12](#).

8 Chapter 8: Core Closure and the AK Projection Boundary

8.1 Final role of the chapter

Chapter 8 contract. This chapter closes the final-audited main body of AK-HPDST v19.0.0. It adds no new external-domain theorem. It fixes the proved mathematical nucleus, the conditional theorem layer, the operational verifier layer, the rejected shortcuts, the canonical Interface boundary, and the only admissible route from an external object to an external conclusion.

Projection may supply a Core-readable input. It does not supply truth.

The governing inference rule is

The Core proves only what is established under declared types and hypotheses.
Projection supplies input; the verifier supplies a typed verdict; only a proved transport theorem supplies an external conclusion.

The final dependency order is

metric repair and gate contract	(Chs. 1–2; App. A)
→ evaluation and representative records	(Ch. 3)
→ eligible Ext-edge and one-way bridge	(Ch. 4)
→ failure taxonomy and terminal-generic Type IV	(Ch. 5)
→ overlap, κ -Restart, and record gluing	(Ch. 6)
→ typed audit and canonical manifest	(Ch. 7)
→ Core closure and Interface boundary.	(Ch. 8)

Remark 8.1 (Closure is not a new external theorem). This chapter introduces no new arithmetic, PDE, geometric, categorical, computational, or cryptographic result. Its strongest assertions are closure contracts and logical consequences of the mathematical results and typed audit rules already established. Its finality is architectural: it closes the permitted inference paths of the main body.

Remark 8.2 (Claim registration follows proof, not conversely). The v19 Claim Register records status and permitted use. Registration proves nothing and cannot increase claim strength. If a register entry conflicts with a theorem statement, proof, explicit hypothesis, or non-claim, the safer weaker reading governs until a new version repairs the inconsistency.

Remark 8.3 (Main-body closure and release-package closure are distinct). This chapter closes the theorem-bearing main body. The publication package is release-complete only after the v19 Claim Register and every imported appendix conform to this closure contract and the canonical Chapter 7 manifest. Governance artifacts record and verify closure; they do not create the mathematics being closed.

8.2 Canonical status vocabulary for the release

Definition 8.4 (Release-level claim classes). Every statement consumed by the release has exactly one registered class:

proved, conditional, operational, spec, deprecated, rejected, unknown.

Here proved permits theorem use under the registered hypotheses; conditional permits theorem use only after all registered conditions are discharged; operational governs typing, audit, manifest, or certification

rather than external mathematics; `spec` is a target or unproved transport statement; and the remaining classes carry no new theorem authority.

Definition 8.5 (Claim invocation role). Every consumed registered item also has exactly one invocation role:

`theorem_evidence`, `operational_rule`, `specification_target`, `provenance_only`.

Status and role are independent fields. Operational material may govern procedure, and specification material may state a target. Neither role promotes the item to theorem evidence.

Definition 8.6 (Safe-use order). A statement may be used only at or below its registered strength and only in a role compatible with its status. In particular:

- (i) a conditional item is theorem evidence only after every condition is discharged;
- (ii) an operational item is not an external mathematical theorem;
- (iii) a `spec` is a target, not proof;
- (iv) deprecated, rejected, and unknown items supply no new theorem evidence;
- (v) ambiguity is resolved by the weaker classification and weaker role.

Proposition 8.7 (Status cannot be promoted by prose). *A stronger verbal description, diagram, implementation success, numerical experiment, Search report, or AI-generated explanation does not promote either the registered class or invocation role of a statement.*

Proof. Status and role are versioned proof-governance data. Promotion requires a proof, the required artifacts, and an explicit versioned registration change; rhetoric and empirical success supply none of these by themselves. $\square$

8.3 The proved mathematical nucleus of v19.0.0

Definition 8.8 (Proved Core nucleus). The *proved Core nucleus* is the collection of mathematical theorem-level statements established in the final-audited Chapters 1–6 and Appendix A, together with their proved counterexamples and exact boundaries. Operational status calculi, manifests, schemas, and certification rules are not included merely because their logical consequences are proved; they belong to the operational layer below. A theorem with hypotheses belongs to the nucleus, while a run may instantiate it only after those hypotheses are discharged.

Proposition 8.9 (Contents of the proved Core nucleus). *The following mathematical statements belong to the proved Core nucleus at exactly the stated strength.*

- (i) *Constructible one-parameter persistence modules over the coefficient field admit the declared barcode decomposition in the working regime.*
- (ii) *The standard bottleneck metric uses endpoint sup-cost for finite bars, half-length diagonal cost, and finite matching of infinite bars only with infinite bars using birth-time displacement.*
- (iii) *Hard barcode threshold deletion deletes precisely finite bars of length at most τ , retains longer finite bars and infinite bars, and is endpoint-independent, idempotent, and threshold-monotone at barcode-profile level.*

- (iv) *Positive-threshold short bars do not form a Serre subcategory in general; hence no global exact positive-threshold Serre reflector, quotient formula, or abelian localization is available in the general Core.*
- (v) *Hard positive-threshold deletion is not globally non-expansive for the standard bottleneck metric.*
- (vi) *If $d_B(B, C) \leq \varepsilon$ and both pre-threshold barcodes avoid the closed band $[\tau - \varepsilon, \tau + \varepsilon]$, then*

$$d_B(T_\tau^{\text{bar}} B, T_\tau^{\text{bar}} C) \leq \varepsilon.$$

The half-width ε is sharp for this two-sided theorem.

- (vii) *Threshold clearance is measured after windowing and before deletion:*

$$\text{gap}_\tau(B) = \text{clear}_\tau(B) = \inf_{I \in B_{\text{fin}}} |\ell(I) - \tau|.$$

A numerical ledger may enter this inequality only through a scalarization proved to upper-bound the actual discrepancy used by that metric comparison.

- (viii) *The repaired evaluation order is*

$$B_{i,W}(F) = W_W \mathbf{P}_i(F), \quad \text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(B_{i,W}(F)).$$

- (ix) *Window restriction and hard deletion do not commute in general. Barcode-multiset commutation holds exactly under the all-retained-bars criterion, including infinite bars clipped by bounded windows.*
- (x) *Admissible representatives of the same repaired profile give the same persistence-level vanishing verdict. Derived or Ext-level invariance requires separately declared realization-equivalence data.*
- (xi) *For $A \in D^{[-1,0]}(k\text{-mod})$, the canonical truncation triangle gives the natural isomorphism*

$$\text{Ext}^1(A, k) \cong \text{Hom}_k(H^{-1}(A), k).$$

- (xii) *Under a complete eligible realization and edge-realization interface,*

$$\text{Eval}_{1,W,\tau}(F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

The converse is false under the present axioms without an additional zero-reflection or joint-faithfulness theorem.

- (xiii) *For terminal-tame profiles, tgdim_W is a finite integer counting bars cofinal in the declared window. Applied to an actual comparison morphism, it defines*

$$\mu_{i,W,\tau} = \text{tgdim}_W(\ker \phi_{i,W,\tau}), \quad \nu_{i,W,\tau} = \text{tgdim}_W(\text{coker } \phi_{i,W,\tau}).$$

For fixed (W, τ) and finite I_{mon} , the aggregate diagnostics are the corresponding finite sums. Cross-window, cross-threshold, or infinite-degree aggregation requires a separate theorem.

- (xiv) *$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ excludes monitored terminal-generic kernel/cokernel defects only. It does not imply full comparison-map isomorphism unless a separate interior-defect theorem proves the entire defect profiles vanish.*

(xv) In *metric_gap_certified* overlap mode, the sharp two-sided theorem transfers the evidenced pre-threshold bound to direct common-window post-threshold profiles. In *exact* mode, the declared equality or profile isomorphism is used directly and supplies no positive-radius robustness by itself.

(xvi) *Certified recurrences*

$$r_{k+1} \geq \kappa_k r_k - \eta_k$$

imply the product-weighted finite-horizon bound and the normalized reserve criterion. Reserve-safe countable Restart gives $r_n > 0$ at every finite stage; a uniform positive lower bound additionally needs control such as $\inf_n \prod_{j < n} \kappa_j > 0$.

Proof. Each item is a restatement at its final-audited strength of a theorem, proposition, corollary, or explicit counterexample in Chapters 1–6 and Appendix A. Definition-only governance statements are deliberately not promoted into this mathematical list. $\square$

Remark 8.10 (The nucleus is small by design). The proved nucleus is intentionally narrower than the surrounding architecture. Its strength comes from sharp hypotheses, typed inputs, explicit counterexamples to stronger claims, and refusal to turn missing evidence into success.

8.4 Conditional mathematics and operational contracts

Definition 8.11 (Conditional Core layer). The *conditional Core layer* consists of proved implications whose instances require separately supplied hypotheses or artifacts. It includes:

- (i) crop–collapse commutation after verification of the exact all-retained-bars criterion;
- (ii) τ -split functorial deletion and exact-admissible regimes only on their declared subdomains;
- (iii) filtered or derived representative invariance beyond the profile level;
- (iv) eligible realization, amplitude, and edge-realization interfaces;
- (v) tower-profile systems, source semantics, actual comparison morphisms, terminal tameness, and defect profiles;
- (vi) mode-correct overlap records, higher simplex compatibility, and object-level descent only when the appropriate theorem is separately proved;
- (vii) κ -admissible continuation artifacts, finite working gaps, passing targets, and normalized reserve-safety;
- (viii) problem-specific Interface and transport theorems.

Definition 8.12 (Operational Core layer). The *operational Core layer* contains the binding contracts that determine what counts as an auditable proof object or verdict. It includes:

- (i) fixed evaluation order and repair-as-new-run discipline;
- (ii) comparison modes

exact, metric_gap_certified, not_compared;

- (iii) gate-scoped ledger composition, bottleneck scalarization, and non-scalar obligations;

- (iv) representative, eligibility, tower, overlap, simplex, continuation, and Restart records;
- (v) Type I–V failure routing;
- (vi) atomic statuses
 - pass, reject, undefined, not_invoked;
- (vii) typed Type IV, Restart, Interface, and manifest statuses;
- (viii) manifest status complete, incomplete, or inconsistent;
- (ix) immutable run identity, complete finite or countable clause registry, content-addressed artifact closure, replayability, and repair ancestry;
- (x) claim class together with invocation role;
- (xi) Core sovereignty and proposer–verifier separation.

Definition 8.13 (Canonical comparison mode). Every consumed profile comparison has exactly one mode:

- (i) `exact`: an actual equality or profile isomorphism and its proof artifacts;
- (ii) `metric_gap_certified`: an evidenced bottleneck bound and a two-sided gap record at the same real budget;
- (iii) `not_compared`: no comparison inference belongs to the declared scope.

The third mode cannot discharge an overlap, transport, or coherence clause that requires comparison.

Proposition 8.14 (Conditional and operational layers do not self-promote). *A conditional statement without discharged hypotheses is not an available theorem instance, and an operational contract is not an external mathematical theorem.*

Proof. The first claim is ordinary implication logic. The second follows from the role of operational contracts: they type and verify evidence but do not supply source-domain semantics. $\square$

Remark 8.15 (Canonical manifest location). The Chapter 1 base manifest and the Chapters 3–6 extensions are conforming fragments of the canonical Chapter 7 schema. An incomplete manifest yields `undefined` unless another evidenced clause already rejects; an inconsistent manifest yields Type V `reject`.

8.5 Deprecated and rejected claims closed by the repair

Definition 8.16 (Metric-repair and architecture rejection set). The following are unavailable for v19 theorem use:

- (i) global non-expansiveness of hard positive-threshold deletion;
- (ii) a general positive-threshold Serre subcategory, exact reflector, quotient formula, or abelian localization;
- (iii) $\text{gap}_\tau = \frac{1}{2} \text{clear}_\tau$;
- (iv) the claim that $[\tau - 2\varepsilon, \tau + 2\varepsilon]$ is the sharp two-sided band;
- (v) unconditional crop–collapse commutation or a finite-only adaptedness test;

- (vi) an automatic global strict filtered collapse functor C_τ ;
- (vii) automatic tower or colimit-to-apex comparison morphisms after threshold deletion;
- (viii) the reverse bridge $\text{Ext}^1 = 0 \Rightarrow \text{Eval}_{1,W,\tau} = 0$ under the current axioms;
- (ix) interpreting $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ as full comparison isomorphism;
- (x) representing a no-tower claim by fabricated zero diagnostics rather than `not_invoked`;
- (xi) a universal Restart constant inferred from adjacency or hard deletion;
- (xii) certified continuation law implying a complete Restart pass without positive target reserve and a passing target record;
- (xiii) ordinary unweighted summability as sufficient under contracting κ_k ;
- (xiv) pointwise-positive reserves implying a uniform positive lower bound;
- (xv) finite-prefix verification implying a countable pass;
- (xvi) forcing an exact comparison to fabricate a metric ledger or threshold reserve;
- (xvii) allowing `not_compared` to discharge a comparison-dependent claim;
- (xviii) normative use of u_{Collapse} in place of ν_{Collapse} ;
- (xix) v19.0.0 theorem status for NBT.

Proposition 8.17 (Rejected claims cannot be repaired locally by a ledger). *No rejected structural claim in Definition 8.16 becomes valid by assigning it a larger numerical error budget.*

Proof. A numerical ledger bounds only a declared metric discrepancy after scalarization and an upper-bound proof. The rejected items concern false global assertions, absent algebraic structure or morphisms, wrong constants, unsupported scope completion, or missing logical hypotheses. $\square$

8.6 The final Core closure contract

Definition 8.18 (v19 Core closure contract). The v19 Core is the smallest versioned theorem-and-verifier package containing:

- (i) the proved mathematical nucleus of Definition 8.8;
- (ii) the conditional layer only at instantiated hypotheses;
- (iii) the operational layer governing evidence, finite or countable scope, and verdicts;
- (iv) final Appendix A as the self-contained proof source for the persistence metric repair;
- (v) the canonical Chapter 7 manifest and immutable run identity;
- (vi) explicitly imported micro-theorems only at registered strength, role, and hypotheses;
- (vii) no external-domain semantics without a separately proved transport theorem.

The package is closed under valid mathematical consequence and versioned proof-preserving reuse. It is not enlarged by suggestion, analogy, implementation, registration, projection, or numerical success alone.

Proposition 8.19 (Main-body Core closure). *Within the declared v19.0.0 scope, every Core verdict is determined by the final-audited mathematical inputs, typed artifacts, comparison modes, any applicable gate-scoped metric ledgers, the canonical manifest, and the Chapter 7 audit rules. No proposer-side object changes that verdict without completing the Core re-entry protocol.*

Proof. The mathematical inputs are closed by Proposition 8.9 and Definition 8.11; the verdict calculus and canonical manifest are fixed by Chapter 7. Core sovereignty excludes direct updates by proposer-side objects. $\square$

Proposition 8.20 (Final-audit closure criterion). *A v19.0.0 main-body run is audit-closed only if:*

- (i) *every consumed item is used at a compatible registered class and invocation role;*
- (ii) *every conditional hypothesis and non-scalar obligation is discharged;*
- (iii) *each required comparison uses `exact` or `metric_gap_certified` mode and passes;*
- (iv) *only metric-mode clauses require a finite scalarized budget, an actual discrepancy upper-bound proof, and positive threshold reserve;*
- (v) *every required finite or countable clause is registered and typed;*
- (vi) *the canonical manifest is complete and consistent;*
- (vii) *the aggregate Chapter 7 verdict is determined.*

A fully evidenced rejection is a closed audit result. A run with unresolved required evidence remains `undefined` and is not audit-closed as a determined mathematical verdict.

Proof. The clauses are exactly the status/role, conditional, comparison-mode, metric, registry, manifest, and aggregate verdict conditions of Chapters 2–7. Under them every required atomic clause is defined as pass or reject. $\square$

Remark 8.21 (This is a closure contract, not a universal completeness theorem). Main-body closure does not claim that the Core decides every mathematical question or represents every valid external proof. It states only that every verdict claimed by this release must arise through the declared package.

8.7 High-dimensional projection as an Interface mechanism

The repaired AK intuition is

$$\text{external object} \mapsto \text{Core-readable shadow} \mapsto \text{audited statement about that shadow}.$$

The first arrow is Interface-side and the second verifier-side. Neither arrow alone proves a theorem in the source domain.

Definition 8.22 (External source class). An *external source class* is a declared category or typed family outside the minimal one-parameter persistence Core. An external object is not a Core object by default.

Definition 8.23 (AK projection datum). An *AK projection datum* consists of:

- (i) a source class $\mathcal{D}$ and object $X \in \mathcal{D}$;
- (ii) a typed projection, extraction, representation, or realization route

$$\mathfrak{R}_{AK} : \mathcal{D} \longrightarrow \text{FiltCh}((\cdot)k) \quad \text{or} \quad \mathfrak{R}_{AK} : \mathcal{D} \longrightarrow \text{Pers}_k^{\text{cons}};$$

- (iii) the projected object $F_X = \mathfrak{R}_{AK}(X)$;
- (iv) windows, thresholds, monitored degrees, coefficient field, and evaluation order;
- (v) constructibility evidence and a precisely stated purpose-faithfulness claim;
- (vi) all approximation, measurement, crop, representative, realization, and comparison obligations;
- (vii) the comparison mode of every consumed comparison;
- (viii) in metric mode, a gate-scoped ledger, scalarization, actual discrepancy upper-bound proof, and positive reserve; in exact mode, the actual equality or profile-isomorphism proof and exact provenance;
- (ix) all non-scalar obligations, claim classes, invocation roles, directions, and non-claim boundaries;
- (x) immutable artifact and manifest references sufficient for reconstruction.

The datum is Interface-level unless its asserted properties are separately proved.

Definition 8.24 (Core-readable projected object). A projected object F_X is *Core-readable* on (i, W, τ) when:

- (i) $B_{i,W}(F_X) = W_W \mathbf{P}_i(F_X)$ is a declared constructible pre-threshold profile;
- (ii) $\text{Eval}_{i,W,\tau}(F_X) = T_\tau^{\text{bar}}(B_{i,W}(F_X))$ is defined;
- (iii) every invoked comparison has a valid comparison mode and evidence;
- (iv) every representative, realization, Type IV, overlap, Restart, claim, and manifest obligation in scope is typed and supplied;
- (v) every countable scope has a complete audit-readable clause registry;
- (vi) the canonical manifest is complete and consistent;
- (vii) the Chapter 7 verifier can evaluate the run.

Core-readability means auditability of the projected data, not truth of any external interpretation.

Proposition 8.25 (Projection supplies input, not verdict). *An AK projection datum determines neither a Core verdict nor an external-domain theorem by itself.*

Proof. A Core verdict additionally requires the applicable mathematical clauses, artifacts, comparison evidence, manifest, and typed audit aggregation. An external theorem additionally requires a proved transport statement back to the source semantics. $\square$

Remark 8.26 (Projection is structural). Projection need not be linear. It may be geometric, arithmetic, analytic, categorical, spectral, computational, tower-theoretic, or hybrid. Its admissibility depends on source/-target typing and proof, not on conceptual elegance.

8.8 Purpose-faithfulness and Interface entry

Definition 8.27 (Purpose-faithful realization). A route is *purpose-faithful* for a declared Interface claim only when the claim identifies the external feature, the Core-visible feature intended to represent it, the direction of use, and the theorem or controlled mismatch hypotheses connecting them. Purpose-faithfulness is relative to that use and is not categorical faithfulness unless separately proved.

Definition 8.28 (Interface entry record). An *Interface entry record* contains at least:

- (i) source object, source semantics, projection datum, and declared purpose;
- (ii) Core-readable target type, constructibility, filtration orientation, windows, thresholds, and degrees;
- (iii) the fixed window-before-threshold evaluation order;
- (iv) a comparison mode for every consumed comparison;
- (v) exact comparison artifacts or metric gap/budget artifacts according to mode;
- (vi) representative and eligible realization data when invoked;
- (vii) actual tower comparison morphisms, source semantics, terminal tameness, and fixed $(W, \tau, I_{\text{mon}})$ when Type IV is invoked;
- (viii) overlap, simplex, or κ -continuation artifacts when invoked;
- (ix) the complete finite or countable clause registry;
- (x) the canonical manifest, immutable run identity, and transitive artifact closure;
- (xi) registered class, invocation role, direction, and hypotheses of every transport statement.

Proposition 8.29 (No admissible Interface without typed entry). *An external object has no theorem-bearing access to the Core until its Interface entry record is complete and consistent and every invoked entry clause is typed.*

Proof. Without a complete typed entry, the verifier cannot reconstruct the Core input. Missing or unresolved entry evidence yields `undefined`; mutually inconsistent or evidenced false entry claims yield `reject`. Neither yields `pass`. $\square$

Remark 8.30 (Interface failure is not automatically Core rejection). A proposed route may be nonconstructible, not purpose-faithful, insufficiently evidenced, or ill-typed. This is an Interface failure or undefined entry. It is a Core rejection only when a properly invoked Core clause is well-typed and evidenced false, or the canonical manifest is inconsistent.

8.9 The repaired evaluation path for external data

Definition 8.31 (Interface evaluation path). Every Core-relevant Interface run uses

$$\begin{aligned}
 X &\longmapsto F_X = \mathfrak{R}_{AK}(X) \\
 &\longmapsto \mathbf{P}_i(F_X) \\
 &\longmapsto B_{i,W}(F_X) = W_W \mathbf{P}_i(F_X) \\
 &\longmapsto \text{Eval}_{i,W,\tau}(F_X) = T_\tau^{\text{bar}}(B_{i,W}(F_X)) \\
 &\longmapsto \text{typed Core clauses and audit verdict.}
 \end{aligned}$$

No raw-domain observable replaces $B_{i,W}$ or $\text{Eval}_{i,W,\tau}$ without a proved typed identification.

Proposition 8.32 (External origin does not relax Core hypotheses). *A projected object is subject to the same constructibility, comparison-mode, representative, eligibility, Type IV, overlap, Restart, claim-role, and manifest conditions as an internally specified Core object.*

Proof. Core theorems and verifier clauses are typed by inputs and hypotheses, not by data origin. $\square$

Remark 8.33 (Approximation must be visible). Sampling, truncation, discretization, smoothing, numerical solution, normalization, compression, and learned representation must be explicit. Their metric effects are ledgered only when scalarizable and used in metric mode; missing semantic, morphism, or exact-comparison obligations remain non-scalarizable.

8.10 Four logically distinct outcomes

Definition 8.34 (Interface outcome). The *Interface outcome* states whether the source was converted into a complete, consistent, Core-readable entry. Its statuses are *pass*, *reject*, and *undefined* under the declared Interface contract.

Definition 8.35 (Core outcome for projected data). The *Core outcome for projected data* is the Chapter 7 verdict for the Core-readable record of F_X . It is a statement about the repaired projected data under the declared hypotheses.

Definition 8.36 (Bridge outcome). A *Bridge outcome* is the registered status of a proposed implication between source semantics and a Core-visible statement. It may be a specification target, candidate, program, toy theorem, conditional Interface theorem, or proved transport theorem; these statuses are not interchangeable.

Definition 8.37 (External-domain outcome). An *external-domain outcome* is a theorem or refutation in the original semantics of X .

Proposition 8.38 (No outcome-layer collapse). *None of*

$$\begin{aligned} \text{Interface pass} &\Rightarrow \text{Core pass}, \\ \text{Core pass} &\Rightarrow \text{external theorem}, \\ \text{toy bridge theorem} &\Rightarrow \text{external theorem} \end{aligned}$$

is valid without the separate hypotheses and transport results connecting the corresponding layers.

Proof. The antecedents certify different source and target types. Each implication needs mathematical content not contained in its antecedent alone. $\square$

8.11 Transport theorems and Bridge promotion

Definition 8.39 (Interface-to-Core transport statement). An *Interface-to-Core transport statement* has the direction

$$\text{external-domain structure} \Longrightarrow \text{specified Core-visible behavior}.$$

Definition 8.40 (Core-to-Interface transport statement). A *Core-to-Interface transport statement* has the direction

$$\text{specified Core-visible behavior} \Longrightarrow \text{external-domain conclusion}.$$

Definition 8.41 (Transport theorem). A *transport theorem* is a proved directional statement containing:

- (i) precise source and target semantics;
- (ii) a purpose-faithful projection or realization theorem;
- (iii) a Core-readable target and constructibility proof;
- (iv) all representative, realization, tower, overlap, and Restart artifacts used;
- (v) the comparison mode of every comparison;
- (vi) in metric mode, every scalarizable transfer defect, its scalarization, actual discrepancy upper-bound proof, and reserve; in exact mode, the exact proof artifacts;
- (vii) all non-scalar obligations and a complete canonical manifest;
- (viii) a proof of the stated implication direction;
- (ix) an explicit statement whether the converse is claimed;
- (x) a non-claim boundary.

Definition 8.42 (Default status of transport statements). Every problem-specific transport statement is `spec` with role `specification_target` by default. It becomes theorem evidence only through an explicit proof, artifact-complete verifier entry, and versioned promotion. The two transport directions are independent.

Proposition 8.43 (No silent Bridge promotion). *A natural-looking projection, successful calibration, stable numerical experiment, Core pass, toy bridge theorem, Search result, or AI-generated proof sketch does not silently promote a transport specification to a theorem.*

Proof. None of these objects by itself supplies all requirements of Definition 8.41. □

Remark 8.44 (External Ext is not automatically the Core Ext). An arithmetic, sheaf-theoretic, deformation-theoretic, or categorical Ext-group is not automatically $\text{Ext}^1(\mathcal{R}(C_{\tau,WF}), k)$. Any identification is itself Interface mathematics and must be typed, directional, and proved.

8.12 Micro-theorems, toy bridges, Search, and Platform material

Definition 8.45 (Permitted adjacent support). Appendices and systems may provide micro-theorems under explicit hypotheses, toy bridge theorems in restricted sources, Interface definitions and transport theorems, Search proposals, Platform logs and proof artifacts, or Companion templates. Such material supports the main body only at its registered class and invocation role.

Proposition 8.46 (Adjacent material does not enlarge the Core silently). *No appendix, Companion, Search report, Platform artifact, micro-theorem, toy theorem, or Interface schema enlarges the v19 Core merely by appearing in the project.*

Proof. Import requires a compatible theorem statement, registered status and role, discharged hypotheses, and all required verifier evidence. □

Remark 8.47 (NBT release boundary). NBT remains a v19.1 candidate program. Its possible control of apex-only Type IV cokernel modes is a future theorem question; no NBT assertion strengthens v19.0.0.

8.13 Explicit non-claims of the closed main body

The closed v19.0.0 main body asserts none of the following.

- (i) Global non-expansiveness of hard threshold deletion.
- (ii) Positive-threshold Serre localization, a global exact reflector, or universal quotient formula.
- (iii) Sharpness of a band of half-width 2ε ; the proved sharp half-width is ε .
- (iv) $\text{gap}_\tau = \frac{1}{2} \text{clear}_\tau$; the alias is $\text{gap}_\tau = \text{clear}_\tau$.
- (v) Unconditional crop-collapse commutation or a finite-only adaptedness criterion.
- (vi) A global strict filtered collapse functor C_τ .
- (vii) Automatic derived or Ext equivalence of profile-level representatives.
- (viii) $\text{Ext}^1 = 0 \Rightarrow \text{PH}_1 = 0$ under the current axioms, or a global $\text{PH}_1 \Leftrightarrow \text{Ext}^1$.
- (ix) Replacement of the repaired profile by the detector E_1 , or use of E_1 as a metric budget.
- (x) An automatic tower comparison morphism from barcodes or bottleneck matchings.
- (xi) Missing Type IV evidence interpreted as $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$, or no-tower scope represented by zero numbers.
- (xii) $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ interpreted as full defect-profile vanishing.
- (xiii) Normative cokernel notation other than ν_{Collapse} .
- (xiv) Aggregation of Type IV totals across different (W, τ) or infinitely many degrees without a separate aggregation or summability theorem.
- (xv) A universal $\kappa > 0$, default $\kappa = 1$, or Restart without a certified continuation artifact.
- (xvi) A complete Restart pass from the recurrence certificate alone.
- (xvii) Ordinary summability as sufficient under contraction, or pointwise positivity as a uniform bound.
- (xviii) Pairwise overlap as higher Čech coherence, or summability as a substitute for face maps.
- (xix) A coherent certificate atlas as an unrestricted global object.
- (xx) Finite-prefix verification as a countable pass.
- (xxi) A scalarization value as a certificate without an actual metric upper-bound proof.
- (xxii) A numerical budget repairing a missing morphism, representative, realization, eligibility, coherence map, status, role, or manifest field.
- (xxiii) An exact comparison required to fabricate a positive threshold reserve.
- (xxiv) `not_compared` used to discharge a comparison-dependent clause.
- (xxv) `undefined` identified with `reject`, `pass`, `zero`, or `not_invoked`.

- (xxvi) Theorem status created merely by Claim Register registration.
- (xxvii) A proposer-side score, potential, validity map, AI confidence, or Search report overwriting a verdict.
- (xxviii) A Core pass by itself proving an external theorem.
- (xxix) An external theorem from a natural projection, micro-theorem, or toy theorem without proved transport.
- (xxx) v19.0.0 theorem status for NBT.
- (xxxi) Completeness of AK theory for all mathematical problems or all realizations.

Remark 8.48 (Non-claims are part of the result). These exclusions identify the exact boundary at which another theorem, artifact, or versioned repair is required. The repaired theory is defined both by its proved implications and by the invalid shortcuts it forbids.

8.14 Final architecture theorem

Proposition 8.49 (Final architecture of the v19 main body). *There are two distinct routes.*
A candidate route may begin with a specified Interface:

$$X \xrightarrow{\text{specified Interface}} F_X,$$

but this produces only a candidate Core input until the Interface entry passes.

A theorem-bearing route has the form

$$\begin{aligned}
X &\xrightarrow{\text{proved, typed Interface entry}} F_X = \mathfrak{R}_{AK}(X) \\
&\xrightarrow{\text{windowed monitored persistence}} B_{i,W}(F_X) \\
&\xrightarrow{\text{hard threshold deletion}} \text{Eval}_{i,W,\tau}(F_X) \\
&\xrightarrow{\text{typed clauses, comparison evidence, artifacts, manifest}} \text{Core verdict} \\
&\xrightarrow{\text{separately proved directional transport theorem}} \text{external-domain conclusion.}
\end{aligned}$$

Every arrow has declared source and target types, direction, hypotheses, class, invocation role, artifacts, and failure boundary. No arrow may be omitted, reversed, or merged by convention.

Proof. The first arrow is governed by the projection, purpose-faithfulness, and Interface-entry definitions. The next two arrows are the repaired evaluation discipline. The Core verdict is governed by Chapters 4–7. The final arrow is precisely the extra content of a transport theorem. A merely specified Interface lacks the proof needed for the theorem-bearing first arrow. $\square$

Remark 8.50 (Final AK projection principle).

High-dimensional projection may reveal a Core-readable shadow.
The repaired Core may certify the collapse behavior of that shadow.
Only a separately proved transport theorem may return that certification to the original domain.

This deliberate asymmetry preserves the ambition of the AK program while preventing projection, collapse, computation, or Search from being mistaken for an external proof.

Remark 8.51 (Closure of the main body). With this chapter, the final-audited v19.0.0 main body is closed. The mathematical nucleus is sharp where proved, conditional where additional structure is required, operational where verifier rules are specified, and silent where transport is absent. Future versions enlarge the package only through new proofs, explicit artifacts, canonical manifest updates, and versioned claim promotion.

8.15 Provenance and status

This chapter consumes final-audited Chapters 1–7 and Appendix A. It records the main-body boundary and does not replace detailed hypotheses or proofs. The v19 Claim Register must register every item at the same or a weaker class and compatible invocation role.

Release-level and closure definitions. Definitions [8.4](#), [8.5](#), [8.6](#), [8.8](#), [8.11](#), [8.12](#), [8.13](#), [8.16](#), and [8.18](#).

Interface definitions. Definitions [8.22](#), [8.23](#), [8.24](#), [8.27](#), [8.28](#), [8.31](#), [8.34](#), [8.35](#), [8.36](#), [8.37](#), [8.39](#), [8.40](#), [8.41](#), [8.42](#), and [8.45](#).

Logical consequences and closure propositions. Propositions [8.7](#), [8.9](#), [8.14](#), [8.17](#), [8.19](#), [8.20](#), [8.25](#), [8.29](#), [8.32](#), [8.38](#), [8.43](#), [8.46](#), and [8.49](#).

Boundary and final-closure remarks. Remarks [8.1](#), [8.2](#), [8.3](#), [8.10](#), [8.15](#), [8.21](#), [8.26](#), [8.30](#), [8.33](#), [8.44](#), [8.47](#), [8.48](#), [8.50](#), and [8.51](#).

Explicit non-claims. Section [8.13](#).

Main-body closure. Propositions [8.19](#), [8.20](#), and [8.49](#).